

Blockchain + Explainable AI

A Research-Grade Self-Study Course

A research-grade self-study course synthesizing top web courses on Blockchain and Explainable AI.

Author: Venkat Rayudu Alapati **Date:** 2026-05-23 **Pages:** see Table of Contents

Blockchain + Explainable AI — Research-Grade Self-Study Course

A complete, research-oriented syllabus and notes covering Blockchain and Explainable AI (XAI), designed for a Master's / early-PhD student aiming to publish at top venues (NeurIPS, ICML, IEEE S&P, CCS, USENIX Security, FAccT, AAAI, IJCAI, IEEE Blockchain).

The course is distilled from the strongest publicly available curricula:

Blockchain sources synthesized

- Princeton — *Bitcoin and Cryptocurrency Technologies* (Narayanan, Bonneau, Felten, Miller, Goldfeder)
- MIT 15.S08 — *Blockchain and Money* (Gary Gensler)
- Berkeley CS294-164 — *Blockchain Technology*
- Stanford CS251 — *Cryptocurrencies and Blockchain Technologies* (Dan Boneh)
- Coursera — *Blockchain Specialization* (University at Buffalo)
- ConsenSys Academy / Ethereum Foundation EthCore curriculum
- a16z Crypto Startup School
- Zero Knowledge Proof MOOC (Boneh, Goldberg, Wahby)

XAI sources synthesized

- Christoph Molnar — *Interpretable Machine Learning* (the canonical reference)
- Kaggle — *Machine Learning Explainability* micro-course
- DeepLearning.AI — *AI Explainability* short courses
- DARPA XAI Program reports (2017–2021)
- Harvard CS282BR — *Topics in Interpretability*
- Stanford CS329T — *Trustworthy Machine Learning*
- CMU 10-718 — *Machine Learning in Practice* (interpretability modules)
- Anthropic / OpenAI / DeepMind mechanistic interpretability work
- Distill.pub Circuits Thread

How to use this course

1. **Read the master [SYLLABUS](#)** — 18-week dual-track plan.
2. **Two parallel tracks:** 12 Blockchain modules, 13 XAI modules, plus a converged "Verifiable AI / ZKML" module.
3. **Per-module structure:** theory notes → key papers → guided lab → research questions.
4. **End every week** with a written research-style summary (1–2 pages) — this is how you build publishable intuitions.

Directory layout

```
blockchain_xai_course/
├─ README.md           # This file
├─ SYLLABUS.md         # 18-week structured plan
├─ blockchain/         # 12 modules – foundations to research frontier
├─ xai/               # 13 modules – feature attribution to mech-interp
├─ intersection/       # ZKML, on-chain ML, verifiable AI
├─ resources/          # Papers, books, courses, tools
└─ labs/              # Hands-on coding labs
```

Suggested cadence

- **15 hrs/week:** 8 hrs blockchain, 6 hrs XAI, 1 hr writing summary.
- **Full immersion:** complete in 12 weeks at 30 hrs/week (matches a focused research sprint).
- **Casual:** 6 months at 6 hrs/week.

Research output target

By the end you should be able to:

- Implement a working consensus protocol, a smart contract system, and a basic zk circuit.
- Reproduce LIME, SHAP, Integrated Gradients, GradCAM, TCAV, and a counterfactual generator from scratch.
- Identify a tractable open problem in **Verifiable / Explainable / Decentralized ML** and write a 6-page workshop paper draft.

Date created: 2026-05-23. Maintained as a living document.

Master Syllabus — Blockchain + Explainable AI (18 Weeks)

Learning objectives

By the end of this course you will:

1. **Theory** — Explain consensus, cryptographic commitments, smart contracts, attribution methods, and mechanistic interpretability rigorously.
 2. **Implementation** — Build small versions of each system: a toy blockchain, a smart contract dApp, a SHAP/LIME implementation, a saliency pipeline, a zk-SNARK circuit.
 3. **Research literacy** — Read papers from S&P, CCS, USENIX, NeurIPS, ICML, FAccT, and write critique notes.
 4. **Original contribution** — Propose a publishable workshop-paper-scale research problem at the intersection of Verifiable AI and Explainability.
-

Week-by-week plan

Wk	Blockchain (8 hrs)	XAI (6 hrs)	Writing (1 hr)
1	Module 1: Cryptographic Foundations	Module 1: What is Explainability?	Define your "why"
2	Module 2: Distributed Systems & CAP	Module 2: Taxonomy of XAI	Read Molnar Ch 1–3 summary
3	Module 3: Bitcoin Deep Dive	Module 3: LIME & SHAP	Reproduce Molnar's penguin example
4	Module 4: Ethereum & Smart Contracts	Module 4: Gradient-based Attribution	Side-by-side LIME vs SHAP
5	Module 5: Consensus Algorithms	Module 5: Concept-based (TCAV, ACE)	Consensus comparison table
6	Module 6: Scalability & L2	Module 6: Counterfactual Explanations	Compare rollup designs
7	Module 7: Privacy & ZKPs	Module 7: Global Methods (PDP/ALE/Surrogate)	First ZK circuit + writeup
8	Module 8: DeFi Architecture	Module 8: Attention & Transformer Interp	DeFi attack postmortem
9	Module 9: Security & Attack Surface	Module 9: Mechanistic Interpretability	One attack reproduced
10	Module 10: Governance & Tokenomics	Module 10: Evaluating Explanations	Faithfulness benchmark

11	Module 11: Advanced Consensus (DAG/HotStuff/Sharding)	Module 11: Human-Centered XAI	User study design
12	Module 12: Research Frontiers	Module 12: XAI in Domains (Med/Finance/Law)	Domain critique
13	—	Module 13: XAI Research Frontiers	Literature gap doc
14	Intersection: Verifiable & Decentralized ML / ZKML	(continue)	Joint reading list
15	Lab week — implement toy ZKML inference	Lab week — implement SHAP from scratch	Lab journal
16	Research proposal draft (3 pages)	Research proposal draft (3 pages)	Merge into one
17	Polish, related work, identify venue	Polish, related work, identify venue	6-page workshop draft
18	Get feedback, iterate, submit to arXiv	—	Submission

Assessment artifacts (you produce these)

1. **Weekly summaries** (18 × 1–2 pages)
2. **12 module quizzes** — see end of each module file
3. **6 implementation labs** — see `labs/`
4. **1 critical literature review** (~10 pages)
5. **1 research proposal** (6-page workshop format)
6. **1 reproducibility report** for an existing paper (any XAI or blockchain paper)

Prerequisites

- Linear algebra, probability, basic statistics
- Python (PyTorch helpful)
- Familiarity with at least one CS theory class (algorithms / complexity)
- For ZK section: comfort with abstract algebra (groups, fields) helps but is not strictly required

Reading shelf (highly recommended)

Title	Why
<i>Bitcoin and Cryptocurrency Technologies</i> — Narayanan et al. (Princeton)	Best free blockchain textbook
<i>Mastering Bitcoin</i> — Andreas Antonopoulos	Practical reference
<i>Mastering Ethereum</i> — Antonopoulos & Wood	Practical Ethereum

<i>Interpretable Machine Learning</i> — Christoph Molnar (free)	XAI canon
<i>Explanatory Model Analysis</i> — Biecek & Burzykowski	Tabular XAI
<i>The Mythical Man-Month</i> (yes, really)	Distributed systems intuition
<i>Anthropic Transformer Circuits Thread</i> (online)	Modern mech-interp
Distill.pub — Building Blocks of Interpretability	Visual intuition

See [resources/papers.md](#), [resources/books.md](#), [resources/courses.md](#), [resources/tools.md](#) for the full curated lists.

--	--	--	--

--	--

Blockchain Track — Overview

A 12-module sequence from cryptographic primitives to research-grade frontiers.

Modules

1. [Cryptographic Foundations](#) — hashes, Merkle trees, signatures, commitments
2. [Distributed Systems & CAP/FLP](#)
3. [Bitcoin Deep Dive](#)
4. [Ethereum & Smart Contracts](#)
5. [Consensus Algorithms](#) — PoW, PoS, BFT, VRF families
6. [Scalability & Layer 2](#) — rollups, channels, DA
7. [Privacy & Zero-Knowledge Proofs](#)
8. [DeFi Architecture](#)
9. [Security & Attack Surface](#)
10. [Governance & Tokenomics](#)
11. [Advanced Consensus \(DAG, HotStuff, Sharding\)](#)
12. [Research Frontiers \(2025–2027\)](#)

How to study this track

- Read modules in order. Each builds on the previous.
- Do the lab at the end of each module before moving on.
- After every 3 modules, write a 1-page comparison of what you learned (cross-module synthesis).

What you will be able to do after

- Read a CRYPTO / S&P / OSDI blockchain paper without translation.
- Implement a toy blockchain, a smart contract, a fraud-proof bisection, and a Circom circuit.
- Critique any new "L2" or "L3" launch with informed skepticism.
- Identify a tractable research problem from one of the 12 frontier areas.

Estimated time

- Casual (6 hrs/week): 24 weeks
- Focused (15 hrs/week): 10 weeks
- Full immersion (30 hrs/week): 5 weeks

Blockchain Module 1 — Cryptographic Foundations

Goal: master the four cryptographic primitives every blockchain rests on: hash functions, Merkle trees, digital signatures, and commitment schemes. Without these, every later concept is hand-waving.

1.1 Why cryptography is the load-bearing pillar

A blockchain is, mechanically, a **replicated state machine** in an adversarial network. Cryptography gives us three guarantees we cannot get any other way:

1. **Integrity** — a hash chain lets every honest node detect tampering.
2. **Authentication** — digital signatures bind actions to identities (public keys).
3. **Commitment** — once published, a value cannot be silently changed.

Everything else (consensus, smart contracts, governance) is engineering on top of those three primitives.

1.2 Cryptographic hash functions

A hash function $H: \{0,1\}^* \rightarrow \{0,1\}^n$ (typically $n=256$) is used to compress arbitrary data to a fixed-length digest.

Required properties

Property	Definition	Why it matters
Pre-image resistance	Given y , infeasible to find x s.t. $H(x) = y$	Hides inputs (commitments, password storage)
Second pre-image resistance	Given x , infeasible to find $x' \neq x$ s.t. $H(x') = H(x)$	Prevents targeted tampering
Collision resistance	Infeasible to find any (x, x') with $H(x) = H(x')$	Prevents two valid blocks colliding
Puzzle-friendliness	For any k , the distribution of $H(k \ x)$ is "random enough" that you must brute-force to find solutions	Foundation of Proof-of-Work

Standard choice: **SHA-256** for Bitcoin; **Keccak-256** for Ethereum; **BLAKE3** is faster but less established.

Birthday bound

Collision attacks on an n -bit hash take roughly $2^{(n/2)}$ work. So SHA-256 gives ~128-bit collision security

— currently safe.

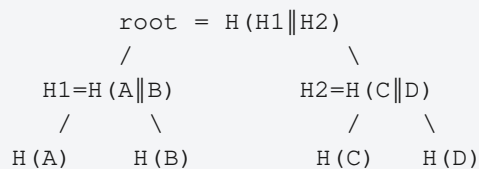
Hash chain

```
block_0    block_1    block_2
[h_prev=1] → [h_prev = H(block_0)] → [h_prev = H(block_1)] → ...
```

Any tampered block changes its hash, breaking every subsequent link. This is the simplest data structure with **tamper evidence**.

1.3 Merkle trees

To prove that one transaction is included in a block without sending the whole block, we use a **Merkle tree** (a.k.a. hash tree).



Inclusion proof for transaction C: send $H(D)$ and $H1$ (log n hashes). The verifier recomputes:

```
H2' = H(H(C) || H(D))
root' = H(H1 || H2')
```

If $root' == published\ root$, C is in the tree.

Real-world variants

- **Bitcoin Merkle tree** — flat binary tree of transactions in a block.
- **Patricia/Verkle tries** (Ethereum) — Merkle trie indexed by account address; supports efficient state proofs.
- **Sparse Merkle trees** — used in Filecoin, rollups.
- **Verkle trees** — replace hashes with vector commitments (e.g., KZG) — proof size $O(1)$ instead of $O(\log n)$. Important for stateless Ethereum.

Research angle

Verkle trees, IVC (incrementally verifiable computation), and "history expiry" are active 2023–2026 research directions; see EIP-6800.

1.4 Digital signatures

Signature scheme = (KeyGen, Sign, Verify).

Scheme	Curve / basis	Used by
ECDSA	secp256k1	Bitcoin, Ethereum (pre-account abstraction)
EdDSA / Ed25519	edwards25519	Solana, Cardano, Tezos
Schnorr (BIP-340)	secp256k1	Bitcoin Taproot, supports key aggregation
BLS	BLS12-381	Ethereum consensus (validator aggregation)

Why BLS matters

A BLS signature scheme allows you to *aggregate* n signatures from n validators into a single 96-byte signature that verifies all of them at once. Without this, Ethereum's PoS could not scale to 1M+ validators.

Public keys are addresses

In Bitcoin: `address = Base58Check(RIPEMD160(SHA256(pubkey)))`. In Ethereum: `address = last 20 bytes of Keccak256(pubkey)`.

This is how a string of letters and numbers binds to a specific cryptographic identity.

1.5 Commitment schemes

A **commitment** lets you publish a binding fingerprint of a value v without revealing it (hiding), and later reveal v such that nobody can dispute it (binding).

Simplest commitment: $c = H(v \parallel r)$ where r is a random nonce.

Pedersen commitments

$c = v \cdot G + r \cdot H$ where G, H are generators of an elliptic curve group with unknown discrete log relation. Pedersen is **homomorphic** — $\text{Commit}(v1) + \text{Commit}(v2) = \text{Commit}(v1+v2)$ — which makes it the basis of confidential transactions (Monero, Mumblewimble).

KZG / polynomial commitments

A KZG commitment lets you commit to a polynomial $p(x)$ and later open it at any point with a constant-size proof. KZG underlies:

- Ethereum's **EIP-4844 blob transactions** (proto-danksharding)
- Verkle trees
- PLONK-family zk-SNARKs

This is *the* commitment scheme of the modern blockchain stack.

1.6 Putting it all together — a toy block

```

class Block:
    prev_hash: bytes          # 32 bytes from previous block
    merkle_root: bytes        # root over transactions
    timestamp: int
    nonce: int                # PoW puzzle solution
    transactions: list[Tx]

    def hash(self) -> bytes:
        return sha256(prev_hash + merkle_root + ts + nonce)

# Each Tx contains: from_pubkey, to_pubkey, amount, signature
# Tx is valid iff verify(from_pubkey, msg=(from,to,amount), signature)

```

This is — quite literally — Bitcoin minus the consensus rules.

1.7 Reading list

- Narayanan et al., *Bitcoin and Cryptocurrency Technologies*, Ch 1.
- Katz & Lindell, *Introduction to Modern Cryptography*, Ch 5 (hash) & Ch 12 (signatures).
- Boneh & Shoup, *A Graduate Course in Applied Cryptography* (free online).
- Boneh, Drake, Fisch, Gabizon — *KZG polynomial commitments and applications* (talks).
- EIP-4844 specification (proto-danksharding).

1.8 Lab (2 hrs)

Implement, in Python with `hashlib` and `ecdsa` only (no blockchain framework):

1. A `Block` class with header + Merkle root.
2. A `Wallet` class that generates (privkey, pubkey, address).
3. A `Transaction` class with signing + verification.
4. A 5-block chain — show that flipping a bit in block 2 breaks all subsequent hashes.

1.9 Quiz

1. Why is collision resistance strictly stronger than second-preimage resistance?
2. What is the size (in hashes) of a Merkle proof for one of 1,048,576 leaves?
3. Why does BLS signature aggregation matter for Proof-of-Stake?
4. State one property that Pedersen has and SHA-based commitments don't.
5. Roughly how many SHA-256 evaluations are needed to find a collision?

Blockchain Module 2 — Distributed Systems & the CAP/FLP Walls

Goal: Understand why "just add a database" doesn't work in an open, adversarial network.
Build the intuition for why blockchains are slow.

2.1 The classical distributed systems triangle

Before blockchains, the problem of "n computers agreeing on the same state" was studied for decades. Two impossibility results define the playing field.

CAP theorem (Brewer, 2000; formalized Gilbert & Lynch, 2002)

In a distributed system you can pick at most **two** of:

- **C — Consistency** (every read gets the latest write)
- **A — Availability** (every request gets a non-error response)
- **P — Partition tolerance** (system continues operating across network splits)

Real networks **always** drop packets, so P is mandatory. The real choice is **CP vs AP**.

System	Choice
PostgreSQL replica	CP
Cassandra	AP
Bitcoin (eventual finality)	AP-ish
Ethereum PoS (with finality gadget)	CP for finalized, AP for tip

FLP impossibility (Fischer, Lynch, Paterson, 1985)

In a purely **asynchronous** system, no deterministic protocol can guarantee consensus if even one node may crash.

This is why every real-world consensus protocol assumes one of:

- **Partial synchrony** (messages eventually arrive within Δ) — used by PBFT, HotStuff, Tendermint, Casper.
- **Randomization** (Ben-Or, common coin) — used by Algorand, HoneyBadger BFT.
- **Synchrony** (bounded message delay) — used by classical Byzantine agreement.

You cannot escape FLP — you can only choose which assumption to weaken.

2.2 Failure models

Model	What can a node do?	Examples
Crash-stop	Halt and stop responding	Paxos, Raft
Crash-recovery	Stop and later restart	Real distributed DBs
Byzantine	Arbitrary, malicious behavior	Bitcoin, Ethereum, PBFT
Rational	Self-interested, follows protocol when incentivized	BAR-Tolerant Replication, blockchain proper

Blockchain's contribution to distributed systems theory was demonstrating **economic-rational Byzantine tolerance at planetary scale**.

2.3 The Byzantine Generals Problem (Lamport, Shostak, Pease, 1982)

n generals, f of whom are traitors. They must agree on attack/retreat using only messages.

Classical bound: agreement is possible iff $n \geq 3f + 1$ for synchronous, signed-message-free protocols. With signatures (authenticated Byzantine), $n \geq 2f + 1$ suffices but requires synchrony.

The number $2/3$ shows up everywhere in BFT:

- Tendermint needs $>2/3$ honest stake to finalize.
- HotStuff (used by Diem, Aptos) — same.
- Ethereum PoS finality — needs $2/3$ honest validators.

2.4 Replicated state machines (RSM)

A blockchain *is* a replicated state machine.

```
state_(t+1) = transition(state_t, tx)
```

For all honest nodes to agree on `state_t`, they must agree on:

1. The **set** of transactions applied
2. The **order** of transactions applied

Consensus protocols solve problem #2. Problem #1 is solved trivially once order is fixed (everyone broadcasts and applies in agreed order).

This framing shows that *consensus is about ordering, not about validity*. A transaction can be ordered before being validated.

2.5 Leader-based vs leaderless consensus

Family	Mechanism	Example
Leader-based	One node proposes a block per round	PBFT, HotStuff, Raft, Tendermint
Probabilistic leader	Random leader per round (VRF)	Algorand, Cardano Ouroboros
Leaderless	DAG of blocks, no single proposer	IOTA Tangle, Aleph, Narwhal-Bullshark

Leader-based is simpler but creates a bottleneck. Leaderless / DAG-based protocols (Narwhal, Bullshark, Mysticeti from Sui; Aleph from Aleph Zero) are the 2023–2026 frontier and achieve >100k TPS in published benchmarks.

2.6 The fork problem & finality

Fork = the chain splits because two nodes propose conflicting blocks.

Finality type	Definition	Example
Probabilistic	Probability of reversal $\rightarrow 0$ as more blocks confirm	Bitcoin (6 blocks $\approx 99.99\%$)
Economic	Reversal would cost $>X$ tokens	PoS slashing
Absolute	Finalized block cannot be reverted without $> f$ Byzantine breach	Tendermint, PBFT-style

Bitcoin: probabilistic, eventual. Ethereum post-Merge: hybrid — tip is probabilistic, finalized checkpoint is absolute (with 1/3 of stake at risk to revert).

2.7 Why throughput is limited

A back-of-envelope for any synchronous BFT protocol:

$$\text{throughput} \leq \text{block_size} / (\text{network_delay} + \text{signature_verification} + \text{state_writes})$$

If you increase block size, propagation slows $\rightarrow$ forks. This is **Nakamoto's tradeoff**.

L2s / sharding / DAG mempools are all attempts to shift parts of this equation off the critical path.

2.8 The mempool

Before consensus, transactions live in each node's **mempool** (memory pool). Selection from mempool into a block is where **MEV** (Maximal Extractable Value) lives. Most blockchain economic exploits originate here.

2.9 Reading list

- Lamport, Shostak, Pease — *The Byzantine Generals Problem*, 1982.
- Fischer, Lynch, Paterson — *Impossibility of distributed consensus with one faulty process*, 1985.
- Castro & Liskov — *Practical Byzantine Fault Tolerance*, OSDI 1999.
- Yin et al. — *HotStuff: BFT consensus in the lens of blockchain*, PODC 2019.
- Garay, Kiayias, Leonardos — *The Bitcoin Backbone Protocol*, Eurocrypt 2015.
- Buchman, Kwon, Milosevic — *The latest gossip on BFT consensus* (Tendermint).

2.10 Lab (2 hrs)

Build a simulator (no real network) of 7 nodes running a toy 2/3-majority commit protocol:

1. Each round one leader proposes.
2. Each node either votes commit or aborts.
3. Crash 2 of the 7 → it should still finalize. Make 3 Byzantine → it must fail.
4. Plot finality latency vs. number of Byzantine nodes.

2.11 Quiz

1. Why does CAP force most blockchains to favor AP at the tip and CP at finality?
2. State the $3f+1$ bound and why it appears in PBFT and Tendermint.
3. What's the difference between probabilistic and absolute finality?
4. Why does increasing block size hurt decentralization?
5. Name two DAG-based consensus protocols and one tradeoff they make vs leader-based ones.

Blockchain Module 3 — Bitcoin Deep Dive

Goal: Understand Bitcoin not as a coin but as the first working solution to Byzantine consensus over an open membership network. Every later blockchain is a delta against Bitcoin.

3.1 The white paper in one paragraph

Satoshi Nakamoto (2008) proposed: combine (a) hash-chained blocks, (b) PoW puzzle, (c) longest-chain rule, (d) economic incentives so that **honest nodes following the protocol find it more profitable than attacking it**. The protocol is open — anyone can join — yet Byzantine-tolerant under the assumption that >50% of compute is honest.

3.2 The UTXO model

Bitcoin's state is not a list of balances. It's a set of **Unspent Transaction Outputs**.

```
Tx {
  inputs:  [ (prev_tx_id, output_index, signature) ... ]
  outputs: [ (amount, locking_script) ... ]
}
```

To spend, you reference previous outputs and prove you can unlock them (via Script). Total input value ≥ total output value; the difference is the miner fee.

Why UTXO?

- **Stateless validation** — verifying a tx needs only the referenced UTXOs, not full chain state.
- **Parallelizable** — txs touching disjoint UTXOs can be validated in parallel.
- **Privacy** — each UTXO can be a fresh address.

Ethereum chose the **account model** instead — simpler smart contracts but harder to parallelize.

3.3 Bitcoin Script

A stack-based, non-Turing-complete language. Standard "P2PKH" locking script:

```
OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

Unlocking script supplies <signature> <pubKey>. Combined, the script evaluates to TRUE iff the signature is valid for that pubKey, whose hash matches <pubKeyHash>.

Why not Turing complete?

To bound execution time and prevent halting-problem attacks. Bitcoin chose this conservatism deliberately. Taproot (2021) and Miniscript expanded expressiveness while keeping safety guarantees.

3.4 Proof-of-Work

The puzzle: find a nonce such that $H(\text{block_header}) < \text{target}$, where `target` is set so that the global network produces one block every 10 minutes.

```
while sha256(sha256(header_with(nonce))) >= target:
    nonce += 1
```

Difficulty adjustment

Every 2016 blocks (~2 weeks), retarget so that the previous 2016 blocks took ~2 weeks of wall time. This is what gives Bitcoin its stable block interval despite enormous hashrate growth.

Nakamoto's security argument

If an attacker has fraction $q < 0.5$ of hashrate and is z blocks behind, the probability they ever catch up follows a Negative Binomial. By $z=6$, $P(\text{reversal}) \approx 0.001$ for $q=0.1$ and falls exponentially.

This is **probabilistic finality** and the source of the "wait 6 confirmations" folklore.

3.5 Block structure

```
Header (80 bytes):
    version, prev_hash, merkle_root, timestamp, bits (target), nonce
Body:
    varint count, then transactions
```

Maximum block size: 1MB (4 MB weight after SegWit). Average tx size: ~250 bytes → ~3.5 tx/sec theoretical max throughput. **This is the famous bottleneck.**

3.6 Mining and the incentive layer

Miner reward per block = `subsidy + transaction fees`.

Era	Block subsidy
2009–2012	50 BTC

2012–2016	25 BTC
2016–2020	12.5 BTC
2020–2024	6.25 BTC
2024–2028	3.125 BTC
...	halves every 210,000 blocks (~4 yrs)

After ~2140, subsidy → 0. Bitcoin's long-term security depends on fees alone — an open research question.

Selfish mining (Eyal & Sirer, 2014)

A miner with >25% hashrate can withhold blocks and gain disproportionate reward — proving Nakamoto's "honest-majority" assumption was naive. Mitigations are limited; this remains the canonical critique.

3.7 The 51% attack

With >50% hashrate, an attacker can:

- Double-spend (rewrite history they were a party to)
- Censor transactions
- **Cannot:** steal coins they don't have keys for; create new coins.

Cost to attack Bitcoin at 2026 hashrate: tens of millions of USD/hour. Cost to attack a small PoW altcoin: <\$1k/hour (see [crypto51.app](#)).

3.8 Lightning Network (L2)

A payment channel network on top of Bitcoin.

1. Alice and Bob open a 2-of-2 multisig channel with 1 BTC each.
2. They update balances off-chain by exchanging signed commitment txs.
3. Either party can broadcast the latest commitment to close.
4. HTLCs allow routing payments across multi-hop channels.

Throughput: theoretically millions of TPS. Tradeoffs: liquidity locked in channels, online requirements (watchtowers), routing UX problems.

3.9 SegWit, Taproot, and the upgrade path

Upgrade	Year	What it did
---------	------	-------------

SegWit (BIP-141)	2017	Separated signature data → fixed malleability, enabled Lightning
Taproot (BIP-340/341/342)	2021	Schnorr signatures + Tapscript → privacy + multi-sig efficiency
Inscriptions / Ordinals	2023	Unintended use of Taproot data → on-chain NFTs
Future: covenants (OP_CAT, CTV)	TBD	Programmable spending conditions, vaults

3.10 What Bitcoin is *not*

- Not anonymous — pseudonymous; chain analysis routinely de-anonymizes addresses.
- Not free — fees fluctuate \$0.50 to \$50+.
- Not energy-efficient — ~150 TWh/yr, comparable to a mid-size country.
- Not fast — 7 tx/sec, 60-minute finality.

These limitations created the design space every other blockchain explores.

3.11 Reading list

- Nakamoto — *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
- Eyal & Sirer — *Majority is not Enough: Bitcoin Mining is Vulnerable*, FC 2014.
- Antonopoulos — *Mastering Bitcoin* (2nd ed).
- Garay, Kiayias, Leonardos — *The Bitcoin Backbone Protocol*, Eurocrypt 2015.
- Sompolinsky & Zohar — *Secure High-Rate Transaction Processing in Bitcoin*, FC 2015.
- Poon & Dryja — *Lightning Network paper*, 2016.

3.12 Lab (3 hrs)

1. Connect to a Bitcoin testnet node (use `bitcoind regtest` or a public Esplora API).
2. Generate two keypairs, fund one from a faucet, create + sign + broadcast a transaction to the second.
3. Decode the resulting transaction and identify every field.
4. Inspect 3 real mainnet blocks and compute their fee revenue vs subsidy.

3.13 Quiz

1. Why does Bitcoin use the UTXO model instead of account balances?
2. What probability of reversal does Nakamoto's analysis give for $q=0.3$, $z=6$?
3. Explain selfish mining in one sentence and the minimum hashrate for it to be profitable.
4. What is the maximum theoretical TPS on Bitcoin L1, and what limits it?
5. Name two changes Taproot enabled and one second-order effect (e.g., Ordinals).

Blockchain Module 4 — Ethereum & Smart Contracts

Goal: Understand Ethereum as a programmable state machine and the design choices that opened (and complicated) the smart-contract ecosystem.

4.1 The Ethereum thesis

Vitalik Buterin (2013): generalize Bitcoin's Script into a **Turing-complete VM** so that arbitrary financial logic — not just payments — can be enforced on-chain. The result was the EVM (Ethereum Virtual Machine).

Account model vs UTXO

- **Externally Owned Accounts (EOAs)** — controlled by a private key.
- **Contract Accounts** — controlled by code (deployed at an address).

Each account has: `nonce`, `balance`, `storage_root`, `code_hash`. State is a Merkle Patricia Trie of all accounts.

4.2 The EVM

A 256-bit, stack-based virtual machine.

Component	Purpose
Stack	1024 slots, 256-bit words
Memory	Volatile, byte-addressable
Storage	Persistent, key→value, per contract
Gas	Metering — every opcode costs gas
Logs	Off-chain indexable events (used by The Graph)

Gas

Every operation consumes gas. Transaction sender sets `gas_limit` and `gas_price`. If gas runs out, execution reverts but fees are still paid. This is how the EVM avoids the halting problem despite Turing completeness.

Op	Gas cost
ADD	3

SLOAD (read storage)	2100 (cold) / 100 (warm)
SSTORE (write storage)	20,000 (new) / 5,000 (modify)
CALL	700 + memory

These costs incentivize gas-aware contract design — a discipline as deep as low-latency C.

4.3 Solidity — the canonical language

```
pragma solidity ^0.8.20;

contract ERC20 {
    mapping(address => uint256) public balanceOf;
    uint256 public totalSupply;

    event Transfer(address indexed from, address indexed to, uint256 value);

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balanceOf[msg.sender] >= amount, "insufficient");
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }
}
```

Other languages: **Vyper** (Python-like, safer subset), **Huff** (low-level for gas-golf), **Fe** (Rust-inspired).

4.4 ERC standards

ERC	Purpose	Where it shows up
ERC-20	Fungible tokens	USDC, DAI, WETH
ERC-721	Non-fungible tokens (NFTs)	CryptoPunks, BAYC
ERC-1155	Multi-token (fungible + NFT)	Game items, OpenSea
ERC-4626	Tokenized vaults	DeFi yield aggregators
ERC-4337	Account abstraction	Smart-contract wallets
ERC-6551	Token-bound accounts	NFTs that <i>are</i> wallets

Each standard is a Schelling point — a contract that conforms slots seamlessly into the rest of the ecosystem.

4.5 The famous reentrancy bug (and the DAO hack)

```
// VULNERABLE
function withdraw() external {
    uint256 bal = balances[msg.sender];
    (bool ok,) = msg.sender.call{value: bal}(""); // ← attacker re-enters here
    require(ok);
    balances[msg.sender] = 0; // ← updated AFTER the call
}
```

If `msg.sender` is a contract, its `receive()` function runs *during* the external call, and it can call `withdraw()` again before `balances[msg.sender]` is zeroed.

The 2016 DAO hack drained 3.6M ETH this way (~\$60M then; \$billions today). The fix:

```
// CHECKS-EFFECTS-INTERACTIONS pattern
function withdraw() external {
    uint256 bal = balances[msg.sender];
    balances[msg.sender] = 0; // effect FIRST
    (bool ok,) = msg.sender.call{value: bal}(""); // interaction LAST
    require(ok);
}
```

The DAO hack triggered the contentious Ethereum / Ethereum Classic hard fork — a foundational lesson in protocol governance.

4.6 The Merge & PoS

September 15, 2022: Ethereum switched from PoW to PoS in a flawless coordination feat ("The Merge"). PoS reduced energy consumption by ~99.95%.

Validator deposit: 32 ETH. Slashing for double-signing or surround voting. Block proposers are selected by RANDAO + VRF.

Finality via **Casper FFG** — a checkpoint finality layer on top of the chain. Every epoch (32 slots × 12s = 6.4 min) validators vote on a checkpoint; once a checkpoint has 2/3 attestations *twice*, it's finalized.

4.7 Account abstraction (ERC-4337)

Traditional wallets are EOAs — keys = identity = trust root. ERC-4337 lets *any* contract act as a wallet via `UserOperations` going through a `Bundler` and `EntryPoint`. This unlocks:

- Social recovery (lose your keys, restore via friends)
- Session keys (sign once, transact for a session)
- Sponsored gas (someone else pays your gas)
- Multi-factor / passkey signing

This is the largest UX shift in Ethereum since deployment.

4.8 Layer 2 ecosystem (preview — full coverage in Module 6)

L2	Type	TPS
Arbitrum	Optimistic rollup	~40k
Optimism	Optimistic rollup	~30k
Base	Optimistic (Coinbase)	~30k
zkSync Era	ZK rollup	~100k*
StarkNet	ZK rollup (Cairo)	~100k*
Polygon zkEVM	ZK rollup	~50k*
Scroll	ZK rollup	~20k*

(* peak theoretical)

4.9 The MEV economy

Block proposers can reorder, insert, or censor transactions for profit (Maximal Extractable Value). MEV-Boost separates **block proposers** from **block builders** — proposers auction off the right to build a block. Roughly 90%+ of Ethereum validators run MEV-Boost.

This created **Proposer-Builder Separation (PBS)** as a research field — see [Module 9](#).

4.10 Reading list

- Buterin — *Ethereum White Paper*, 2013.
- Wood — *Ethereum Yellow Paper*, 2014–present.
- Antonopoulos & Wood — *Mastering Ethereum*, O'Reilly.
- Daian et al. — *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in DEXes*, S&P 2020.
- Buterin & Griffith — *Casper the Friendly Finality Gadget*, 2017.
- ERC-4337 spec — eips.ethereum.org/EIPS/eip-4337.

4.11 Lab (3 hrs)

1. Install Foundry. `forge init` a project.
2. Write an ERC-20 with `mint`, `transfer`, and `burn`. Write tests achieving 100% line coverage.
3. Write a deliberately reentrancy-vulnerable bank contract and an attacker contract that drains it.
4. Apply the checks-effects-interactions fix and prove the attack now fails.
5. Deploy to Sepolia testnet. Verify on Etherscan.

4.12 Quiz

1. State three differences between the UTXO and account models.
2. Why is gas necessary even though the EVM is Turing-complete?
3. What is the checks-effects-interactions pattern and which class of bugs does it prevent?
4. Explain the role of RANDAO and VRF in PoS proposer selection.
5. What problem does ERC-4337 solve, and at what cost?

Blockchain Module 5 — Consensus Algorithms

Goal: Compare every major consensus mechanism on the same axes: security model, finality type, throughput, decentralization, and known attacks.

5.1 The consensus design space

A consensus protocol must answer:

1. **Who proposes** the next block? (Sybil-resistance mechanism)
2. **Who votes** on its validity?
3. **When is it final?**
4. **How are honest behavior and dishonest behavior incentivized / punished?**

Combinations of answers give us all known protocols.

5.2 Proof-of-Work (PoW)

Sybil resistance via computational cost.

Property	Value
Proposer selection	Solve hash puzzle
Finality	Probabilistic
Energy use	Very high
Hardware barrier	High (ASICs)
Throughput	Low
Known attacks	Selfish mining, 51%, eclipse

Variants:

- **SHA-256** (Bitcoin) — ASIC-friendly
 - **Ethash** (pre-Merge Ethereum) — memory-hard, GPU-friendly (intentionally ASIC-resistant; partially failed)
 - **RandomX** (Monero) — CPU-friendly via virtual machine
 - **Scrypt, X11, Equihash** — earlier altcoin attempts
-

5.3 Proof-of-Stake (PoS)

Sybil resistance via locked economic value.

A validator deposits collateral; misbehavior is **slashed** (collateral burned). Honest behavior earns issuance + fee rewards.

Variants

Protocol	Mechanism	Used by
Casper FFG + LMD-GHOST	Hybrid finality + fork choice	Ethereum
Tendermint / CometBFT	Round-based BFT, instant finality	Cosmos chains
Ouroboros	Slot leader from VRF	Cardano
Algorand BA*	Cryptographic sortition + BA	Algorand
Snowball / Avalanche	Repeated random sampling	Avalanche
PoH + ToS	Verifiable delay + TowerBFT	Solana

Slashing conditions (Ethereum)

1. **Double proposal** — sign two different blocks for the same slot.
2. **Surround vote** — attesting to a checkpoint that surrounds your previous one.
3. **Inactivity leak** — if chain can't finalize, validators that aren't voting bleed stake.

"Nothing at stake" (early PoS problem, solved)

In naive PoS, a validator could vote on every fork (it's costless), preventing convergence. Slashing solves this by making conflicting votes punishable.

5.4 Proof-of-Authority (PoA)

A fixed set of permissioned validators. Used for consortium chains (POA Network, Binance Smart Chain, many enterprise blockchains).

Pros: very high throughput. Cons: trust-based, single-org risk.

5.5 Practical Byzantine Fault Tolerance (PBFT, 1999)

1. Pre-prepare: leader proposes
2. Prepare: all replicas exchange "I saw the proposal"
3. Commit: all replicas exchange "I'm ready to commit"
4. Reply: once $2f+1$ commits, finalized

$O(n^2)$ messages per round → doesn't scale beyond ~100 nodes. HotStuff (2019) reduced this to linear with chained pipelined voting.

5.6 HotStuff

The basis of Diem, Aptos, and modern BFT chains.

- **Linear view change** — replacing a faulty leader costs $O(n)$ messages instead of $O(n^2)$.
 - **Pipelined** — three-phase QC chained so each block contributes to multiple commits.
 - **Optimistic responsiveness** — protocol moves at network speed when network is healthy.
-

5.7 Tendermint / CometBFT

Used by Cosmos chains.

- Round-based: propose → pre-vote → pre-commit → commit.
 - Instant absolute finality (1 block).
 - Validator set fixed per epoch.
 - Cannot fork — if 2/3 are honest, only one block per height.
 - Limit: ~100–200 validators (gossip cost).
-

5.8 Ouroboros (Cardano)

Time is divided into **epochs** → **slots**. A VRF selects a slot leader. Multiple variants:

- **Ouroboros Classic** — synchronous, multi-party coin tossing.
- **Ouroboros Praos** — semi-synchronous, secure under adaptive corruption.
- **Ouroboros Genesis** — full bootstrap security (no trusted setup of recent chain).
- **Ouroboros Hydra** — L2 with off-chain state channels.

Notable for being one of the most academically rigorous protocol families.

5.9 Algorand BA*

Pure PoS with cryptographic sortition.

- Each round, ~1000 validators are sampled (private VRF reveal).
 - BA* algorithm runs Byzantine agreement in ~5 seconds.
 - No forks, instant finality.
 - Designed under partial synchrony with adaptive corruption.
-

5.10 Avalanche consensus (Snowball/Snowflake/Avalanche)

A **gossip-and-sample** family:

```

loop:
  query K random peers for their preference
  if  $\alpha \cdot K$  agree, increment confidence
  if confidence >  $\beta$ , decide

```

Sub-second finality, very high throughput. Used by Avalanche network.

5.11 Proof-of-History (Solana)

A VDF (verifiable delay function) produces a cryptographic clock so validators don't need to negotiate timestamps. Combined with TowerBFT for finality.

Pros: massive throughput claims (50k+ TPS). Cons: high hardware requirements → centralization pressure; several major outages.

5.12 Comparison table

Protocol	Finality	TPS (real)	Validators	Energy	Sybil
Bitcoin PoW	Probabilistic 60min	7	~thousands (miners)	~150 TWh/ yr	Compute
Ethereum PoS	12.8min absolute	15 (L1), 100k+ (L2)	~1M	Low	Stake
Cosmos (CometBFT)	1 block, ~6s	~1k	~150	Low	Stake
Solana (PoH+TowerBFT)	~13s	4–6k typical	~2k	Mid	Stake
Avalanche	<2s	1–5k	~1.5k	Low	Stake
Cardano (Ouroboros)	Probabilistic	~250	~3k stake pools	Low	Stake

5.13 Reading list

- Castro & Liskov — *Practical Byzantine Fault Tolerance*, 1999.
- Yin et al. — *HotStuff: BFT Consensus with Linearity and Responsiveness*, PODC 2019.
- Kiayias et al. — *Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol*, CRYPTO 2017.
- Chen & Micali — *Algorand: A Secure and Efficient Distributed Ledger*, 2017.
- Rocket et al. — *Scalable and Probabilistic Leaderless BFT Consensus through Metastability* (Avalanche).
- Yakovenko — *Solana: A new architecture for a high performance blockchain*, 2018.
- Buterin & Griffith — *Casper the Friendly Finality Gadget*, 2017.

5.14 Lab (3 hrs)

Pick **two** protocols and implement a stripped-down simulator:

1. Tendermint round (propose → prevote → precommit → commit) with 4 nodes, 1 Byzantine.
2. Avalanche Snowball with 100 nodes — show convergence as α and K vary. Plot finality time vs Byzantine fraction.

5.15 Quiz

1. Why does naive PoS suffer from "nothing at stake" and how does slashing fix it?
2. State HotStuff's improvement over PBFT in big-O terms.
3. What's the role of the VRF in Ouroboros and Algorand?
4. Explain why Solana's PoH is *not* a consensus mechanism by itself.
5. Why is Tendermint capped at ~150 validators in practice?

Blockchain Module 6 — Scalability & Layer 2

Goal: Internalize the **scalability trilemma**, understand every major L2 design (rollups, channels, sidechains, validiums, plasma), and reason about which design fits which application.

6.1 The scalability trilemma (Vitalik, 2017)

A blockchain can have at most two of: **Scalability**, **Security**, **Decentralization**.

Strictly informal but useful as a planning lens:

- Bitcoin/Ethereum L1: Security + Decentralization, weak Scalability.
- BSC / EOS: Scalability + Security, weak Decentralization.
- Many sidechains: Scalability + Decentralization, weaker Security.

L2s aim to inherit L1 Security while gaining Scalability.

6.2 Scaling axes

Axis	Lever
Vertical	Bigger blocks, faster nodes (Solana)
Horizontal	Sharding (Near, Ethereum data sharding)
Off-chain	Channels (Lightning)
Rollups	Bundle txs off-chain, post compressed proof on-chain
Sidechains	Independent chain, bridged

6.3 State / payment channels

```
Open:    on-chain multisig deposit
Update:  off-chain signed state transitions
Close:   on-chain settlement
```

- **Bitcoin Lightning** — payment channels with HTLC routing.
- **Raiden** — Lightning for Ethereum ERC-20s.
- **Connex, Hydra (Cardano)** — generalized state channels.

Pros: instant, near-free. Cons: requires liquidity, online watchtowers, doesn't generalize beyond fixed participants.

6.4 Plasma (deprecated but instructive)

Vitalik & Joseph Poon, 2017. A child chain commits Merkle roots to the parent chain. Users could exit fraudulent state.

Why it died: exit games are hard, mass exit problem (one operator hides data → all users must exit within window), unsuited to general smart contracts.

The exit-game intellectual heritage lives on in **Optimistic Rollups**.

6.5 Sidechains

A separate blockchain with its own consensus, bridged to L1.

Example	Mechanism
Polygon PoS	Heimdall consensus + checkpoint commits to Ethereum
Gnosis Chain	Independent PoS, xDAI
Liquid (Bitcoin)	Federation-secured

Security is **independent of L1** — you only trust the sidechain's own consensus. A bridge hack is a sidechain hack.

6.6 Rollups — the dominant L2 paradigm

A **rollup** executes transactions off-chain, posts:

- **Compressed transaction data** to L1 (data availability)
- **A validity argument** that the new state is correct

Two flavors

Type	Validity argument
Optimistic Rollup	Assume correct; allow fraud proofs within a 7-day challenge window
ZK Rollup (a.k.a. Validity Rollup)	Post a zk-SNARK / zk-STARK proving correctness — instant finality

Why both exist

	Optimistic	ZK
Finality	7 days	minutes

Gas cost on L1	Cheap	Currently expensive (prover cost) but shrinking
EVM equivalence	Easy	Hard (zkEVM is bleeding edge)
Maturity	Higher	Catching up fast

6.7 Optimistic Rollups in depth

Examples: **Arbitrum One, Optimism, Base, Mantle, Blast.**

Fraud proof game (interactive):

1. Sequencer posts a state root.
2. Within 7 days, anyone can challenge.
3. Bisection narrows down to the disputed step.
4. L1 EVM executes that single step. Whoever was wrong loses bond.

This is the "bisection" trick that makes fraud proofs $O(\log n)$ instead of $O(n)$.

Arbitrum vs Optimism

- **Arbitrum** uses a WASM-based interactive proof and has a custom precompile model.
- **Optimism** built **OP Stack**, a modular L2 framework now powering Base, Worldcoin, Zora, etc. → "Superchain" vision.

6.8 ZK Rollups in depth

Examples: **zkSync Era, StarkNet, Polygon zkEVM, Scroll, Linea.**

The sequencer:

1. Executes a batch of txs.
2. Generates a SNARK/STARK proving $(prev_state, txs) \rightarrow new_state$ is valid under EVM semantics.
3. Posts proof + minimal data to L1.

L1 verifies the proof (~200k gas). Done.

zkEVM equivalence levels (Vitalik's classification)

- **Type 1**: byte-exact EVM (Taiko aims here)
- **Type 2**: equivalent for any Solidity program (Scroll, Polygon zkEVM)
- **Type 2.5**: minor gas-cost differences
- **Type 3**: most contracts work (early zkSync)
- **Type 4**: high-level language compiles to ZK-native (StarkNet/Cairo, zkSync's Era)

Type 1 is hardest (most EVM-compatible, slowest proving). Type 4 is easiest (custom VM).

6.9 Data availability (DA) — the next bottleneck

Rollups *must* post tx data on L1 so anyone can reconstruct state. Posting raw data is expensive.

Solutions

Solution	Approach
EIP-4844 (proto-danksharding)	Blob transactions — cheap, ephemeral (18 days), 128KB blobs
Full Danksharding	128 blobs per slot, ~1.3MB/slot DA → ~100k TPS L2
Celestia	Standalone DA layer using data availability sampling (DAS)
EigenDA	Restaking-secured DA on EigenLayer
Avail	Polygon's standalone DA chain
Validium	DA off-chain (DAC committee) — cheaper but weaker security

DAS lets light clients verify with high probability that data was published, without downloading all of it — see Mustafa Al-Bassam's thesis.

6.10 Hybrid: Volitions and DACs

A **volition** lets users choose per-transaction whether their data goes on L1 (rollup security) or off-chain (validium speed). StarkEx pioneered this.

6.11 The "modular blockchain" thesis

Decompose monolithic blockchains into:

Layer	Responsibility	Examples
Execution	Run tx state transitions	rollups, OP Stack, Arbitrum Orbit
Settlement	Resolve disputes, anchor canonicity	Ethereum
Data availability	Make tx data retrievable	EigenDA, Celestia, Avail, blobs
Consensus	Order blocks	Ethereum, Tendermint

Cosmos, Celestia, EigenLayer, and the OP Superchain are all bets on modularity. **Solana** is the monolithic counter-bet.

6.12 Shared sequencers & cross-rollup atomicity

If every rollup has its own sequencer, cross-rollup composability dies. Active research:

- **Espresso, Astria** — shared sequencer networks
- **SUAVE (Flashbots)** — decentralized block builder
- **Booster Rollups** — designed to share execution

6.13 Reading list

- Buterin — *An Incomplete Guide to Rollups*, 2021 (vitalik.eth.limo).
- Al-Bassam, Sonnino, Buterin — *Fraud and Data Availability Proofs*, 2018.
- Kalodner et al. — *Arbitrum: Scalable, private smart contracts*, USENIX Security 2018.
- Ben-Sasson et al. — *Zerocash + zk-STARK* papers.
- *EIP-4844: Proto-Danksharding*.
- *Celestia whitepaper*.
- Buterin — *Endgame*, 2021.

6.14 Lab (3 hrs)

1. Deploy the same Solidity contract on Sepolia (L1) and Optimism Sepolia (L2). Compare gas cost.
2. Use `op-batcher` data on Etherscan to inspect a real L2 batch.
3. Read a single blob from a Beacon node via `/eth/v1/beacon/blob_sidecars/{slot}`.
4. Compute the cost-per-byte savings of EIP-4844 vs calldata.

6.15 Quiz

1. State the scalability trilemma and one counter-example to it.
2. Why do Optimistic Rollups need a 7-day challenge window?
3. What's the difference between a Type-2 and Type-4 zkEVM?
4. Define data availability sampling in one sentence.
5. What does EIP-4844 reduce, and by roughly how much?

Blockchain Module 7 — Privacy & Zero-Knowledge Proofs

Goal: Build the mental model for ZK proofs — what they prove, what they hide, and the algebraic machinery that makes it possible. ZK is the most transformative cryptographic tool of the decade.

7.1 What is a zero-knowledge proof?

A protocol by which a **prover** convinces a **verifier** that a statement $x \in L$ (some language L) is true, without revealing anything beyond that fact.

Three properties

1. **Completeness** — if $x \in L$, an honest prover convinces an honest verifier.
2. **Soundness** — if $x \notin L$, no malicious prover can convince an honest verifier (except with negligible probability).
3. **Zero-knowledge** — the verifier learns nothing about the witness w beyond $x \in L$.

Formalized by Goldwasser, Micali, Rackoff, 1985 (STOC).

7.2 Why ZK matters for blockchains

Use case	Why ZK
Privacy (Zcash, Aleo)	Hide sender, receiver, amount
Scalability (ZK rollups)	One proof verifies thousands of txs
Identity (Worldcoin, Sismo)	Prove "I am unique human" without revealing who
Compliance (Aztec, zkKYC)	Prove "I am not on a sanctions list" without revealing identity
ML (zkML)	Prove inference was done correctly without revealing model or input

7.3 Interactive vs non-interactive

Type	Form	Use case
Interactive (IP)	Multi-round dialogue	Theory, identification protocols

Non-interactive (NIZK)	Single message via random oracle (Fiat-Shamir)	Blockchains (no interaction with miners possible)
-------------------------------	--	---

Fiat-Shamir heuristic: replace verifier challenges with hash of transcript. Compatible with public broadcast.

7.4 The SNARK / STARK family

Property	SNARK	STARK
Trusted setup	Often needed (Groth16)	Never
Proof size	~200 bytes	~50–200 KB
Verifier time	~5 ms	~10–40 ms
Prover time	Slow	Faster (with FRI)
Post-quantum	No (most)	Yes
Crypto basis	Pairings (BN254/BLS12-381)	Hash functions only

SNARK construction families

- **Groth16** (2016) — smallest proofs, per-circuit trusted setup. Used by Zcash Sapling.
- **PLONK** (2019) — universal trusted setup (one per chain). Halo2, plonky2/3 build on this.
- **Marlin** — universal setup, smaller proofs than PLONK in some settings.
- **Halo / Halo2** — no trusted setup, recursive composition. Used by Zcash Orchard, Mina.
- **Nova / SuperNova / HyperNova** — incrementally verifiable computation (IVC) via folding. Hot 2023–2026 research.

STARK

Ben-Sasson, Bentov, Horesh, Riabzev (2018). Transparent (no trusted setup), post-quantum, uses FRI for polynomial commitments. Used by StarkNet.

7.5 The circuit abstraction

To prove "I know w such that $c(x, w) = 0$ ", we express c as an **arithmetic circuit** over a finite field.

Inputs: public x , private w
Each gate: addition or multiplication over F_p
Output: 0 if predicate holds

Then we reduce circuit satisfaction to a polynomial identity (e.g., **R1CS** → **QAP** for Groth16, **AIR** → **FRI** for STARKs).

7.6 A concrete tiny example

Prove "I know x such that $H(x) = y$ " without revealing x :

1. Write SHA-256 as an arithmetic circuit (~25k constraints in R1CS).
2. Run the SNARK prover with witness x and public input y .
3. Verifier checks the proof.

This is what every privacy coin does for $(sender, amount)$.

7.7 zkSNARKs in writing real circuits

Modern toolchains:

DSL / framework	Backend	Used for
Circom	Groth16, PLONK	Hands-on, large ecosystem
Noir (Aztec)	UltraPlonk	Rust-like syntax
Cairo (StarkWare)	STARK	StarkNet
Halo2	Halo2 / IPA	zkEVMs (Scroll)
Risc Zero zkVM	STARK + recursion	"Prove any Rust program"
SP1 (Succinct)	STARK / Plonky3	General zkVM

zkVMs (Risc Zero, SP1, Jolt, Zeth) are the "compiler" wave — instead of hand-writing circuits, compile arbitrary programs into ZK-provable form.

7.8 Privacy-preserving cryptocurrencies

Project	Mechanism
Zcash	zk-SNARK over Sapling/Orchard circuits
Monero	Ring signatures + RingCT + stealth addresses (not ZK, but related)
Aleo	zkSNARK-based programmable privacy
Aztec	zk-private smart contracts (Noir + UltraPlonk)
Penumbra	Private DeFi in the Cosmos ecosystem

7.9 Multi-Party Computation (MPC) and FHE (briefly)

Tool	What it does
MPC	Multiple parties compute $f(x_1, \dots, x_n)$ without revealing individual x_i
FHE	Compute on encrypted data; only key holder can decrypt result
TEE	Hardware enclave (Intel SGX, AMD SEV) for confidential computation

These are complements to ZK, not substitutes. ZK proves correctness; MPC/FHE compute confidentially.

7.10 ZK + AI = zkML (preview)

Active research: prove a neural network's inference was performed correctly without revealing the model weights (or the input).

Why hard:

- Floating-point $\rightarrow$ field arithmetic
- Non-linear ops (ReLU, softmax) $\rightarrow$ expensive constraints
- Large models $\rightarrow$ billions of constraints

Frameworks: **EZKL**, **ZKonduit**, **Modulus Labs**, **Giza**, **Risc Zero zkML**.

Covered in [Intersection module](#).

7.11 Reading list

- Goldwasser, Micali, Rackoff — *The knowledge complexity of interactive proof systems*, 1985.
- Groth — *On the size of pairing-based non-interactive arguments*, 2016.
- Gabizon, Williamson, Ciobotaru — *PLONK*, 2019.
- Ben-Sasson et al. — *Scalable, transparent, and post-quantum secure computational integrity*, 2018.
- Bowe, Grigg, Hopwood — *Halo: Recursive proof composition without a trusted setup*, 2019.
- Kothapalli, Setty, Tzialla — *Nova*, CRYPTO 2022.
- Boneh, Drake — *zkSNARKs in practice* (Stanford lectures).

7.12 Lab (4 hrs)

1. Install Circom + snarkjs.
2. Write a circuit that proves knowledge of x such that $x * x = 25$ without revealing whether $x=5$ or $x=-5$.
3. Compile, do trusted setup, generate proof, verify on-chain (deploy verifier contract to Sepolia).
4. Then write a Poseidon hash circuit and a Merkle inclusion proof. (Use existing libs.)

7.13 Quiz

1. State the three properties of a ZKP.
2. What problem does Fiat-Shamir solve?

3. Compare Groth16 and PLONK on (setup, proof size, prover time).
4. Why are STARKs considered post-quantum and SNARKs (mostly) not?
5. What is a zkVM and how does it differ from writing a custom circuit?

--	--	--

--	--	--

--	--	--

--	--	--

--	--	--

Blockchain Module 8 — DeFi Architecture

Goal: Understand DeFi as a system of **composable financial primitives**, the math behind AMMs, and the systemic risks (oracles, MEV, governance attacks).

8.1 What makes DeFi different

DeFi = **Decentralized Finance** — financial applications running on permissionless smart contract platforms. Distinctive properties:

- **Non-custodial** — users hold their own keys.
- **Composable ("money legos")** — protocols call other protocols permissionlessly.
- **Public state** — every position is visible and analyzable.
- **24/7** and global.

These are also its risk surface.

8.2 The DeFi stack

Layer	Examples
Stablecoins	USDC, USDT, DAI, FRAX, GHO
DEXes	Uniswap, Curve, Balancer, dYdX
Lending	Aave, Compound, Morpho, Spark
Derivatives	dYdX, GMX, Synthetix, Hyperliquid
Yield aggregators	Yearn, Beefy, Pendle
Liquid staking	Lido, Rocket Pool, EtherFi
Restaking	EigenLayer, Symbiotic, Karak
Bridges	Wormhole, LayerZero, Across, Hop
Oracles	Chainlink, Pyth, RedStone, UMA

8.3 Automated Market Makers (AMMs)

Replace order books with a deterministic price function $f(x, y) = k$.

Uniswap v2 — Constant Product

$$x * y = k$$

If you swap dx X-tokens in, you receive $dy = y - k / (x+dx)$ Y-tokens. The price slips along the curve.

Slippage and impermanent loss

- **Slippage** — large trades move price along curve.
- **Impermanent loss (IL)** — LPs lose vs. just holding when prices diverge:

$$IL(r) = 2 \cdot \sqrt{r} / (1+r) - 1, \quad \text{where } r = \text{price_now} / \text{price_initial}$$

A 2x price move → ~5.7% IL. A 5x move → ~25%.

Curve — StableSwap

Optimized for assets that trade near parity (USDC/DAI). Hybrid of constant-sum (near parity) and constant-product (at extremes), via an amplification coefficient A .

Uniswap v3 — Concentrated Liquidity

LPs supply liquidity within a **price range** $[p_a, p_b]$. Capital efficiency 100x–4000x higher than v2, but requires active management and IL is amplified.

Balancer

Weighted constant-product across N tokens:

$$\prod x_i^{w_i} = k, \quad \text{where } \sum w_i = 1$$

Lets you build custom index funds as AMMs.

Uniswap v4 — Hooks

Pools become extensible — anyone can attach pre/post hooks (custom fees, dynamic fees, on-chain limit orders). The standard for AMM customization in 2024–2026.

8.4 Lending markets

Two dominant designs:

Pool-based (Aave, Compound)

Suppliers deposit into a pool, borrowers draw with overcollateralization. Interest rate is a deterministic function of utilization.

```
borrow_rate = base + slope1·U + slope2·max(U - kink, 0)
```

Isolated / Peer-to-Pool (Morpho)

Match P2P when possible, fall back to pool. Improves capital efficiency.

Liquidation

If collateral / debt < liquidation_threshold, anyone can repay part of debt in exchange for collateral at a discount (liquidation bonus). This keeps the system solvent.

Liquidation cascades during 2020's "Black Thursday" and 2022's collapse showed the system works, but with brutal slippage.

8.5 Stablecoins

Type	Mechanism	Example
Fiat-backed	1:1 USD reserves	USDC, USDT
Crypto-backed (CDP)	Over-collateralized vaults	DAI, LUSD
Algorithmic	Mint/burn against another token	UST (failed catastrophically), AMPL
Hybrid / partially-collateralized	Mix	FRAX (legacy), GHO
LSD-backed	Liquid-staking tokens as collateral	eUSD, USDe

The **Terra/UST collapse** (May 2022) eliminated the pure-algo design space for years.

8.6 Oracles — DeFi's biggest single risk

Smart contracts can't read external prices. They depend on **oracles**.

Type	Example
Push	Chainlink price feeds update on-chain at threshold/interval
Pull	Pyth — signed prices fetched on demand
Optimistic	UMA — assertions, dispute window
TWAP	Uniswap v3 time-weighted price

Oracle attacks

Flash-loan-fueled price manipulation drained Mango Markets (\$117M, 2022), bZx, Cream, and many others. Mitigations:

- Use TWAPs (manipulating averages requires sustained capital).
- Use chain-of-custody-signed oracles (Pyth Pull).
- Cross-check multiple sources.

8.7 Flash loans

Borrow any amount with **no collateral**, provided the loan is repaid in the same transaction. If not, the tx reverts atomically.

```
function flashLoan(uint256 amount, bytes calldata data) external {
    uint256 before = token.balanceOf(address(this));
    token.transfer(msg.sender, amount);
    IFlashBorrower(msg.sender).onFlashLoan(amount, data);
    require(token.balanceOf(address(this)) >= before + fee, "not repaid");
}
```

Legitimate use: arbitrage, collateral swaps, self-liquidation. Attack use: capital-free price manipulation (Mango, bZx, Cream).

The asymmetry — anyone can borrow \$100M for one transaction — is uniquely DeFi.

8.8 MEV (Maximal Extractable Value)

Profit available to whoever orders transactions. Categories:

Category	Example
Arbitrage	Same asset different prices on two DEXes
Sandwich	Insert buy before / sell after a victim's swap
Liquidation	Race to liquidate underwater positions
Long-tail	NFT mints, contract-state predictions

Flashbots introduced auctions for MEV (MEV-Boost). On Ethereum, ~90%+ of validators run MEV-Boost, separating proposers from builders (**PBS**).

MEV mitigation

- **Encrypted mempools** (Shutter, SUAVE)
- **Order flow auctions** (CoWSwap, UniswapX)
- **Fair ordering** (Themis, Pompê)
- **Threshold encryption** (Osmosis)

8.9 Composability — and its dark side

Aave → Lido → Curve → Yearn → Convex — a single user position can span 5+ protocols. This is brilliant and dangerous:

- **Contagion** — one exploit propagates through every composing protocol.
- **Reentrancy across protocols** — calls back through hooks can violate invariants.
- **Read-only reentrancy** — even view functions are unsafe during external calls.

Major incidents: Curve reentrancy (2023, \$73M), Euler (\$197M), Compound oracle glitch.

8.10 Restaking and shared security

EigenLayer (2023): let ETH stakers opt-in to securing additional services (oracles, DA layers, sidechains, AVSs). Liquid restaking tokens (LRTs) compound positions.

Risk: **systemic slashing** — a single AVS bug could cascade. Aggregate restaked ETH at peak: tens of billions of USD.

8.11 Reading list

- Adams, Zinsmeister, Salem, Keefer, Robinson — *Uniswap v3 Core*, 2021.
- Egorov — *StableSwap — efficient mechanism for Stablecoin liquidity*, 2019.
- Daian et al. — *Flash Boys 2.0*, S&P 2020.
- Qin, Zhou, Gervais — *Quantifying Blockchain Extractable Value*, S&P 2022.
- Werner et al. — *SoK: Decentralized Finance (DeFi)*, AFT 2022.
- Lo, Verma — *DeFi protocols for loanable funds*, AFT 2021.
- Klages-Mundt, Minca — *(In)stability for the Blockchain*, 2020 (algorithmic stablecoin analysis).

8.12 Lab (3 hrs)

1. Fork Ethereum mainnet locally with `anvil --fork-url`.
2. Deposit ETH into Uniswap v3 ETH/USDC pool at a chosen tick range. Measure fees earned for 1 hour of simulated swaps.
3. Implement a flash-loan arbitrage between Uniswap v2 and Sushiswap (clone Uni v2). Find a path where it's profitable in fork state.
4. Write a postmortem of one major 2022–2024 DeFi exploit.

8.13 Quiz

1. Derive the IL formula $2\sqrt{r}/(1+r) - 1$ for constant-product AMMs.
2. Why are TWAP oracles harder to manipulate than spot oracles?
3. Explain the difference between push and pull oracles.
4. State three legitimate uses of flash loans.
5. Why does Uniswap v3 concentrated liquidity require active management?

Blockchain Module 9 — Security & Attack Surface

Goal: Build a catalog of every major attack class. Read at least 3 postmortems. Internalize that most blockchain "hacks" are app-layer bugs, not consensus breaks.

9.1 Threat models

Adversary	Capability
External attacker	Can call any public function, see all state
Malicious validator	Can reorder/censor (within their slots), double-sign (gets slashed)
Compromised oracle	Can push arbitrary prices
Compromised bridge	Can mint unbacked tokens on destination chain
Governance attacker	Can pass malicious proposal with enough tokens
Nation-state	Long-range attacks, mining cartel, regulatory capture

9.2 Smart-contract bugs

Top 10 classes

1. **Reentrancy** — external call before state update.
2. **Integer over/underflow** — pre-0.8 Solidity; still possible with `unchecked {}`.
3. **Access control** — missing `onlyOwner` modifiers, default-public functions.
4. **Front-running / MEV exposure** — public mempool reveals intent.
5. **Oracle manipulation** — single source of truth.
6. **Logic bugs** — math wrong, accounting wrong (most common in DeFi).
7. **Improper initialization** — unprotected `initialize()`.
8. **Delegatecall to untrusted code** — full storage access.
9. **`tx.origin` for auth** — phishable.
10. **Unbounded loops** — gas griefing, DoS.

Less obvious

- **Read-only reentrancy** — view functions can return stale values mid-call.
- **Cross-function reentrancy** — re-enter a *different* function that shares state.
- **Signature replay** — missing `nonce` or `chainId` lets one signature be reused.
- **Permit2 / approval phishing** — drainer scams.

9.3 Famous exploits — a starter list

Incident	Year	Loss	Class
The DAO	2016	\$60M	Reentrancy → ETH/ETC fork
Parity multisig	2017	513k ETH frozen	Library kill
bZx	2020 (×2)	\$1M	Oracle / flash loan
Poly Network	2021	\$611M	Access control
Wormhole	2022	\$325M	Signature verification
Ronin Bridge	2022	\$625M	Validator key compromise
Beanstalk	2022	\$182M	Flash-loan governance
Nomad	2022	\$190M	Init bug, copy-paste pillage
Mango Markets	2022	\$117M	Oracle manipulation
Euler Finance	2023	\$197M	Donation-attack accounting
Curve Vyper compiler	2023	\$73M	Compiler reentrancy lock bug
Multichain	2023	\$130M	Insider / opaque
Atomic Wallet	2023	\$100M+	Wallet-side compromise
Orbit Bridge	2024	\$80M	Bridge key compromise

Lesson: Bridges and cross-chain are the single most-attacked surface.

9.4 Consensus-layer attacks

Attack	Target	Mechanism
51% attack	PoW chain	Majority hashrate rewrites history
Long-range attack	PoS chain	Old keys re-create alternate history (mitigated by weak subjectivity)
Eclipse attack	Single node	Isolate node from honest network → feed fake chain
Sybil attack	Network	Many fake identities; Sybil-resistance (PoW/PoS) defeats it
Selfish mining	PoW	Withhold blocks to gain disproportionate reward
Time-bandit	EVM (theoretical)	Reorg to capture historical MEV
Bribery	PoS validators	Pay validators to break protocol
Inactivity leak	PoS chains	If chain can't finalize, non-voting validators bleed stake

9.5 Bridges — the trillion-dollar honey pot

Bridges typically lock tokens on Chain A and mint wrapped tokens on Chain B. Trust models:

Model	Examples	Weakness
Multi-sig federation	early Wormhole, Ronin	Key compromise (Ronin: 5 of 9 keys)
Light client (IBC, etc.)	Cosmos IBC	Strong, but only works for compatible chains
ZK light client	zkBridge, Polyhedra	Strong, expensive
Optimistic	Across, Nomad	Long challenge window
LayerZero / Wormhole / Axelar	Relayer + Oracle separation	Trust assumptions on relayers/guardians

Vitalik's "trilemma for bridges": you can have 2 of {trust-minimization, generality, extensibility-across-chains}.

9.6 Frontend attacks

The chain may be secure but the **frontend** is HTML/JS served from CDNs.

- Curve's frontend was hijacked via DNS hijack (2022) → users signed malicious tx.
- BadgerDAO's CDN-hosted JS injected approval-drainer (2021, ~\$120M).

Mitigations: IPFS-pinned + ENS-resolved frontends, hardware wallet "blind signing" warnings, wallet drainers protection (Blockaid, Wallet Guard).

9.7 Phishing and the user surface

Drainers ("Inferno Drainer", "Pink Drainer", "Angel Drainer") use:

- Fake mints/airdrops
- Permit / Permit2 signature theft
- SetApprovalForAll tricks
- Address poisoning

User-side losses to drainers exceeded \$300M in 2023. The chain didn't fail — the UX did.

9.8 Formal verification & static analysis

Tool	What it does
Slither	Static analysis for Solidity

Mythril	Symbolic execution
Echidna	Property-based fuzzing
Foundry invariant tests	Invariant fuzzing
Certora Prover	SMT-backed formal verification
K-framework	Formal semantics for EVM (KEVM)
Halmos	Symbolic Foundry tests

Best practice 2026 stack: Foundry unit tests + invariant tests + Slither in CI + Certora spec for critical invariants + a professional audit before mainnet.

9.9 The audit industry

Top firms (alphabetical, partial): **Trail of Bits**, **OpenZeppelin**, **ConsenSys Diligence**, **Spearbit**, **Code4rena**, **Zellic**, **Cantina**, **Sherlock**, **Halborn**.

Audits do not prove correctness — they reduce risk. **Code4rena** competitive audits typically find more issues than fixed-fee audits but at higher cost.

9.10 Incentive-compatible disclosure

Practice	Detail
Bug bounty	Immunefi, HackerOne; payouts up to \$10M for L1 critical
Whitehat retrievals	Some exploits returned for 10% bounty
War rooms	Coordinated multi-protocol response (e.g., Curve July 2023)
Pause / kill switches	Trade-off: safety vs decentralization purity

9.11 Reading list

- Atzei, Bartoletti, Cimoli — *A Survey of Attacks on Ethereum Smart Contracts*, POST 2017.
- Daian et al. — *Flash Boys 2.0*, S&P 2020.
- Heilman et al. — *Eclipse Attacks on Bitcoin's Peer-to-Peer Network*, USENIX Security 2015.
- Eyal & Sirer — *Majority Is Not Enough*, FC 2014.
- *SoK: Cross-Chain Communication* (any of several).
- Rekt News — postmortem corpus (rekt.news).

9.12 Lab (3 hrs)

1. Fork Curve's affected pool at the exact block before the July 2023 reentrancy. Reproduce the drain in Foundry.
2. Run Slither on a small Solidity project. Triage the findings.
3. Write an invariant test in Foundry that would have caught the bug.

9.13 Quiz

1. State three differences between reentrancy and read-only reentrancy.
2. Why are bridges disproportionately attacked?
3. Define long-range attack and the standard mitigation.
4. What problem does MEV-Boost solve, and what new one does it introduce?
5. Name one tool from each of: static analysis, fuzzing, formal verification.

Blockchain Module 10 — Governance & Tokenomics

Goal: Understand how decisions get made in protocols, why token economics shapes (and breaks) protocols, and what's actually known from empirical research.

10.1 Two governance layers

1. **Protocol governance** — chain itself (Bitcoin BIPs, Ethereum EIPs).
2. **Application governance** — on-chain DAOs (MakerDAO, Compound, Uniswap, Arbitrum DAO).

The mechanisms and incentive structures look superficially similar but operate very differently.

10.2 Bitcoin's governance — "rough consensus"

- BIPs (Bitcoin Improvement Proposals) → reference implementation → miners signal → users decide via running software (UASF).
- 2017 SegWit / Big Block war showed that **users**, not miners, are the ultimate authority — bitcoin (BTC) followed the user-activated soft fork while Bitcoin Cash split off.

The lesson: in PoW, **hashrate is hired, not sovereign**.

10.3 Ethereum's governance — coordinated chaos

- **EIPs** (Ethereum Improvement Proposals) drafted by core devs.
- **All Core Devs Calls** (ACD) — weekly meeting, no formal vote.
- **Client diversity** — Geth, Nethermind, Erigon, Besu, Reth — implementations must agree or chain forks.
- **Validators / users** ratify by running upgraded clients.

This loose process has produced the smoothest L1 upgrade cycle in the space (London, Shanghai, Cancun, Pectra, Fusaka).

10.4 DAO governance — the on-chain experiment

Mechanisms

Type	Example	Notes
------	---------	-------

Token-weighted voting	Compound, Uniswap	Plutocratic; whales dominate
Conviction voting	1Hive	Stake longer → more weight
Quadratic voting	Gitcoin Grants	Mitigates whales; Sybil-vulnerable
Reputation / soulbound	Coordinape, BrightID	Non-transferable
Optimistic / delegated	Optimism Citizens' House	Bicameral hybrid
veToken	Curve, Velodrome	Lock tokens to gain voting + boost

The veToken / bribe model

CRV locked → veCRV → vote weight on gauges → directs CRV emissions → projects bribe veCRV holders for emissions → secondary market on vICVX (Convex). This is the **Curve Wars** — a real, multi-billion-dollar incentive market.

10.5 Tokenomics as a discipline

A protocol's token simultaneously plays three roles:

1. **Security** — bonded stake for PoS, fees for L1s, slashing collateral.
2. **Governance** — voting rights.
3. **Cash flow** (sometimes) — buybacks, fee dividends.

Token supply schedules

Pattern	Example
Fixed cap, halving	Bitcoin (21M)
Fixed cap, vesting	Most VC tokens (cliff + linear)
Inflationary	Ethereum issuance, Cosmos
Burn-deflationary	EIP-1559 (Ethereum), BNB
Rebasing	OHM, AMPL
Bond-curve / continuous	Olympus, Fei (RIP)

Vesting cliff "unlock walls"

Most VC-funded tokens unlock 6–36 months after TGE. Look at **Token Unlocks** or **TokenTerminal** to spot incoming sell pressure.

10.6 Real fundamentals: fees & revenue

After 2021's hype, the industry shifted to real revenue analysis:

Metric	What it means
Fees	Total paid by users
Revenue	Fees minus distributions back to users (e.g., to LPs)
PE-style ratios	FDV / annualized revenue
Take rate	Protocol's share of total fees

Tools: **Token Terminal**, **DefiLlama**, **Dune Analytics**, **Artemis**.

10.7 Governance attacks

Beanstalk (2022)

Attacker took a \$1B flash loan, used it to vote in a malicious proposal that drained the treasury. Lesson: **time-lock all proposals**; never allow flash-loaned governance.

Tornado Cash governance (2023)

Attacker proposed an "upgrade" with a hidden malicious payload, passed it, took control. Lesson: review every line of every proposal, including innocuous-looking ones.

Quorum / voter apathy

Most DAOs see <5% voter turnout. A motivated minority can dominate. Mitigations: delegation (Compound's COMP delegates), proxy advisors (Tally), executive committees.

10.8 Mechanism design themes

Mechanism	Use
Vickrey auctions	NFT mints (rarely used yet)
Harberger taxes	Radical Markets, partial-common-ownership land
Quadratic funding	Public goods (Gitcoin)
Curation markets	Token-curated registries (TCRs)
Conviction voting	Long-term decisions

Most live DAOs still use simple token-weighted voting because anything else is harder to explain to participants.

10.9 Public goods funding

Open problem: how do you fund clients, libraries, audits — the underfunded layer of every blockchain?

- **Optimism RetroPGF** — retroactive payouts to ecosystem builders.
 - **Bitcoin Grants** — quadratic funding rounds.
 - **Protocol Guild** — Ethereum core dev vesting from app projects.
 - **EIP-1559** — burns fees, distributes value indirectly to holders, not to builders.
-

10.10 Regulation

Jurisdiction	2026 stance (illustrative)
US	Patchwork; CFTC/SEC turf battle; ETH ETFs approved 2024
EU	MiCA in force since 2024
UK	Phased FCA framework
Singapore, UAE, HK	Active licensing regimes
China	Trading banned; CBDC heavy

Effects on protocols:

- KYC requirements for fiat ramps and stablecoin issuers.
 - Sanctions screening (Tornado Cash sanctions, 2022).
 - "Decentralization sufficiency" — ongoing legal test.
-

10.11 Reading list

- Buterin — *Notes on Blockchain Governance*, 2017.
- Werner et al. — *SoK: Decentralized Finance (DeFi)*, AFT 2022 (governance section).
- Posner & Weyl — *Radical Markets*, Princeton UP, 2018 (quadratic voting, Harberger).
- Buterin, Hitzig, Weyl — *A Flexible Design for Funding Public Goods*, 2019.
- Fritsch, Müller, Wattenhofer — *Analyzing Voting Power in Decentralized Governance*, 2022.
- *MakerDAO Whitepaper*, *Uniswap Governance docs*.

10.12 Lab (2 hrs)

1. Browse Tally.xyz. Pick one active proposal in a top-5 DAO; read it end-to-end.
 2. Plot a token's circulating supply over time + scheduled unlocks (use Token Unlocks data).
 3. Compute Compound's PE = FDV / annualized revenue from DefiLlama. Compare to Aave, Uniswap.
-

10.13 Quiz

1. Why is the Bitcoin SegWit/BCH split evidence that hashrate isn't sovereign?
2. State one weakness of token-weighted voting and one of quadratic voting.
3. Explain the veToken bribe market in one paragraph.
4. What did Beanstalk teach about flash-loan governance defense?
5. Why might fees $\neq$ revenue for an AMM?

Blockchain Module 11 — Advanced Consensus (DAGs, Sharding, HotStuff Family)

Goal: Move from textbook BFT to the 2023–2026 research frontier. After this module you can read SOSP/NSDI/PODC papers on consensus without translation.

11.1 The performance ceiling of leader-based BFT

Leader-based protocols (PBFT, HotStuff, Tendermint) have a fundamental ceiling:

$$\text{throughput} \leq \text{leader_bandwidth} / \text{block_size}$$

Because **one node** must disseminate the block per round. The 2020–2024 wave of DAG protocols breaks this by decoupling **transaction dissemination** from **ordering**.

11.2 DAG-based consensus

Narwhal & Bullshark (Facebook/Meta, then Sui/Aptos lineage)

- **Narwhal:** reliable, scalable mempool. Every validator runs its own "worker" lanes broadcasting batches. Produces a DAG of certified batches.
- **Bullshark:** a *zero-message-overhead* consensus on top of Narwhal. Total ordering is derived deterministically from the DAG structure.

Benchmarks: 130k+ TPS, sub-2-second finality.

Mysticeti (Sui, 2024)

Single-round commit, multi-leader DAG. Targets 300k+ TPS.

Aleph BFT (Aleph Zero)

Asynchronous DAG protocol, sub-second finality, randomized leader.

IOTA Tangle (historical)

A DAG where each new transaction validates two prior ones. Pioneered the DAG idea but had centralization issues (the Coordinator) that took years to resolve.

11.3 Sharding

Split state and execution across κ parallel chains.

Challenges

Problem	Description
Cross-shard communication	Atomic txs across shards require commit protocols
State fragmentation	A user's state may live across shards
Validator security	Smaller shards = easier to corrupt
Data availability	Each shard's data must be retrievable by anyone

Approaches

Project	Sharding flavor
Ethereum (original plan, then abandoned execution sharding)	64 shards
Ethereum (current)	Data sharding only (danksharding) — execution off-loaded to L2s
Near	Nightshade — dynamic resharding
Polkadot	Shared security via relay chain; parachains are heterogeneous shards
Cosmos	App-chain sovereignty + IBC; not "sharding" in the classic sense

Ethereum's pivot: instead of execution sharding, do **data sharding** so rollups can scale execution. The "rollup-centric roadmap".

11.4 HotStuff lineage

Classic HotStuff (Yin et al., 2019)

Three-phase chained voting → linear leader rotation.

LibraBFT / DiemBFT → Aptos (AptosBFT v4)

Extends HotStuff with reputation-based leader selection and pipelined sub-second finality.

Jolteon / Ditto (Meta)

Merges HotStuff happy-path linearity with PBFT's view-change for adversarial scenarios.

Carousel, Bullshark, Tusk, Shoal

Each iteration squeezes another constant factor out.

11.5 Asynchronous protocols

If you don't want to assume **any** synchrony, you need randomness.

Protocol	Trick
HoneyBadger BFT (Miller et al., 2016)	Threshold encryption + atomic broadcast
Dumbo	Reduces HBBFT round count
Tusk	Async DAG on top of Narwhal

Async protocols are slower in good conditions but never wedge.

11.6 Restaking & shared security as a consensus pattern

EigenLayer (2023) introduces a new design pattern: validators on chain A also validate AVSs (Actively Validated Services). The AVS's safety budget = restaked ETH at risk.

This blurs the line between consensus, security, and the application layer — a 2024–2026 research area.

Risks: AVS slashing definitions are heterogeneous; correlated failures across AVSs; centralization through restaking aggregators.

11.7 MEV-aware consensus

Active research:

- **Crypto-economic encrypted mempools** (Shutter, F3B, Themis)
- **Order-fairness consensus** (Aequitas, Themis, Pompê)
- **Proposer-Builder Separation (PBS)** with in-protocol enshrinement (ePBS, on Ethereum roadmap)
- **Inclusion lists** — proposer can force certain txs into the next block to defeat censorship

11.8 Cross-chain consensus

Approach	Example
Light client bridges	IBC (Cosmos), zkBridge
Shared sequencers	Espresso, Astria

Shared validity proving	Polygon AggLayer, EigenDA + Espresso
Universal settlement	Polygon Pos→AggLayer; OP Superchain shared bridge

The 2024–2026 trend: minimize trust through ZK light clients; minimize fragmentation through shared infrastructure.

11.9 Reading list

- Yin et al. — *HotStuff: BFT consensus with linearity and responsiveness*, PODC 2019.
- Danezis et al. — *Narwhal & Tusk: A DAG-based Mempool and Efficient BFT Consensus*, EuroSys 2022.
- Spiegelman, Giridharan, Sonnino, Kokoris-Kogias — *Bullshark*, CCS 2022.
- Miller et al. — *The Honey Badger of BFT Protocols*, CCS 2016.
- Kelkar et al. — *Order-Fair Consensus*, CRYPTO 2020.
- Buterin — *Endgame + PBS roadmap* posts.

11.10 Lab (2 hrs)

Pick one Bullshark / Mysticeti benchmark paper. Reproduce the throughput-vs-latency plot in a Python simulator (not actually networked — just simulate the message exchange). Discuss what assumptions made the original 130k TPS achievable.

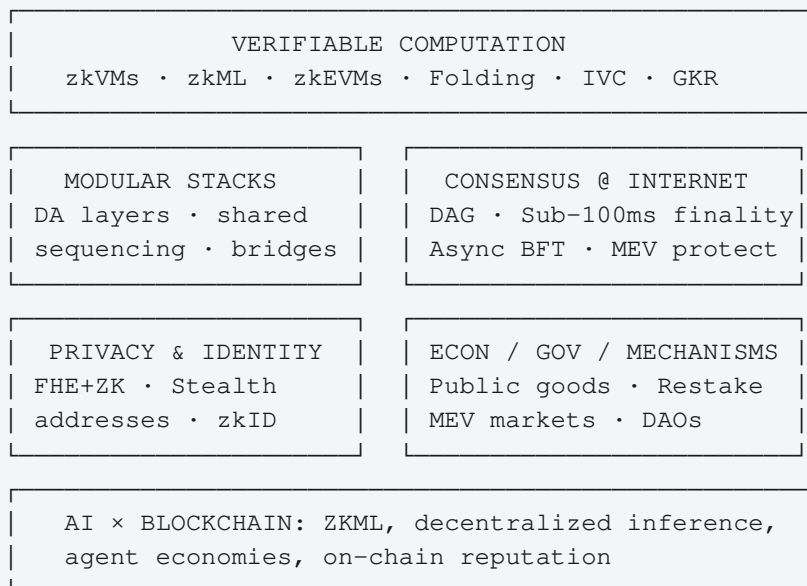
11.11 Quiz

1. Why do DAG-based protocols outperform leader-based BFT in throughput?
2. What did Ethereum trade away when it dropped execution sharding for data sharding?
3. State one tradeoff of asynchronous BFT protocols vs partially synchronous ones.
4. Describe one risk of EigenLayer's shared security model.
5. Why does MEV motivate consensus-level redesign and not just app-level fixes?

Blockchain Module 12 — Research Frontiers (2025–2027)

Goal: Identify what's actually new, where the citations are clustering, and which lines of work could plausibly become a Master's or PhD thesis.

12.1 Frontier map



12.2 Top open research questions

1. Practical zkML at scale

- Can we prove inference for >1B parameter LLMs in seconds with sub-\$1 cost?
- New: GKR-based proving (sumcheck), lookup arguments, folding schemes (Nova/HyperNova).

2. Stateless / minimally-stateful clients

- Verkle trees, history expiry (EIP-4444), light clients with full security.
- Ethereum's "The Verge" milestone.

3. Folding schemes & IVC

- Nova, SuperNova, HyperNova, ProtoStar, CycleFold.
- Goal: incrementally verifiable computation with $O(1)$ prover overhead per step.

4. Encrypted mempools and MEV defense

- Threshold encryption (Shutter), FHE-based, time-lock encryption.
- Tradeoffs: latency, liveness under failure.

5. Sovereign vs shared sequencing

- Single rollup sequencer = censorship risk.
- Shared sequencers (Espresso, Astria) trade composability for centralization risk.
- Active design space.

6. Account abstraction beyond ERC-4337

- Native AA (EIP-7702 path), recovery, MFA, social recovery, biometric signing.

7. Restaking risk modeling

- Correlated slashing across AVSs.
- What's the right capital-budget for slashable bonds across services?

8. Privacy-preserving compliance

- "Selective disclosure" — prove you're not OFAC-sanctioned without revealing identity.
- Aztec, Zcash deposit-screening, Sismo Connect.

9. Long-range and quantum resistance

- Post-quantum signature migration paths.
- Hash-based signatures (XMSS, SPHINCS+) and lattice schemes (CRYSTALS-Dilithium).

10. On-chain games & autonomous worlds

- Dark Forest (zk fog of war), MUD, Argus — game state as smart contract.
- Pushes ZK proving and gas optimization to extremes.

11. Decentralized AI infrastructure

- Compute marketplaces (Akash, Render, io.net).
- Model marketplaces and reputation (Bittensor, Ritual).
- See [intersection module](#).

12. Formal verification at scale

- Specs auto-generated from upgrades; ML-assisted invariant inference.

12.3 Venues to track

Venue	Tracks
IEEE S&P, USENIX Security, CCS, NDSS	Security
CRYPTO, EUROCRYPT, ASIACRYPT, TCC	Cryptography & ZK
PODC, DISC, OSDI, SOSP, NSDI	Distributed systems
AFT (Advances in Financial Technologies)	DeFi, mechanism design
FC (Financial Cryptography)	Applied crypto + DeFi
DSN	Dependability + BFT
NeurIPS / ICML / ICLR	ZKML, decentralized ML
ACM EC, WINE	Mechanism design, governance

Workshops worth submitting to as a first paper:

- DeFi @ CCS, ConsensusDay @ ESORICS, ZKProof workshop, Decentralizing Finance @ FC, FROST.

12.4 Toolchains worth building on

Tool	Why
Foundry (Solidity dev)	Industry standard test/fuzz framework
Risc Zero, SP1, Jolt	zkVMs — prove any Rust program
Circom, Noir, Halo2	Hand-written circuits
EigenLayer SDK	AVS development
Lambdaclass plonky3	Modern STARK toolkit
Hardhat / Tenderly	Mainstream Solidity tooling

12.5 How to pick a thesis-scale topic

1. Pick one of the 12 open questions above.
2. Read the **5 most-cited papers** of the last 18 months in that subarea.
3. Find a **clean primitive missing** or a **clean attack class undeveloped**.
4. Build the smallest possible empirical artifact (a benchmark, a measurement study, a circuit).
5. Submit to a workshop first. Iterate to a full venue.

The fastest first-paper recipe in blockchain research: **a measurement paper**. Empirical chain data is abundant; few researchers can both query archives and reason about protocol semantics.

12.6 Lab / capstone (open-ended)

Pick **one** of:

- Build a working zkML pipeline that proves an MNIST CNN forward pass with EZKL or Risc Zero. Measure proof time and verifier gas cost.
- Reproduce a benchmark from the Narwhal-Bullshark paper using their open-source code.
- Conduct a measurement of MEV-Boost relay behavior over 1 month of Ethereum blocks.
- Implement a stealth address scheme (EIP-5564) and write a usability analysis.

12.7 Quiz

1. Name two recent folding schemes and what they improve.
 2. What does Ethereum's "The Verge" milestone aim for?
 3. Why might shared sequencing recentralize what rollups decentralized?
 4. State one open question in zkML at scale.
 5. Why is *measurement* often the easiest first paper for a new researcher?
-

XAI Track — Overview

A 13-module sequence covering Explainable AI from foundations to the 2026 research frontier.

Modules

1. [Foundations: What Is Explainability?](#)
2. [Taxonomy of XAI Methods](#)
3. [LIME, SHAP & Feature Attribution](#)
4. [Gradient-Based Attribution](#)
5. [Concept-Based Explanations](#) — TCAV, ACE, CRAFT
6. [Counterfactual & Contrastive Explanations](#)
7. [Global Methods](#) — PDP, ICE, ALE, surrogates
8. [Attention & Transformer Interpretability](#)
9. [Mechanistic Interpretability](#)
10. [Evaluating Explanations](#)
11. [Human-Centered XAI](#)
12. [Domain Applications](#) — medicine, finance, law, NLP
13. [Research Frontiers \(2025–2027\)](#)

How to study this track

- Read in order. Modules 1–2 are conceptual; 3–9 are method-focused; 10–13 are evaluation and frontier.
- Do all six labs from `../labs/`.
- After every module, add 5–10 paper-cards (Zettelkasten-style) to your research notes.

What you will be able to do after

- Pick the right XAI method for any model + audience combination.
- Implement LIME, SHAP, IG, Grad-CAM, TCAV, and a counterfactual generator from scratch.
- Reproduce mechanistic-interpretability case studies (induction heads, IOI) on small models.
- Critically evaluate XAI papers — spot unfaithful methods, missing sanity checks, weak baselines.
- Identify a tractable open problem and write a workshop-paper-scale proposal.

Estimated time

- Casual (6 hrs/week): 24 weeks
- Focused (15 hrs/week): 10 weeks
- Full immersion (30 hrs/week): 5 weeks

XAI Module 1 — Foundations: What Is Explainability?

Goal: Get the vocabulary right. Most XAI confusion comes from conflating interpretability, explainability, transparency, justification, and fairness. They are not synonyms.

1.1 Why XAI exists as a field

Three forcing functions:

- 1. **Regulation** — GDPR Article 22 (right to explanation), EU AI Act high-risk categories, US Algorithmic Accountability Act drafts, India's DPDP Act.
- 2. **Safety** — deployed ML systems fail in non-obvious ways (medical misdiagnosis, biased lending, hallucination in LLMs).
- 3. **Science** — understanding why a model works is itself a research goal (Distill.pub, mechanistic interpretability).

DARPA's XAI Program (2017–2021) crystallized the field. Two major surveys: Adadi & Berrada (2018), Guidotti et al. (2018).

1.2 Definitions you must be able to recite

Term	Definition (working)
Interpretability	The degree to which a human can <i>understand the cause</i> of a decision. (Miller 2017)
Explainability	The degree to which a system can <i>produce explanations</i> — often via post-hoc methods.
Transparency	The model's mechanism is itself inspectable (linear models, small decision trees).
Simulatability	A human can mentally simulate the model's full computation.
Decomposability	Each part of the model has an intuitive meaning.
Algorithmic transparency	Understanding of how the learning algorithm produces a model.
Justification	Reasons that <i>defend</i> a decision, not necessarily its true cause.
Causality	The actual mechanism producing the output. (Pearl)

Critical: an "explanation" is not the same as the *cause*. LIME tells you what would have changed the output locally; it doesn't tell you the model's internal mechanism.

1.3 Lipton's framework (Lipton, 2016, "The Mythos of Model Interpretability")

A foundational paper. Lipton split "interpretability" into:

Properties of models (transparency)

- **Simulatability** — a person can mentally run the model.
- **Decomposability** — each parameter / node has a meaning.
- **Algorithmic transparency** — convergence is understood.

Properties of explanations (post-hoc)

- Text explanations
- Visualizations
- Local explanations
- Explanation by example

This split is the canonical taxonomy. Use it.

1.4 Doshi-Velez & Kim (2017) — evaluation framework

Level	What's evaluated	Cost
Application-grounded	Real users on real task	High
Human-grounded	Real users on simpler proxy task	Medium
Functionally-grounded	Formal definition of explanation quality	Low

Most papers operate at the *functionally-grounded* level (e.g., faithfulness scores). Papers that do application-grounded evaluation are scarce and disproportionately influential.

1.5 Who needs an explanation, and why?

Stakeholder	Need
End user	Understand a single decision affecting them
Domain expert (doctor, judge)	Validate model reasoning against domain knowledge
Developer / data scientist	Debug model, audit failures
Regulator / auditor	Establish accountability, check fairness
Researcher	Understand emergent behavior, science

Different stakeholders need *different* explanations. A SHAP plot is useless to a patient. A saliency map is useless to a regulator.

This is the **alignment-to-purpose** problem.

1.6 The accuracy–interpretability tradeoff (myth?)

The folk claim: more complex models → better accuracy but worse interpretability.

Cynthia Rudin's counter (2019, *Nature ML*): "*Stop explaining black-box ML models for high-stakes decisions and use interpretable models instead.*" On many tabular problems, well-tuned interpretable models (sparse logistic regression, EBMs, falling-rule lists) match black-box accuracy.

The tradeoff is empirically narrower than usually assumed, but **does exist for high-dimensional perceptual tasks** (vision, speech, language).

1.7 Interpretable model families (intrinsic)

Model	Why interpretable
Linear / logistic regression	Coefficients are weights
Generalized Additive Models (GAMs)	Each feature → its own learned curve
Explainable Boosting Machines (EBMs)	GAM + pairwise interactions, often as accurate as XGBoost
Decision trees (small)	Path through tree = rule
Decision rules / RIPPER / SLIM	If-then rules
Falling Rule Lists	Ordered if-then with monotone risk
Risk scores (e.g., 2HELPS2B for seizures)	Integer-coefficient scoring
Generalized linear rule models	Convex combinations of rules
Prototype networks	Predictions justified by similar training examples

A serious XAI engineer **tries an interpretable model first**, then escalates only if needed.

1.8 Post-hoc methods (preview)

For complex models where intrinsic interpretability is unavailable, post-hoc methods explain *after the fact*.

Method	Output
Feature attribution (LIME, SHAP, IG, gradient×input)	Importance score per feature

Saliency maps	Pixel importance per image
Concept-based (TCAV)	Importance of human concept
Counterfactuals	"What would change the prediction?"
Example-based	Nearest training examples
Surrogate models	A simpler model fit to the black box
Mechanistic	Direct circuit analysis (mech interp)

Detailed coverage in [Module 3](#) onwards.

1.9 The "right to explanation"

GDPR's Article 22 grants the right to "meaningful information about the logic involved" in automated decisions. Legal scholars debate whether this is a *right to explanation* or merely a *right to be informed*.

EU AI Act (2024) adds explicit transparency obligations for high-risk AI systems. India's DPDP Act, Canada's AIDA, US state laws all moving similar directions.

This pushes XAI from research curiosity to **compliance requirement**.

1.10 Common pitfalls (memorize)

1. **Confusing correlation explanations with causal ones.** SHAP tells you what the model uses, not what's true in the world.
2. **Trusting explanations of an unfit model.** If accuracy is poor, no explanation is meaningful.
3. **Aggregating local explanations into "global" claims** — disagreement is common (Krishna et al. 2022, "The Disagreement Problem").
4. **Using explanations as fairness audits.** Feature importance $\neq$ fairness; you need fairness metrics directly.
5. **Optimizing explanation methods for human-likeable outputs** rather than faithfulness.

1.11 Reading list

- Lipton — *The Mythos of Model Interpretability*, ICML WHI 2016.
- Doshi-Velez & Kim — *Towards a Rigorous Science of Interpretable ML*, 2017.
- Miller — *Explanation in Artificial Intelligence: Insights from the Social Sciences*, AI Journal 2019.
- Rudin — *Stop Explaining Black Box ML Models*, Nature MI 2019.
- Adadi & Berrada — *Peeking Inside the Black-Box*, IEEE Access 2018.
- Guidotti et al. — *A Survey of Methods for Explaining Black Box Models*, CSUR 2018.
- Molnar — *Interpretable Machine Learning* (free online book), Ch 1–3.

1.12 Lab (1 hr)

1. Fit a logistic regression and an XGBoost on the same tabular dataset (UCI Adult is fine).
2. Tabulate accuracy + a 3-sentence "explanation" you would give to (a) a regulator, (b) an end user, (c) a developer for each model.
3. Note which audience you struggled to serve.

1.13 Quiz

1. Distinguish *interpretability* and *explainability* in your own words.
2. State Rudin's argument against post-hoc explanations for high-stakes decisions.
3. List the three Doshi-Velez/Kim evaluation levels in order of rigor.
4. Why is "right to explanation" legally contested?
5. Give one example each of: an intrinsically interpretable model, a post-hoc local method, a post-hoc global method.

--	--

--	--	--

--	--

--	--

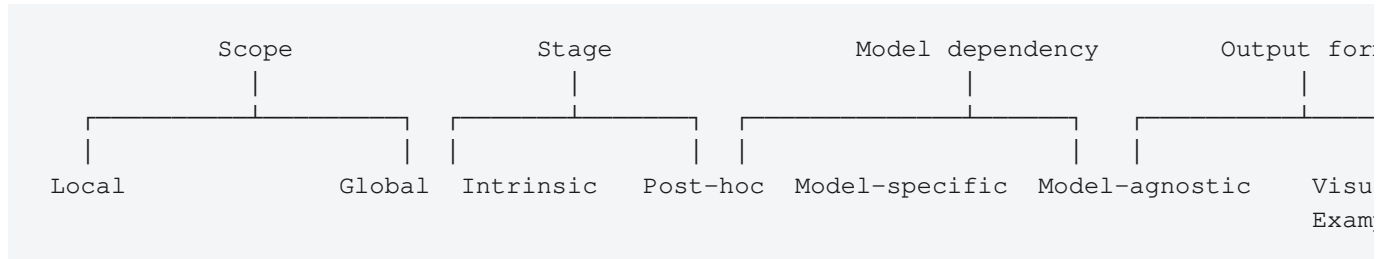
--	--

XAI Module 2 — Taxonomy of XAI Methods

Goal: Build a mental map that lets you place any new XAI paper within 30 seconds.

2.1 The four-axis taxonomy

Every XAI method can be classified along four axes:



A typical paper introduces a method that is e.g. **local + post-hoc + model-agnostic + visual** (LIME for images).

2.2 Scope: Local vs Global

Local explanations

Why did the model predict $\hat{y}$ for *this* input x ?

- **LIME** — locally fit a linear model around x .
- **SHAP (local)** — Shapley values for the single prediction.
- **Integrated Gradients** — attribution along path from baseline to x .
- **Counterfactuals** — minimum change to x that flips the prediction.

Global explanations

How does the model behave *overall*?

- **PDP / ICE / ALE** — marginal effect of a feature.
- **Feature importance** (permutation, gain-based).
- **Surrogate models** — fit a decision tree to the black box's predictions.
- **Concept Activation Vectors (TCAV)** — sensitivity to human-defined concepts.
- **Mechanistic interpretability** — circuit-level reverse-engineering.

Cohort explanations

Subsets between local and global — e.g., "Why does the model behave differently for women than men?"

Tools: SliceFinder, Aequitas, FairLearn.

2.3 Stage: Intrinsic vs Post-hoc

Type	Description	Example
Intrinsic	Model is interpretable by construction	Linear, GAM, EBM, small tree, prototype net
Post-hoc	Explain a trained black-box model	LIME, SHAP, Grad-CAM

Hybrid: Self-explaining neural networks (Alvarez-Melis & Jaakkola, 2018), ProtoPNet, Concept Bottleneck Models — *trained* to produce explanations, sit between the two.

2.4 Model dependency

Type	Works on
Model-agnostic	Any model (only needs predict()) — LIME, SHAP, PDP, permutation importance
Model-specific	Exploits internals — Grad-CAM (CNNs), Attention Rollout (Transformers), TreeSHAP (trees)

Model-specific methods are usually **faster and more faithful** to the model's actual computation; model-agnostic ones are **more portable**.

2.5 Output form

Form	When useful
Feature attribution (numeric per feature)	Tabular, NLP, CV
Saliency map (per pixel)	Vision
Attention heatmap	Transformer-based models
Concept score	High-level semantic reasoning
Counterfactual example	"What would change the prediction?"
Prototype / nearest example	Case-based reasoning
Natural-language rationale	LLMs, multimodal
Decision rule / tree	High-trust auditing

2.6 The disagreement problem (Krishna et al., 2022)

When you run multiple explanation methods on the same model and prediction, they often disagree.

Practical consequence: an analyst can pick the explanation that supports their narrative.

Implication: never rely on a single XAI method. Triangulate.

2.7 Evaluating an XAI method — three lenses

1. **Faithfulness** — does the explanation reflect what the model actually does?

- Perturbation-based tests (delete top-k features → measure accuracy drop)
- Sanity checks (Adebayo et al. 2018 — saliency maps that survive random-weight randomization are not faithful)

2. **Plausibility / human alignment** — does it make sense to humans?

- User studies
- Agreement with human-annotated rationales (ERASER benchmark)

3. **Robustness** — does the explanation change under small input perturbations?

- Adversarial XAI papers (Slack et al. 2020, "Fooling LIME and SHAP")

Detailed in [Module 10](#).

2.8 The XAI method tree (cheat sheet)

```
INTRINSIC
├─ Linear / Logistic / Ridge / Lasso
├─ GAM / EBM
├─ Decision trees (small)
├─ Decision rules (RIPPER, SLIM)
├─ Risk scores (integer-coef)
├─ Prototype networks (ProtoPNet, this-looks-like-that)
└─ Concept Bottleneck Models

POST-HOC
├─ Local
│   ├── LIME
│   ├── SHAP (KernelSHAP, TreeSHAP, DeepSHAP)
│   ├── Anchors
│   ├── Counterfactuals (Wachter, DiCE, FACE, MACE, GeCo)
│   ├── Gradient methods
│   │   ├── Vanilla saliency
│   │   ├── Integrated Gradients
│   │   ├── SmoothGrad
│   │   ├── Guided Backprop
│   │   └─ Grad-CAM / Grad-CAM++
│   └─ Influence functions
└─ Global
    ├── PDP / ICE / ALE
    ├── Permutation importance
    ├── Surrogate models (global tree, distillation)
    └─ Concept-based (TCAV, ACE, CRAFT)
```

- └ Mechanistic interpretability (probing, circuits)
- └ Functional ANOVA decomposition

EVALUATION

- └ Faithfulness (deletion / insertion, sufficiency, comprehensiveness)
- └ Sanity checks (model/data randomization)
- └ Stability / robustness
- └ Plausibility (human studies, ERASER)
- └ Run-time / computational cost

Print this. Stick it on your wall.

2.9 Reading list

- Adadi & Berrada — *Peeking Inside the Black-Box*, IEEE Access 2018.
- Guidotti et al. — *A Survey of Methods for Explaining Black Box Models*, CSUR 2018.
- Arrieta et al. — *Explainable AI (XAI): Concepts, Taxonomies, Opportunities and Challenges*, Information Fusion 2020.
- Krishna et al. — *The Disagreement Problem in Explainable ML*, 2022.
- Adebayo et al. — *Sanity Checks for Saliency Maps*, NeurIPS 2018.

2.10 Lab (1 hr)

Take last lab's XGBoost model. Run:

- Permutation importance
- TreeSHAP global
- PDP for top-3 features
- One LIME explanation

Note where they agree and disagree.

2.11 Quiz

1. Place SHAP for one prediction on all four axes.
2. Why is Grad-CAM model-specific?
3. Define faithfulness in one sentence.
4. Why does the disagreement problem matter for compliance?
5. Give two examples of intrinsic-interpretable models that can match black-box accuracy on tabular tasks.

XAI Module 3 — LIME, SHAP, and the Feature Attribution Family

Goal: Master the two most-cited XAI methods. Be able to implement both from scratch in ~100 lines of Python each.

3.1 LIME — Local Interpretable Model-agnostic Explanations

Ribeiro, Singh, Guestrin — KDD 2016. **The first widely-adopted post-hoc method.**

Idea

For a single instance x , fit a simple, interpretable surrogate model g (e.g., sparse linear) in the local neighborhood of x such that g approximates f there. The surrogate's coefficients become the explanation.

Algorithm

```
Input: model  $f$ , instance  $x$ , num_samples  $N$ 
1. Sample  $N$  perturbations  $x'_i$  in the neighborhood of  $x$ .
   - Tabular: perturb features around  $x$ .
   - Text: randomly delete words.
   - Image: superpixels (SLIC) toggled on/off.
2. Compute model predictions  $y'_i = f(x'_i)$ .
3. Weight each sample by a kernel  $\pi(x, x'_i)$  — closer to  $x \rightarrow$  higher weight.
4. Fit a linear model  $g$  on  $(x'_i, y'_i)$  weighted by  $\pi$  and with sparsity (L1).
5. Return  $g$ 's coefficients as the explanation.
```

Strengths

- Truly model-agnostic.
- Intuitive output (sparse linear).

Weaknesses

- **Instability** — different random seeds give different explanations (Alvarez-Melis & Jaakkola, 2018).
 - **Choice of neighborhood** is arbitrary and influences results.
 - **Adversarial vulnerability** — Slack et al. (2020) showed a model can be made to lie to LIME while still being biased.
-

3.2 SHAP — SHapley Additive exPlanations

Lundberg & Lee — NeurIPS 2017. **The single most-cited XAI method.**

Idea

Borrow Shapley values from cooperative game theory. For each feature, ask: "What is its marginal contribution to the prediction, averaged over all possible coalitions of other features?"

Shapley value formula

For player i in game v with N players:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \left[\frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} \right] \cdot [v(S \cup \{i\}) - v(S)]$$

In ML: $v(S) = E[f(X) \mid X_S = x_S]$ — expected prediction conditioned on the features in S being fixed at x 's values.

Why Shapley values matter

They are the **unique** allocation satisfying:

1. **Efficiency** — $\sum \phi_i = f(x) - E[f(X)]$ (attributions sum to the gap from baseline)
2. **Symmetry** — symmetric features get equal credit
3. **Dummy** — irrelevant features get 0
4. **Linearity** — for two games combined, attributions add

This axiomatic uniqueness is what gives SHAP its theoretical edge over LIME.

Variants

Variant	Speed	Faithfulness
KernelSHAP	Slow (model-agnostic via weighted regression)	Good
TreeSHAP	Fast (exact, exploits tree structure)	Exact for tree ensembles
DeepSHAP / Deep LIFT	Medium (chain rule with reference)	Good for NNs
PartitionSHAP	Medium (hierarchical Owen values)	Useful for high-dim features
FastSHAP	Very fast (amortized via a learned predictor)	Good after training

Conditional vs marginal expectation

- **Marginal** ($E[f \mid \text{do}(X_S = x_S)]$) — break dependence, like in interventional causal inference.
- **Conditional** ($E[f \mid X_S = x_S]$) — preserve dependence.

Aas et al. (2021) showed marginal SHAP can credit features that aren't in the model. Use conditional when features are correlated. Most libraries default to marginal — beware.

3.3 SHAP visualizations to know

- **Force plot** — per-prediction, additive.
- **Waterfall plot** — per-prediction, sequential.
- **Summary / beeswarm plot** — global, distribution of attributions across data.
- **Dependence plot** — feature value vs SHAP value with color = interacting feature.
- **Decision plot** — multi-prediction comparison.

3.4 LIME vs SHAP — when to use which

Use case	Pick
Tree ensemble (XGBoost, LightGBM, RF)	TreeSHAP (fast, exact)
Black box, prototype phase	LIME (faster setup)
Need theoretical guarantees	SHAP
Need to explain text or images	Either, but consider gradient methods too
Tight compute budget	LIME
Compliance documentation	SHAP (axiomatic story is easier to defend)

3.5 Anchors — Ribeiro et al. 2018

A rule-based local explanation: "Anchor" = a set of feature predicates such that, whenever those predicates hold, the model's prediction is the same with high probability (e.g., precision ≥ 0.95).

```
IF age > 50 AND education = "Bachelor" THEN predict = "income > 50K" (precision 0.97,
```

Pros: high-precision, easy to read. Cons: anchors may not exist for all instances; coverage trade-off.

3.6 DeepLIFT / Layer-wise Relevance Propagation (LRP)

Both methods backpropagate attributions through a neural network using a reference / baseline input, distributing credit to inputs.

- **DeepLIFT** (Shrikumar et al., 2017) — chain rule with multipliers relative to reference.
- **LRP** (Bach et al., 2015) — propagate relevance backward via custom rules per layer.

Both are precursors to Integrated Gradients and underlie DeepSHAP.

3.7 Influence functions — Koh & Liang (2017)

A different kind of explanation: *which training points most influenced this prediction?*

Approximates leave-one-out retraining without retraining, via implicit Hessian-vector products.

Useful for debugging mislabeled data, finding adversarial training examples, and credit attribution.

Computationally expensive for large models; scalable variants (Tracln, Datamodels) are active research.

3.8 Implementing KernelSHAP in 60 lines (sketch)

```
import numpy as np
from itertools import combinations
from math import comb

def kernel_weight(M, s):
    if s == 0 or s == M: return 1e6 # large weight, edge case
    return (M - 1) / (comb(M, s) * s * (M - s))

def kernel_shap(f, x, background, num_samples=2000):
    M = len(x)
    masks, weights, ys = [], [], []
    for _ in range(num_samples):
        z = np.random.randint(0, 2, M)
        x_masked = np.where(z, x, background.mean(0))
        masks.append(z)
        weights.append(kernel_weight(M, z.sum()))
        ys.append(f(x_masked))
    Z = np.array(masks); w = np.array(weights); y = np.array(ys)
    # Weighted linear regression with intercept = E[f]
    base = f(background).mean()
    y_centered = y - base
    W = np.diag(w)
    phi = np.linalg.lstsq(Z.T @ W @ Z, Z.T @ W @ y_centered, rcond=None)[0]
    return phi # one Shapley value per feature
```

This is enough to convince yourself the math works. Production SHAP is much more optimized.

3.9 Reading list

- Ribeiro, Singh, Guestrin — *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, KDD 2016.
- Lundberg & Lee — *A Unified Approach to Interpreting Model Predictions*, NeurIPS 2017.
- Lundberg et al. — *From local explanations to global understanding with explainable AI for trees*, Nature MI 2020.
- Ribeiro, Singh, Guestrin — *Anchors: High-Precision Model-Agnostic Explanations*, AACL 2018.
- Shrikumar, Greenside, Kundaje — *Learning Important Features Through Propagating Activation Differences (DeepLIFT)*, ICML 2017.
- Aas, Jullum, Løland — *Explaining individual predictions when features are dependent*, 2021.
- Slack et al. — *Fooling LIME and SHAP*, AIES 2020.
- Koh & Liang — *Understanding Black-box Predictions via Influence Functions*, ICML 2017.

3.10 Lab (3 hrs)

1. Implement KernelSHAP from scratch (use the sketch above as starter).
2. Verify it matches `shap` library output on 5 examples from a small XGBoost model.
3. Run TreeSHAP on the same model; compare runtime.
4. Implement an "adversarial" model that detects when it's being explained and returns a different (benign) function (per Slack et al.).

3.11 Quiz

1. State the four Shapley axioms.
2. Why is TreeSHAP exact and KernelSHAP only approximate?
3. Explain marginal vs conditional SHAP and when each is appropriate.
4. How does LIME's neighborhood choice affect explanations?
5. What is an "anchor" and when does it not exist?

XAI Module 4 — Gradient-Based Attribution

Goal: Master the saliency / gradient family — the workhorse of computer-vision XAI.

4.1 The core intuition

If you change pixel x_i by a tiny amount, how much does the output $f(x)$ change?

That's the partial derivative $\partial f / \partial x_i$. Gradient-based methods are variations on this single idea, fixing its (many) flaws.

4.2 Vanilla saliency (Simonyan et al., 2013)

$$\text{saliency}(x_i) = \left| \partial f_c(x) / \partial x_i \right|$$

Visualize as a heatmap over the input image. Simplest possible attribution.

Problems

- Noisy — gradients fluctuate wildly across nearby inputs.
 - Saturation — for ReLU networks, large activations zero out the gradient even when the feature matters.
 - Doesn't distinguish *what* makes the prediction (positive vs negative evidence).
-

4.3 SmoothGrad (Smilkov et al., 2017)

Average saliency over noisy copies:

$$\text{SmoothGrad}(x) = (1/N) \cdot \sum \text{saliency}(x + \epsilon_i), \epsilon_i \sim N(0, \sigma^2)$$

Visually much cleaner. Essentially a denoising of vanilla saliency.

4.4 Integrated Gradients (Sundararajan, Taly, Yan — ICML 2017)

Solve the saturation problem by integrating along a straight path from a baseline x' to input x :

$$\text{IG}_i(x) = (x_i - x'_i) \cdot \int_{\alpha=0}^1 \partial f(x' + \alpha(x - x')) / \partial x_i \, d\alpha$$

Approximated by Riemann sum with ~50 steps.

Why IG matters

Satisfies two axioms:

1. **Sensitivity** — if changing a feature changes prediction, attribution is nonzero.
2. **Implementation invariance** — two functionally-equivalent networks yield identical attributions.

Vanilla saliency violates Sensitivity (saturation). Pre-IG methods (DeConvNet, Guided Backprop) violate Implementation Invariance.

Choice of baseline matters

Baseline	Use case
All-zeros image	Default but problematic for "black" matters
Random noise	Hides biases
Blurred version of x	"What changed from blurry-known to detailed-classified"
Mean of training set	Centroidal
Multiple baselines, averaged	Most robust (Expected Gradients)

Expected Gradients (Erion et al., 2021) averages over a distribution of baselines — combines IG with SHAP's expectation.

4.5 Guided Backprop & DeConvNet

Modify the backward pass through ReLU to suppress negative gradients. Visually nice but fails sanity checks (Adebayo et al., 2018) — they produce similar-looking saliency maps **even when the model is randomly initialized**.

Conclusion: Guided BP is essentially an edge detector, not a faithful explanation.

4.6 Grad-CAM (Selvaraju et al., ICCV 2017)

Compute the gradient of the target class score with respect to the activations of the **last conv layer**:

$$\alpha_k = (1/Z) \cdot \sum_{i,j} \partial y_c / \partial A^k_{i,j}$$
$$\text{Grad-CAM} = \text{ReLU}(\sum_k \alpha_k \cdot A^k)$$

Produces a low-resolution (e.g., 7×7) heatmap upsampled to the input. **The most-used saliency method in practice.**

Variants

- **Grad-CAM++** — better localization when multiple instances of the class are present.
 - **Score-CAM** — replaces gradients with forward-pass weights (more faithful).
 - **Eigen-CAM** — uses the first principal component of activation maps.
 - **HiResCAM** — corrects a small flaw in Grad-CAM's element-wise multiplication.
-

4.7 Layer-wise Relevance Propagation (LRP)

A backward propagation rule for redistributing the prediction relevance through the network. Different "rules" (LRP- ϵ , LRP- γ , LRP- $\alpha\beta$) work best for different layer types.

Most popular for clinical/medical imaging because it's deterministic and faithful by construction (relevance conservation across layers).

4.8 Sanity checks (Adebayo et al., NeurIPS 2018)

A paper everyone should read once.

Two tests:

1. **Model randomization test** — re-initialize weights from the top layer down. A faithful explanation should change drastically.
2. **Data randomization test** — train on permuted labels. A faithful explanation should produce different attributions than the correctly-trained model.

Result: Guided Backprop and Guided Grad-CAM **fail** both tests. Integrated Gradients and vanilla saliency pass model-randomization but only partially data-randomization.

Takeaway: a pretty heatmap may be edge-detection, not explanation.

4.9 Attribution for Transformers

Gradient methods generalize to transformers but care is needed:

- Saliency works token-wise.
- Integrated Gradients with token embedding baselines.
- **Attention Rollout** (Abnar & Zuidema 2020) — multiply attention matrices across layers.
- **Attention Flow** — max-flow through attention graph.
- **Chefer et al.** (2021) — generic Transformer attribution combining gradients + attention.

(More in [Module 8](#).)

4.10 The IG family today

The 2024–2026 SOTA cluster:

- **Path-Integrated Gradients with optimal paths**
 - **Bias-Free Integrated Gradients**
 - **GAM (Guided Attention Maps)** for ViTs
 - **CAFE / TracIn-Influence** for input + training attribution
-

4.11 Reading list

- Simonyan, Vedaldi, Zisserman — *Deep Inside Convolutional Networks: Visualising Image Classification Models*, 2013.
- Sundararajan, Taly, Yan — *Axiomatic Attribution for Deep Networks (Integrated Gradients)*, ICML 2017.
- Smilkov et al. — *SmoothGrad: removing noise by adding noise*, 2017.
- Selvaraju et al. — *Grad-CAM*, ICCV 2017.
- Bach et al. — *On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation*, PLoS ONE 2015.
- Adebayo et al. — *Sanity Checks for Saliency Maps*, NeurIPS 2018.
- Erion et al. — *Improving performance of deep learning models with axiomatic attribution priors and expected gradients*, Nature MI 2021.

4.12 Lab (3 hrs)

1. Use a pretrained ResNet-50 on an ImageNet image.
2. Implement and visualize: vanilla saliency, SmoothGrad, Integrated Gradients (50 steps), Grad-CAM.
3. Run Adebayo's randomization test on each method. Report which survive.
4. Repeat the IG step with three different baselines. Discuss differences.

4.13 Quiz

1. What does the "implementation invariance" axiom rule out?
2. Why does Grad-CAM produce a low-resolution map?
3. State the SmoothGrad equation.
4. Explain in one sentence why Guided Backprop fails the sanity check.
5. Why is baseline choice central to IG's interpretation?

XAI Module 5 — Concept-Based Explanations

Goal: Move from pixel-level attribution to *human-meaningful concepts* (e.g., "stripes", "wheel", "femininity in a face").

5.1 The motivation

Pixel-level saliency tells you *where*, not *what*. A doctor wants "the model flagged this because of irregular cell shape", not "pixel (243, 187) was important".

Concept-based methods answer **what concept drove the prediction**.

5.2 TCAV — Testing with Concept Activation Vectors

Kim, Wattenberg, Gilmer, Cai, Wexler, Viegas, Sayres — ICML 2018.

Idea

1. **Define a concept** by collecting a set of positive examples (e.g., 50 striped images) and random negatives.
2. **Compute activations** at a chosen layer l for both sets.
3. **Train a linear classifier** to separate concept vs random in activation space → the normal to this hyperplane is the **Concept Activation Vector (CAV)**.
4. **Compute directional derivatives** of the target class prediction with respect to the CAV.
5. **TCAV score** = fraction of input samples for which the directional derivative is positive.

$$S_{\{C,k,l\}}(x) = \partial_{h_{\{l,k\}}(f_l(x))} / \partial C \quad (\text{directional derivative})$$
$$\text{TCAV}_{\{C,k,l\}} = \left| \{x \text{ in } X_k : S_{\{C,k,l\}}(x) > 0\} \right| / \left| X_k \right|$$

A statistical significance test (vs random concepts) rules out spurious results.

Why TCAV matters

- Uses human-understandable concepts, not pixels.
- Works for any pretrained model.
- Established a research line that includes ACE, CRAFT, Network Dissection, Concept Bottleneck Models.

Limits

- Requires curated concept datasets.
- Concept linear-separability assumption.
- Can suffer from concept entanglement.

5.3 ACE — Automated Concept Extraction

Ghorbani, Wexler, Zou, Kim (NeurIPS 2019).

Automates the "where do concepts come from" step:

1. Segment images into superpixels.
2. Cluster segments in activation space.
3. Treat each cluster centroid as an auto-discovered concept.
4. Run TCAV against it.

ACE finds concepts the researcher might not have thought to define.

5.4 CRAFT — Fel et al. (CVPR 2023)

Combines:

- Recursive non-negative matrix factorization (NMF) of activations
- Per-region attribution
- Per-region concept attribution

Output: a hierarchical decomposition of the prediction by concept, with localization. State of the art for concept-based CV explanations as of 2024.

5.5 Network Dissection (Bau, Zhou, Khosla, Oliva, Torralba — CVPR 2017)

For each conv filter, find the human-labeled concept (from the Broden dataset: objects, parts, materials, textures, scenes) it best aligns with. Result: assign filters to concepts.

Shows that some networks have surprisingly "monosemantic" units even without being trained for it.

Inspired modern **mechanistic interpretability** (see [Module 9](#)).

5.6 Concept Bottleneck Models (Koh et al., ICML 2020)

Train the model in two stages:

$$x \rightarrow C(x) = \text{predicted concepts} \rightarrow \hat{y}$$

The concept layer is **explicit** — humans can inspect and even *intervene* on it ("test-time concept editing"). If the model says "this bird has white belly + curved beak" and you know that's wrong, you can correct it before the final prediction.

Trade-off: requires concept labels for training; accuracy gap if concepts are insufficient.

Post-hoc CBMs (Yuksekgonul, Wang, Zou — NeurIPS 2022) — convert any trained model into a CBM by fitting a concept layer on its features. Much cheaper.

5.7 Prototype networks — "this looks like that"

Chen et al. — *This Looks Like That: Deep Learning for Interpretable Image Recognition*, NeurIPS 2019.

ProtoPNet: the network learns a set of prototypes per class. Predictions are explained as "this part of input looks like this prototype of training class X".

Visually beautiful, used in medical imaging. Variants: ProtoTree, Deformable ProtoPNet, ProtoPool.

5.8 Concept Bottleneck for LLMs

Active 2024–2026 area:

- **Concept embeddings** for transformer features.
- **Sparse Autoencoders (SAEs)** decompose activations into thousands of monosemantic concepts (Anthropic, Bricken et al. 2023; Cunningham et al. 2024; Templeton et al. 2024 — *Scaling monosemanticity*).
- Probing classifiers find what concepts are encoded at each layer.

This is the most active research thread in XAI right now — see [Module 9](#).

5.9 Limits & critiques

- **Concept arbitrariness** — different researchers pick different concept sets.
 - **Concept leakage** — concept labels may encode unintended info (e.g., demographics).
 - **Faithfulness** — concept predictors may not reflect what the underlying model actually uses (Margeloiu et al., 2021).
 - **Concept entanglement** — single filters often respond to multiple concepts.
-

5.10 Reading list

- Kim et al. — *Interpretability Beyond Feature Attribution: Quantitative Testing with CAVs (TCAV)*, ICML 2018.
- Ghorbani et al. — *Towards Automatic Concept-based Explanations (ACE)*, NeurIPS 2019.
- Fel et al. — *CRAFT: Concept Recursive Activation FacTorization*, CVPR 2023.
- Bau et al. — *Network Dissection*, CVPR 2017.
- Koh et al. — *Concept Bottleneck Models*, ICML 2020.
- Chen et al. — *This Looks Like That*, NeurIPS 2019.

- Bricken et al. — *Towards Monosemanticity*, Anthropic 2023.
- Templeton et al. — *Scaling Monosemanticity*, Anthropic 2024.

5.11 Lab (3 hrs)

1. Pick a small ImageNet subset.
2. Build a "stripes" concept set (50 images of stripes) and a random negative set.
3. Compute the CAV at the last conv layer of ResNet-50.
4. Run TCAV for class "zebra" — verify that "stripes" matters; check significance vs random concepts.
5. Repeat for a class where "stripes" should *not* matter (e.g., "spaniel").

5.12 Quiz

1. Why is TCAV more useful than saliency for clinical communication?
2. What does the TCAV score quantify and how is significance tested?
3. State two differences between standard CBMs and post-hoc CBMs.
4. How does ProtoPNet explain a prediction?
5. What does the "monosemanticity" research line aim to demonstrate?

XAI Module 6 — Counterfactual & Contrastive Explanations

Goal: Master the "what-if" family of explanations — arguably the most actionable form of XAI.

6.1 The contrastive idea

People naturally explain by contrast: "I was denied the loan because my income was too low" implicitly says "if my income had been \$X higher, I would have been approved".

Counterfactual explanations operationalize this. They are:

- **Actionable** ("change feature X to Y")
 - **Aligned with how humans actually explain** (Miller, 2019)
 - **Stakeholder-friendly** (no model internals required)
-

6.2 Wachter, Mittelstadt, Russell (2017) — the canonical formulation

"What is the smallest change to x that would change the prediction to y' ?"

$$x_{cf} = \operatorname{argmin}_{x'} d(x, x') + \lambda \cdot L(f(x'), y')$$

Where:

- $d(x, x')$ = distance (often weighted L1 on normalized features)
- $L(f(x'), y')$ = loss pushing prediction to desired class

Optimized via gradient descent for differentiable models or genetic/discrete search for non-differentiable ones.

Desiderata

A good counterfactual is:

1. **Valid** — actually flips the prediction.
2. **Proximate** — close to x in input space.
3. **Sparse** — changes few features.
4. **Actionable** — only changes mutable features (can't change race, age).
5. **Plausible** — lies on the data manifold (don't recommend "earn \$1,000,000 next month").
6. **Diverse** — multiple distinct paths to flip the decision.

6.3 DiCE — Diverse Counterfactual Explanations

Mothilal, Sharma, Tan — FAT* 2020.

Generates K counterfactuals that are simultaneously:

- Valid
- Close to x
- Mutually **diverse** (via DPP / determinantal-point-process-like regularizer)
- Respect feature constraints (mutable / immutable, monotonic)

Library: `dice-ml`. Widely used in finance / lending.

6.4 FACE — Feasible and Actionable Counterfactual Explanations

Poyiadzi et al. — AIES 2020.

Builds a graph over training data; the counterfactual is the shortest path from x to a flipping region through high-density regions. Ensures **plausibility** by construction.

6.5 GeCo — Genetic Counterfactuals

Searches counterfactual space via genetic algorithm with mutation, crossover, and constraint handling. Works for arbitrary black-box models including non-differentiable trees.

6.6 MACE — Model-Agnostic Counterfactual Explanations via SAT/SMT

Karimi et al. — FAT* 2020.

Encode the model + counterfactual desiderata as a SAT/SMT problem. Provably optimal counterfactuals for piecewise-linear models (trees, ReLU nets).

6.7 Algorithmic recourse — the causal turn

Karimi, Schölkopf, Valera (FAccT 2021) point out: classical counterfactuals ignore causal structure. "Change your education from BS to PhD" isn't a single intervention — it requires years and money.

Recourse = the **actions** required to achieve the counterfactual, accounting for causal effects.

```
intervention I → features x' (downstream causal effects propagate) → prediction f(x')
```

Requires a causal graph (often unavailable in practice — see "recourse under uncertain causal knowledge").

6.8 Contrastive explanations (pertinent positives / negatives)

Dhurandhar et al. — NeurIPS 2018, "Explanations based on the Missing".

Two halves of any prediction:

- **Pertinent positives (PP)** — minimal subset of features that *sufficiently* causes the prediction.
- **Pertinent negatives (PN)** — features whose absence is necessary (their addition would flip).

A complete contrastive explanation: "Prediction is X because of PP, and not Y because of PN."

6.9 Counterfactuals for NLP

Method	Approach
Polyjuice (Wu et al. 2021)	GPT-fine-tuned counterfactual generator
CheckList (Ribeiro et al. 2020)	Templated minimal pairs for behavioral testing
MICE (Ross et al. 2021)	Minimal contrastive edits via Levenshtein
CAT	LLM-as-editor for counterfactual NLP

Used to find spurious correlations (e.g., model predicts "positive" for *any* sentence with the word "amazing").

6.10 Counterfactuals for images

Hard problem — pixel space is high-dimensional and "small change" is meaningless without a manifold prior.

Approach	Method
Latent-space (StyleGAN inversion)	Edit w vector → decode
Diffusion-based	Counterfactual via guided diffusion
Saliency-region edits	Inpaint salient region differently
Concept-aware (e.g., StyleEx, Lang et al. 2021)	GAN with classifier-aligned latent directions

6.11 Counterfactuals as compliance tools

Wachter et al. argued counterfactuals satisfy GDPR's right-to-explanation. They don't reveal model internals (good for trade secrets) but give the user actionable information.

Adopted in:

- Credit denial letters (US ECOA adverse-action notices)
- Algorithmic hiring tools
- Insurance underwriting

6.12 Failures and critiques

- **Manifold off-distribution** — gradient-descent counterfactuals often produce inputs no human would recognize.
- **Adversarial-like artifacts** — minimum-change counterfactuals can be adversarial perturbations.
- **Causal naivety** — ignoring causal relations gives unactionable advice.
- **Fairness** — counterfactual recommendations can be discriminatory if mutable features differ across groups (e.g., harder for some groups to "increase income").
- **Multiplicity** — many equally-good counterfactuals exist; which do you show?

6.13 Reading list

- Wachter, Mittelstadt, Russell — *Counterfactual Explanations Without Opening the Black Box*, Harvard JOLT 2017.
- Mothilal, Sharma, Tan — *Explaining Machine Learning Classifiers through Diverse Counterfactual Explanations (DiCE)*, FAT* 2020.
- Poyiadzi et al. — *FACE: Feasible and Actionable Counterfactual Explanations*, AIES 2020.
- Karimi et al. — *A Survey of Algorithmic Recourse*, ACM Computing Surveys 2022.
- Dhurandhar et al. — *Explanations based on the Missing*, NeurIPS 2018.
- Ross, Marasović, Peters — *Explaining NLP Models via Minimal Contrastive Editing*, ACL 2021.
- Verma, Dickerson, Hines — *Counterfactual Explanations and Algorithmic Recourse for Machine Learning: A Review*, 2024.

6.14 Lab (3 hrs)

1. Train a logistic regression on UCI Adult (income $\geq$ \$50k).
2. For 10 denied applicants, generate counterfactuals using:
 - Vanilla Wachter (gradient)
 - DiCE
3. Audit: which features did each method change? Are they mutable / actionable?
4. Discuss fairness — do counterfactuals for women differ systematically from men's?

6.15 Quiz

1. State Wachter's objective in math.

2. Distinguish counterfactual explanation from algorithmic recourse.
3. Why is plausibility hard to enforce via gradient descent alone?
4. Define pertinent positives and pertinent negatives.
5. Why might counterfactuals discriminate even when the model itself is "fair"?

XAI Module 7 — Global Methods: PDP, ICE, ALE, Surrogates, Permutation Importance

Goal: Master the global / cohort-level methods for tabular ML — the bread and butter of every applied XAI engagement.

7.1 Partial Dependence Plots (PDP)

Friedman, 2001.

For feature(s) s , marginalize out the rest:

$$\text{PDP}_S(x_S) = (1/N) \sum_i f(x_S, x_{-i}^{\{C\}})$$

For each value of x_S , average the prediction over the dataset, holding x_{-S} fixed.

Visualization

- 1D feature → line plot.
- 2D pair → heatmap (shows interactions).

Limits

- **Assumes feature independence** — if x_S is correlated with x_C , the marginalization creates impossible synthetic data points (a 6-foot 60-pound person).
 - Only shows **average** effect — hides heterogeneity.
-

7.2 ICE — Individual Conditional Expectation

Goldstein et al., 2015.

Instead of averaging, plot **one line per instance**: how does *this person's* prediction change as we vary x_S for them?

PDP = average of all ICE curves.

When ICE curves diverge significantly, the PDP is misleading and you have heterogeneous effects.

7.3 ALE — Accumulated Local Effects

Apley & Zhu, 2020.

Solves PDP's correlation problem.

```
For interval k of feature x_j:  
  compute local effect = E[ f(x_j = upper) - f(x_j = lower) | x_j in interval k ]  
Accumulate and center across intervals.
```

ALE conditions on the actual data distribution, so it doesn't fabricate impossible samples. **Should be the default for correlated features.**

Method	Handles correlation?	Computational cost
PDP	No	Low
ICE	No	Low
ALE	Yes	Low
SHAP global	Conditionally yes	Medium

7.4 Permutation Feature Importance

Breiman, 2001 (random forests); model-agnostic since.

```
Importance(j) = error(f, X_permuted_j, y) - error(f, X, y)
```

Shuffle one feature's column, measure performance drop. Bigger drop = more important feature.

Pitfalls

- Same correlation issue (creates impossible data points).
- Importance is **with respect to the loss**, not the prediction — interpret accordingly.
- High-cardinality features can be over-credited (use grouped permutation).

Conditional permutation importance fixes some of these by shuffling within strata.

7.5 Global SHAP

Aggregate local SHAP values across the dataset:

- Mean(|SHAP|) per feature → global importance
- Summary / beeswarm plot → distribution
- Dependence plots → effect across the data

This is "the SHAP plot" you see in 90% of Kaggle kernels.

7.6 Surrogate models

Train a simpler, interpretable model to mimic the black box.

Global surrogate

```
f_black_box(x) → ŷ  
g_interpretable(x) → ŷ (regress g on f's predictions over a large sample)
```

If g 's $R^2 \approx 1$, you can use g as the explanation. Otherwise, g 's explanations are unreliable.

Knowledge distillation

A specific form of global surrogate where a *smaller neural net* mimics a larger one. Hinton et al., 2015. Not strictly XAI but related.

Soft Decision Trees (Frosst & Hinton, 2017)

Trees with stochastic routing that can match a neural net while remaining traceable.

7.7 Functional ANOVA decomposition

Decompose $f(x) = f_{\emptyset} + \sum f_i(x_i) + \sum f_{\{ij\}}(x_i, x_j) + \dots$ into main effects + interactions.

EBMs do this exactly — and that's why they tend to be both accurate and interpretable on tabular data.

7.8 H-statistic for interactions

Friedman & Popescu, 2008. Measures whether feature pair (j, k) interacts beyond their additive effects.

$$H^2_{jk} = \frac{\sum_i [PDP_{jk}(x_i) - PDP_j(x_i) - PDP_k(x_i)]^2}{\sum_i PDP_{jk}(x_i)^2}$$

Useful when you suspect interactions are driving the model.

7.9 Anchors at the global level

Aggregate Anchors across the dataset to surface high-precision rule patterns the model uses.

7.10 Functional cohorts — SliceFinder

Chung, Polyzotis, Tae, Whang (2019) — automatically find data slices where the model performs worst. Critical for fairness audits.

```
slice = {x : age > 60 AND device = "android"}
if loss on slice >> overall loss → investigate
```

Tools: Slice Finder, Aequitas, FairLearn, Robustness Gym.

7.11 When to use which

Task	Use
Quick global feature rank	Permutation importance or global SHAP
Visualize a single feature's effect	PDP + ICE (small dataset), ALE (large/correlated)
Inspect interaction	PDP-2D or H-statistic
Build an explanation for stakeholders	Surrogate tree
Audit subgroup performance	SliceFinder + Aequitas
Need axiomatic guarantees	SHAP global

7.12 Reading list

- Friedman — *Greedy Function Approximation: A Gradient Boosting Machine*, Annals 2001 (PDP section).
- Goldstein et al. — *Peeking Inside the Black Box: Visualizing Statistical Learning with ICE*, JCGS 2015.
- Apley & Zhu — *Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models (ALE)*, JRSS B 2020.
- Breiman — *Random Forests*, ML 2001 (permutation importance).
- Friedman & Popescu — *Predictive Learning via Rule Ensembles*, AOAS 2008.
- Chung et al. — *Slice Finder: Automated Data Slicing for Model Validation*, ICDE 2019.
- Molnar — *Interpretable Machine Learning*, Ch 5–7.

7.13 Lab (2 hrs)

1. On the California Housing dataset, train XGBoost.
2. Generate PDP, ICE, ALE, permutation importance, global SHAP plots for the top-5 features.
3. Find at least one feature where PDP and ALE disagree → explain why (correlation).
4. Use SliceFinder to find the worst-performing data slice.

7.14 Quiz

1. Why can PDP be misleading for correlated features?
2. State the formal difference between PDP and ICE.
3. When would you choose ALE over SHAP?
4. Give one situation where permutation importance over-credits a feature.
5. What's the relationship between EBMs and functional ANOVA?

--	--	--

--	--

XAI Module 8 — Attention, Transformer & LLM Interpretability

Goal: Understand the methods for interpreting transformer-based models — the dominant architecture of the LLM era.

8.1 The attention-as-explanation debate

In 2019, two papers fought:

- **Jain & Wallace** — *Attention Is Not Explanation* (NAACL 2019). Shows attention weights can be radically altered without changing predictions.
- **Wiegrefe & Pinter** — *Attention Is Not Not Explanation* (EMNLP 2019). Shows attention is often informative, even if not the unique explanation.

Consensus (2021–2026):

- Raw attention weights are *not* faithful explanations on their own.
 - Combined with gradients (Grad×Attention, Chefer et al. 2021), they can be faithful.
 - Better: use methods designed for transformers (Attention Rollout, gradient-weighted attention).
-

8.2 Attention Rollout (Abnar & Zuidema, 2020)

Multiply attention matrices across layers, adding the identity for residual flow:

```
A_l_rollout = (A_l + I) · A_{l-1}_rollout  
normalize each row
```

Yields a single matrix showing each token's accumulated influence on the [CLS] / output token. Used heavily for ViT interpretation.

8.3 Attention Flow

Treat the network as a graph where edges = attention weights. Compute max-flow from input tokens to output. Slower than rollout but cleaner attribution.

8.4 Generic Transformer Attribution (Chefer et al., CVPR 2021)

A method-of-choice for ViTs combining:

- Gradients with respect to attention
- Layer-wise relevance propagation rules
- Aggregation across heads and layers

Produces clean, faithful heatmaps for both encoder-only (BERT/ViT) and encoder-decoder transformers.

8.5 Probing classifiers

Train a simple classifier (logistic regression) on top of intermediate layer activations to predict some property $\mathcal{P}$ (POS tag, syntactic role, sentiment).

If the probe scores high $\rightarrow$ the model encodes $\mathcal{P}$ at that layer.

Pitfalls

- Probe accuracy $\neq$ model use of the property.
 - Hewitt & Liang (2019) — *control task* baselines necessary to separate "info is there" from "info is usable".
-

8.6 Logit Lens (nostalgebraist, 2020 $\rightarrow$ formalized later)

Apply the unembedding matrix to intermediate residual stream activations to see what tokens the model "would predict" at each layer.

Shows the gradual emergence of the prediction — useful for hallucination analysis.

Tuned Lens (Belrose et al., 2023)

Train a small learned linear probe per layer instead of using the unembedding directly. Smoother trajectories, less noise.

8.7 Activation patching / causal tracing

ROME (Meng et al., NeurIPS 2022). To find *where* in the network a fact lives:

1. Run a forward pass with the original input $\rightarrow$ cache activations.
2. Run with corrupted input (e.g., "Eiffel Tower" $\rightarrow$ "Statue of Liberty").
3. Patch in cached clean activations at one site at a time.
4. Site that recovers correct answer $\rightarrow$ "this is where the fact lives".

Why this matters

Localized facts can then be **edited** (ROME, MEMIT, GRACE) — opening model editing as a discipline. Hot 2023–2026.

8.8 The induction-head story (Anthropic, 2022)

Olsson et al., *In-context Learning and Induction Heads*. Identified two-layer attention heads that implement $[A][B] \dots [A] \rightarrow [B]$ copying — the basic mechanism of in-context learning.

This is the **canonical mechanistic-interpretability case study**. Read it.

8.9 Sparse Autoencoders (SAEs) — the 2023–2025 breakthrough

The residual stream of a transformer carries many superimposed features (superposition hypothesis, Elhage et al. 2022). SAEs decompose activations into **sparse, monosemantic features**.

Pipeline:

1. Pick a layer; collect millions of activations.
2. Train an SAE: `encoder` to k-sparse latent, `decoder` to reconstruct.
3. Each learned feature $\rightarrow$ one neuron in latent space — often human-meaningful.
4. Use **feature attribution** and **steering** to test causality.

Papers:

- Bricken et al. (Anthropic) — *Towards Monosemanticity*, 2023.
- Cunningham et al. — *Sparse Autoencoders Find Highly Interpretable Features in LMs*, ICLR 2024.
- Templeton et al. (Anthropic) — *Scaling Monosemanticity to Claude 3 Sonnet*, 2024.

This is the **best lead** the field has on understanding LLMs as of 2026.

8.10 LLM-specific interpretability tools

Tool	Purpose
TransformerLens	Hooks & interventions for HF models
nnsight	Remote interventions on large models (NDIF)
SAELens / Neuronpedia	Browse SAE features
Garcon / Inspect (Anthropic)	Internal Anthropic tooling, partly OSS
bertviz	Attention head visualization
OpenAI Microscope (legacy)	Neuron visualizations for vision models

8.11 Hallucination explanation

Open problem: *why* does a given LLM hallucinate?

Approaches:

- Self-consistency (multiple samples → disagreement = uncertainty)
- Logit Lens for divergence between layers
- SAE feature attribution at the hallucination token
- RAG attribution (which retrieved chunks influenced the output?)

Industry deployment (Anthropic Claude Citations, OpenAI Structured Outputs) now ships citation-grounded outputs as a partial answer.

8.12 Reading list

- Vaswani et al. — *Attention Is All You Need*, NeurIPS 2017 (architecture baseline).
- Jain & Wallace — *Attention Is Not Explanation*, NAACL 2019.
- Wiegrefe & Pinter — *Attention Is Not Not Explanation*, EMNLP 2019.
- Abnar & Zuidema — *Quantifying Attention Flow in Transformers*, ACL 2020.
- Chefer, Gur, Wolf — *Transformer Interpretability Beyond Attention Visualization*, CVPR 2021.
- Meng et al. — *Locating and Editing Factual Associations in GPT (ROME)*, NeurIPS 2022.
- Elhage et al. — *A Mathematical Framework for Transformer Circuits*, Anthropic 2021.
- Olsson et al. — *In-context Learning and Induction Heads*, Anthropic 2022.
- Bricken et al. — *Towards Monosemanticity*, Anthropic 2023.
- Templeton et al. — *Scaling Monosemanticity*, Anthropic 2024.
- Conmy et al. — *Towards Automated Circuit Discovery*, NeurIPS 2023.

8.13 Lab (4 hrs)

1. Install TransformerLens. Load GPT-2 Small.
2. Reproduce the induction-head visualization on a simple `A B A B` pattern.
3. Run Logit Lens through the layers for a factual prompt; visualize trajectory.
4. (Stretch) Use a public SAE (Neuronpedia) — find a feature that activates on "Python code" and ablate it to see effect on next-token predictions.

8.14 Quiz

1. Why is raw attention often not a faithful explanation?
2. Describe activation patching in one paragraph.
3. What's the difference between Logit Lens and Tuned Lens?
4. State the superposition hypothesis.
5. Why are SAEs a promising path for monosemanticity?

XAI Module 9 — Mechanistic Interpretability

Goal: Understand the program of reverse-engineering neural networks into human-understandable algorithms — the most active frontier of interpretability research.

9.1 The mech-interp thesis

Neural networks learn algorithms. Those algorithms can, in principle, be reverse-engineered into circuits humans can read.

If true, this offers something stronger than post-hoc explanation: a **mechanistic understanding** that can be verified, edited, and used for safety.

The thesis is most associated with:

- **Distill.pub Circuits Thread** (2017–2020): Olah, Cammarata, Schubert, Carter, Voss, Goh, Mordvintsev.
 - **Anthropic Mechanistic Interpretability team** (2021–present).
 - **Neel Nanda** (Google DeepMind / Anthropic): popularized the program with TransformerLens and educational write-ups.
-

9.2 Why mech-interp matters for AI safety

If you can read what a model is computing, you can:

- Detect deceptive alignment.
- Find dangerous capabilities before deployment.
- Verify whether a model "knows it's lying".
- Edit knowledge / behavior surgically.

These are why Anthropic, DeepMind, OpenAI invest heavily in interpretability research.

9.3 The Distill Circuits Thread — vision

Olah et al. analyzed InceptionV1 unit by unit.

Findings

- **Curve detectors** in early layers — clean equivariant features.
- **High-low frequency detectors** — distinguish foreground/background.
- **Multimodal neurons** (Goh et al., 2021) — single neurons that fire for both an image *and* the text of a concept (e.g., "Spider-Man" neuron).
- **Polysemantic neurons** — one neuron responding to multiple unrelated concepts → superposition.

- **Equivariance** — networks learn rotated/translated copies of similar filters.

These are direct evidence that **interpretable structure exists**.

9.4 A Mathematical Framework for Transformer Circuits (Elhage et al., 2021)

Decompose transformer behavior into:

- **Residual stream** as the communication medium.
- **Attention heads** as read/write operations.
- **MLP layers** as memory / nonlinear gates.
- **QK and OV circuits** as separable read- vs write-direction operators.

This gave the field a vocabulary and made later work (induction heads, ROME) possible.

9.5 Induction heads — Olsson et al., 2022

A specific two-layer circuit that implements in-context completion:

- **Previous-token head** at layer L : copies token T_1 to position of T_2 .
- **Induction head** at layer $L+1$: looks back for similar pattern, copies the continuation.

Together these produce: $[T_1 \ T_2] \ \dots \ [T_1] \rightarrow \text{predict } T_2$.

These heads appear **across model scales** and explain a phase transition in in-context learning ability.

9.6 ROME / MEMIT — knowledge editing

Meng et al. (NeurIPS 2022) found that factual associations like "Eiffel Tower is in Paris" are stored in specific MLP layers. ROME edits them via rank-one updates to MLP weights.

Followups:

- **MEMIT** — mass editing (many facts at once).
- **GRACE** — robust against rollback.
- **ICE** — in-context editing without weight changes.

Used in **alignment research** to test whether edited knowledge generalizes (often it doesn't, perfectly).

9.7 Automatic circuit discovery

Manual circuit discovery is slow. Recent work automates it:

- **ACDC** (Conmy et al., NeurIPS 2023) — Automated Circuit Discovery via edge ablation.
- **EAP / EAP-IG** — Edge Attribution Patching with Integrated Gradients.
- **Path Patching** — measures direct causal paths.

These tools take a hypothesized task (e.g., "indirect object identification") and produce the minimal circuit responsible.

9.8 Indirect Object Identification (IOI) — Wang et al. 2022

Classic case study: GPT-2 Small correctly completes "When John and Mary went to the store, John gave a drink to ____" with "Mary".

Discovered circuit:

- Duplicate-token heads
- S-inhibition heads
- Name mover heads
- Backup name mover heads (redundancy!)

The redundancy explains why simple ablation studies can mislead.

9.9 Superposition and SAEs (recap)

The superposition hypothesis: networks pack more features than they have neurons, in **interfering** directions.

SAEs (covered in [Module 8](#)) decompose the residual stream into ~tens of thousands of sparse, near-monosemantic features. This may be the **technical breakthrough** that unblocks mech-interp at LLM scale.

Recent: **Gemma Scope** (Google DeepMind, 2024) — SAEs on every layer of Gemma 2; **SAELens**, **Neuronpedia**, **GoodFire** — tooling.

9.10 Causal analysis with abstractions

Geiger, Lu, Icard, Potts (NeurIPS 2021, ICML 2023) — **Causal Abstraction**: hypothesize a human-readable algorithm, then verify by intervention that the model implements it.

Formalizes "the model computes X" as a causal claim that can be falsified.

9.11 Limits of mech-interp

- **Scaling** — most case studies are GPT-2 Small or smaller. Big models are mostly opaque.
- **Polysemanticity / superposition** — even with SAEs, decomposition is imperfect.
- **Compositional behavior** — circuits found for one task may not compose.

- **Reproducibility** — many findings are checkpoint-specific.
- **Selection bias** — researchers report wins; we don't know how often circuits failed to be found.

Be honest about these in any paper you write.

9.12 How to start a mech-interp research project

1. Pick a small, well-defined behavior (1-token completion task).
2. Pick a small model (GPT-2 Small, Pythia 70M–410M).
3. Use TransformerLens. Run ACDC or EAP to find a candidate circuit.
4. Validate via causal interventions (ablation, patching).
5. Write up with diagrams + interactive demos.

The bar to enter is unusually low; the bar to make a *general* claim is high.

9.13 Reading list

- Olah, Mordvintsev, Schubert — *Feature Visualization*, Distill 2017.
- Olah et al. — *Zoom In: An Introduction to Circuits*, Distill 2020.
- Goh et al. — *Multimodal Neurons in Artificial Neural Networks*, Distill 2021.
- Elhage et al. — *A Mathematical Framework for Transformer Circuits*, Anthropic 2021.
- Olsson et al. — *In-Context Learning and Induction Heads*, Anthropic 2022.
- Wang et al. — *Interpretability in the Wild: A Circuit for Indirect Object Identification*, 2022.
- Meng et al. — *Locating and Editing Factual Associations in GPT (ROME)*, NeurIPS 2022.
- Conmy et al. — *Towards Automated Circuit Discovery for Mechanistic Interpretability (ACDC)*, NeurIPS 2023.
- Bricken et al. — *Towards Monosemanticity*, Anthropic 2023.
- Templeton et al. — *Scaling Monosemanticity*, Anthropic 2024.
- Geiger et al. — *Inducing Causal Structure for Interpretable Neural Networks*, ICML 2022.
- Nanda — *A Comprehensive Mechanistic Interpretability Explainer*, neelnanda.io.

9.14 Lab (4 hrs)

1. Install TransformerLens + GPT-2 Small.
2. Reproduce the IOI prompt: "When John and Mary went to the store, John gave a drink to ____".
3. Identify name-mover heads via attention patching. Show that ablating them drops accuracy from ~80% to ~20%.
4. Visualize one head's attention pattern.

9.15 Quiz

1. Why is superposition a problem for naive neuron-level interpretation?
2. State the difference between activation patching and gradient-based attribution.
3. What is an induction head and why does it matter for in-context learning?
4. Describe one finding from the IOI circuit paper that complicates naive ablation.

5. Why might SAEs be the practical breakthrough for LLM interpretability?

XAI Module 10 — Evaluating Explanations

Goal: A paper that introduces an explanation method is only as good as its evaluation. Master the metrics, sanity checks, and adversarial tests.

10.1 Three lenses (revisited)

Lens	Question	Cost
Faithfulness	Does the explanation reflect what the model actually computes?	Low–Medium
Plausibility	Does it make sense to humans?	High (user studies)
Robustness	Does it change under tiny input perturbations?	Low

A method can be plausible but unfaithful (Slack et al. attack), or faithful but not human-aligned (mech interp circuits).

10.2 Faithfulness metrics

Deletion / Insertion (Petsiuk, Das, Saenko, 2018 — RISE paper)

```
deletion:  remove top-k most-attributed features → measure prediction drop  
           area-under-curve (AUC) = lower is better (explanation captures important feat  
  
insertion: add top-k from blank baseline → measure prediction rise  
           AUC = higher is better
```

ROAR (RemOve And Retrain) — Hooker et al., NeurIPS 2019

Remove top features, **retrain** the model, then measure accuracy. Tests whether the attributed features were actually informative — not just "the model uses them now".

Comprehensiveness / Sufficiency — DeYoung et al. 2020 (ERASER benchmark)

- **Comprehensiveness** = drop in prediction after removing the explanation rationale.
- **Sufficiency** = prediction when *only* the rationale is kept.

A good explanation has high comprehensiveness *and* low sufficiency gap.

Infidelity (Yeh et al., NeurIPS 2019)

$$\text{INFD}(g, f, x) = \mathbb{E}_{\mathbf{I}} [(\mathbf{I}^T g(x) - (f(x) - f(x - \mathbf{I})))^2]$$

Where $\mathbf{x}$ is a perturbation vector. Measures expected difference between attributions and actual prediction change under perturbations.

AOPC (Area Over Perturbation Curve)

Same family as deletion / insertion AUC.

10.3 Sanity checks (Adebayo et al., NeurIPS 2018)

Two tests every saliency method should pass:

Model parameter randomization

Re-initialize the top layers downward. A faithful saliency must change.

Data randomization

Train the model on randomly-reabeled data. A faithful saliency must produce noticeably different attributions than the correctly-trained model.

Methods that fail: Guided Backprop, Guided Grad-CAM. Their outputs are too similar across these manipulations to count as model-specific explanations.

10.4 Robustness / Stability

Local stability (Alvarez-Melis & Jaakkola, 2018)

$$\text{Stability}(\mathbf{x}) = \sup_{\{\mathbf{x}' \sim \mathbf{x}\}} \|\mathbf{g}(\mathbf{x}) - \mathbf{g}(\mathbf{x}')\| / \|\mathbf{x} - \mathbf{x}'\|$$

For nearby $\mathbf{x}'$, the explanation $\mathbf{g}$ shouldn't jump.

LIME and SHAP can fail this badly without averaging.

Adversarial XAI

Slack et al. (AIES 2020) constructed a "biased model that hides its bias from LIME and SHAP". Showed that explanation methods themselves can be **gamed**.

Aïvodji et al. (2019) — "Fairwashing" — building a fair-looking explanation for an unfair model.

10.5 Plausibility (human-aligned) metrics

ERASER (DeYoung et al., 2020)

NLP benchmark with human-annotated rationales. Compare explanation tokens to human-marked tokens via IoU / token-F1.

Human studies

- **Simulatability** — can humans predict model output given the explanation? (Hase & Bansal, 2020)
- **Forward simulation** — humans use the explanation to make decisions; measure accuracy / time / trust.
- **Counterfactual simulatability** — given an explanation, can the user predict what would change if they changed feature X?

These are the gold standard. They are expensive and rare.

10.6 Specialized benchmarks

Benchmark	Domain
ERASER	NLP rationales
CLEVR-X	VQA explanations
GRADELLM / FActScore	LLM hallucination grounding
OpenXAI	Tabular feature attribution standard
Quantus	Comprehensive XAI metric library (Hedström et al., 2023)
Funke et al. BAM	Benchmark for Attribution Methods (synthetic ground truth)

Use **Quantus** for any new tabular/vision XAI work — it implements 30+ metrics.

10.7 The "disagreement problem" revisited (Krishna et al. 2022)

Empirically:

- Different methods rank features differently.
- Rank correlation across (LIME, SHAP, IG, SmoothGrad) often 0.3–0.6.
- Practitioners pick the explanation that confirms their priors.

Practical mitigation: report multiple methods + consistency metrics; pre-register the explanation method you'll use, before training.

10.8 What to report in a new XAI paper

A minimum checklist:

- [] At least one faithfulness metric (deletion/insertion AUC, ROAR, or Infidelity).

- [] Sanity checks (Adebayo).
- [] Stability metric.
- [] Comparison to ≥ 3 baselines.
- [] Disagreement analysis with at least 2 alternative methods.
- [] If applicable, plausibility metric (ERASER overlap, human study).
- [] Computational cost.

Without these, reviewers will ask.

10.9 Reading list

- Adebayo et al. — *Sanity Checks for Saliency Maps*, NeurIPS 2018.
- Hooker et al. — *A Benchmark for Interpretability Methods in Deep Neural Networks (ROAR)*, NeurIPS 2019.
- Yeh, Hsieh, Suggala, Inouye, Ravikumar — *On the (In)fidelity and Sensitivity of Explanations*, NeurIPS 2019.
- Petsiuk, Das, Saenko — *RISE: Randomized Input Sampling for Explanation*, BMVC 2018.
- DeYoung et al. — *ERASER: A Benchmark to Evaluate Rationalized NLP Models*, ACL 2020.
- Slack, Hilgard, Jia, Singh, Lakkaraju — *Fooling LIME and SHAP*, AIES 2020.
- Aïvodji et al. — *Fairwashing: the risk of rationalization*, ICML 2019.
- Krishna et al. — *The Disagreement Problem in Explainable Machine Learning*, 2022.
- Hedström et al. — *Quantus: An Explainable AI Toolkit for Responsible Evaluation*, JMLR 2023.

10.10 Lab (2 hrs)

1. Take last week's saliency methods.
2. Compute deletion AUC and insertion AUC for each.
3. Run Adebayo's model randomization sanity check.
4. Use Quantus to compute Infidelity & Stability.
5. Tabulate results across methods. Which is most faithful? Which is most stable?

10.11 Quiz

1. Why is deletion AUC a faithfulness metric but not a plausibility metric?
2. State the ROAR procedure and what it adds over simple deletion.
3. What did Slack et al. demonstrate, and what does it imply for compliance?
4. Define comprehensiveness and sufficiency from ERASER.
5. Why is the disagreement problem dangerous in regulated industries?

XAI Module 11 — Human-Centered XAI

Goal: Understand explanations as a **human-computer interaction** problem. The best metric is "did the user understand and act correctly", not "does the heatmap look pretty".

11.1 The HCI turn

Until ~2018, XAI papers measured success by mathematical properties. Then a wave of HCI researchers (Miller, Hoffman, Mueller, Wang, Lakkaraju, Wattenberg, Viégas) shifted the question:

Do explanations actually help users make better decisions?

The empirical answer is: **often, no**.

11.2 Miller (2019) — what explanation actually is

Explanation in Artificial Intelligence: Insights from the Social Sciences — Artificial Intelligence Journal.

Key findings from 250+ social-science papers on explanation:

1. **Explanations are contrastive** — people ask "why P rather than Q?" not "why P?"
2. **Explanations are selected** — humans don't want every cause, only the most relevant ones.
3. **Explanations are social** — they're a conversation, not a one-shot output.
4. **Probabilities don't help much** — people want causal stories.
5. **People are biased** — they prefer simple, plausible, generalizable explanations even when those are wrong.

Implication: faithful, complete explanations may actively confuse users. Good XAI is selection + framing, not raw attribution dumps.

11.3 Cognitive load & explanation overload

Studies (Bansal et al. 2021, Poursabzi-Sangdeh et al. 2021):

- Showing more features = more confusion.
- Numerical attributions (SHAP values) are misread by non-experts.
- Highlighting *one* feature outperforms showing all features.

Best practice: limit to top-3 to top-5 features for non-expert users. Use plain-language framing ("This loan was likely denied because of low income and short credit history").

11.4 Trust calibration

The goal of an explanation is not "more trust" — it is **appropriately calibrated trust**.

- **Over-reliance** — user trusts wrong answers because the explanation looked convincing.
- **Under-reliance** — user ignores correct answers because of vague unease.

Vasconcelos et al. (CSCW 2022): explanations often **increase reliance regardless of model correctness** — a problem.

Bussone, Stumpf, O'Sullivan (2015) — even nonsensical explanations can increase trust ("automation bias").

11.5 Anchoring & confirmation effects

When an explanation aligns with the user's prior, they adopt the model's prediction. When it contradicts, they often dismiss the model. This is **anchoring**, not understanding.

Mitigation: present alternative hypotheses, not single explanations.

11.6 The "AI-assisted decision-making" framing

Modern HCI–XAI research treats the unit of analysis as the **human-AI team**, not the model alone.

Key questions:

- When should AI defer to humans?
- When should AI override?
- How should AI communicate uncertainty?
- How do we design explanations that improve *team* accuracy, not just model accuracy?

Read: Bansal et al., *"Does the Whole Exceed its Parts?"* (CHI 2021).

11.7 Explanation interfaces

Form	Example product
Local feature highlight	Loan denial reasons
Counterfactual ("if X were different")	Credit improvement coach
Conversational	LLM-grounded "Why did you say that?"
Visualization (PDP, heatmap)	Model debugging dashboards
Concept-based	Medical imaging tools

The *form* should match the user's mental model. A radiologist wants concepts; a borrower wants counterfactuals.

11.8 Designing a user study for XAI

A minimal protocol:

1. **Task:** real domain task (e.g., loan approval).
2. **Conditions:** no AI / AI-only / AI + explanation type 1 / type 2 / ...
3. **Metrics:** accuracy, time, calibration, trust survey, NASA-TLX (workload).
4. **Participants:** ≥30 per condition; domain experts when possible.
5. **Pre-register** hypotheses and analysis plan.
6. **Report** effect sizes, not just p-values.

Most XAI papers skip the user study. The ones that include one tend to overturn the assumption that the "better" explanation is more useful.

11.9 Fairness, accountability, transparency (FAccT)

XAI and fairness intersect but are not the same.

- An explanation may *reveal* unfairness (e.g., heavy weight on a proxy for race).
- An explanation can also *launder* unfairness (Aïvodji 2019, *fairwashing*).

For fairness, use dedicated tools (Fairlearn, AI Fairness 360, Aequitas) **alongside** XAI.

11.10 LLM explanations and "self-explanations"

LLMs can produce natural-language rationales. Beware:

- **Hallucinated rationales** — Turpin et al. (2023) showed CoT explanations can be unfaithful — the model decided early, then rationalized.
- **Sycophancy** — model gives the user the explanation they want.
- **Plausible-sounding mistakes** — the very fluency makes errors more dangerous.

Best practice: verify with non-LLM methods (probes, SAEs, citation-grounded outputs).

11.11 Reading list

- Miller — *Explanation in Artificial Intelligence: Insights from the Social Sciences*, AI Journal 2019.
- Hoffman, Mueller, Klein, Litman — *Metrics for Explainable AI: Challenges and Prospects*, 2018.

- Poursabzi-Sangdeh et al. — *Manipulating and Measuring Model Interpretability*, CHI 2021.
- Bansal et al. — *Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance*, CHI 2021.
- Vasconcelos et al. — *Explanations Can Reduce Overreliance on AI Systems During Decision-Making*, CSCW 2022.
- Bussone et al. — *The Role of Explanations on Trust and Reliance in Clinical Decision Support Systems*, ICHI 2015.
- Turpin et al. — *Language Models Don't Always Say What They Think*, NeurIPS 2023.
- ACM FAccT proceedings — annual.

11.12 Lab (2 hrs)

1. Pick a tabular classifier you trained earlier.
2. Design a 3-condition user study (no-explanation / SHAP / counterfactual).
3. Recruit 6 friends; have each make 20 decisions in each condition.
4. Measure accuracy, time, self-reported confidence.
5. Write a 1-page reflection on what you'd improve.

11.13 Quiz

1. State Miller's four key findings about how humans explain.
2. What is over-reliance and how can XAI cause it?
3. Why might counterfactual explanations be more actionable than SHAP for an end user?
4. What did Turpin et al. show about CoT explanations?
5. Why is faithfulness without plausibility insufficient for end-user XAI?

XAI Module 12 — XAI in Domains: Medical, Finance, Law, NLP

Goal: Translate generic XAI methods into domain-specific value (and risk).

12.1 Medical XAI

Why it matters

- High-stakes decisions
- Clinicians need to validate model reasoning before acting
- Regulatory: FDA SaMD, EU MDR, India CDSCO classify ML-based devices

Common methods by modality

Modality	XAI methods
Radiology (CXR, CT, MRI)	Grad-CAM, LRP, ProtoPNet, BagNet
Pathology (WSI)	Attention-MIL highlights, concept-based
EHR / tabular	SHAP, EBM, GAM
Genomics	DeepLIFT, IG with biological priors
Clinical NLP	Rationale-based, Anchors, IG on tokens

Pitfalls

- **Shortcut learning** — model uses scanner artifacts, image markers, hospital tags (DeGrave et al. 2021 on COVID-19 CXR models).
- **Saliency ≠ diagnosis** — heatmap on lung doesn't mean "consolidation"; could be "rib".
- **Concept misalignment** — concepts defined by ML researchers may not match clinical vocabulary.

Best practices

- Always include an **out-of-distribution test** with a different hospital / scanner.
- Combine pixel attribution with **concept-based** (TCAV, ProtoPNet) for clinical translation.
- Report calibration alongside performance.
- Engage clinicians in evaluation (DECIDE-AI, CONSORT-AI reporting standards).

Key papers

- Caruana et al. — *Intelligible Models for HealthCare*, KDD 2015 (pneumonia EHR example, "asthma is protective" finding — classic shortcut).

- DeGrave, Janizek, Lee — *AI for radiographic COVID-19 detection selects shortcuts over signal*, Nature MI 2021.
 - Rajpurkar et al. — CheXpert, CheXNet papers.
 - Tonekaboni et al. — *What Clinicians Want: Contextualizing Explainable Machine Learning for Clinical End Use*, MLHC 2019.
-

12.2 Financial XAI

Why it matters

- Credit decisions are regulated (US ECOA / Fair Lending; EU AI Act high-risk; India RBI guidelines).
- Adverse-action notices legally require reasons.
- Algorithmic trading needs post-mortem analysis.

Common methods

- **SHAP** (industry standard for tabular credit models)
- **Counterfactuals** / **DiCE** for adverse-action / recourse
- **PDP** / **ALE** for model debugging
- **EBM** (Microsoft InterpretML) as a credit-modeling baseline
- **Fairness toolkits**: Aequitas, Fairlearn

Pitfalls

- **Proxy features** for protected attributes (zipcode → race).
- **Disparate impact** invisible in SHAP rankings.
- **Adversarial robustness** — gaming the model is profitable.
- **Distribution shift** — credit models built pre-COVID failed badly during the pandemic.

Regulatory landscape (2026)

- US: CFPB / OCC enforce model risk management (SR 11-7).
 - EU: AI Act — credit scoring is "high risk", requires logging, explanation, human oversight.
 - India: RBI guidelines on AI/ML in financial services.
-

12.3 Legal XAI

Use cases

- Predictive policing (controversial)
- Recidivism prediction (COMPAS)
- Bail / sentencing risk scores
- Judicial decision prediction (academic, not deployed)

Cautions

- ProPublica 2016 — *Machine Bias* — found COMPAS racially biased. Northpointe disputed. Sparked the **fair prediction debate** (Chouldechova 2017, Kleinberg et al. 2016: impossibility theorem — you can't satisfy multiple fairness criteria simultaneously).
- Rudin (2019) — explicit case for interpretable risk scores over black-box ones in criminal justice.

Best practice

- Use **transparent risk scores** (e.g., 2HELPS2B-style integer-coefficient models).
- Document features, weights, and provenance.
- Audit subgroup performance.

12.4 NLP / LLM XAI

Methods

- Saliency / IG over tokens
- Attention rollout, Chefer-style attribution
- Rationale extraction (ERASER benchmark)
- Influence functions, Datamodels
- Mechanistic interpretability (SAEs, circuits)
- Citation grounding (RAG attribution)

Hot 2024–2026 topics

- **Citation-grounded LLM outputs** (Anthropic Claude Citations, Perplexity sources, OpenAI references)
- **Hallucination detection** via self-consistency, SAE features
- **Sycophancy explanations**
- **Steering vectors / activation engineering** as interpretability + control
- **Constitutional AI / RLHF interpretability**

Open problems

- Faithfulness of natural-language self-explanations (Turpin et al. 2023).
- Cross-lingual interpretability.
- Tool-use explanation in agentic LLMs.

12.5 Computer vision in industry

Application	XAI need
Autonomous driving	Debug failure modes; counterfactual scenes
Manufacturing defect detection	Validate spurious cues
Retail (cashierless)	Audit demographic disparity

12.6 Recommender systems

- Show *why* an item was recommended ("Because you watched X").
 - Inverse / counterfactual recommendations.
 - Methods: PIRL (Persuasive Interpretable Recommendation), counterfactual recommender literature.
-

12.7 Education

- Adaptive learning systems: explain *why* a concept was recommended.
 - AI tutors: rationale-based explanations of answers.
 - Diagnostic models for student misconceptions.
-

12.8 Cybersecurity

- Anomaly detection systems should justify flagged events.
 - SHAP/LIME over network features.
 - Adversarial-aware explanation (the system being attacked also attacks the explanation).
-

12.9 Climate / scientific ML

- ML on climate models: which features drive forecast?
 - Physics-informed XAI (Toms et al. 2020).
 - Concept-aligned attribution for scientific reasoning.
-

12.10 Domain-XAI checklist (universal)

For any deployment:

- ☐ Stakeholder identified
- ☐ Explanation format matched to stakeholder
- ☐ Faithfulness validated
- ☐ Fairness audit performed independently
- ☐ Shortcut/spurious correlation test passed
- ☐ OOD performance characterized
- ☐ User-facing language tested with domain experts
- ☐ Logging + audit trail in place

12.11 Reading list

- Caruana et al. — *Intelligible Models for HealthCare*, KDD 2015.
- DeGrave, Janizek, Lee — *AI for radiographic COVID-19 detection selects shortcuts*, Nature MI 2021.
- Tonekaboni et al. — *What Clinicians Want*, MLHC 2019.
- Bhatt et al. — *Explainable Machine Learning in Deployment*, FAccT 2020.
- Chouldechova — *Fair Prediction with Disparate Impact*, FAT* 2017.
- Kleinberg, Mullainathan, Raghavan — *Inherent Trade-Offs in the Fair Determination of Risk Scores*, 2016.
- Rudin et al. — *Interpretable Machine Learning: Fundamental Principles and 10 Grand Challenges*, Stat Sci 2022.
- Turpin et al. — *Language Models Don't Always Say What They Think*, NeurIPS 2023.

12.12 Lab (3 hrs)

Pick *one* domain:

- Build a small model (e.g., MIMIC-III mortality / German Credit / IMDB sentiment).
- Apply 2 XAI methods.
- Critique results against domain norms.
- Write a 2-page "would you deploy?" memo.

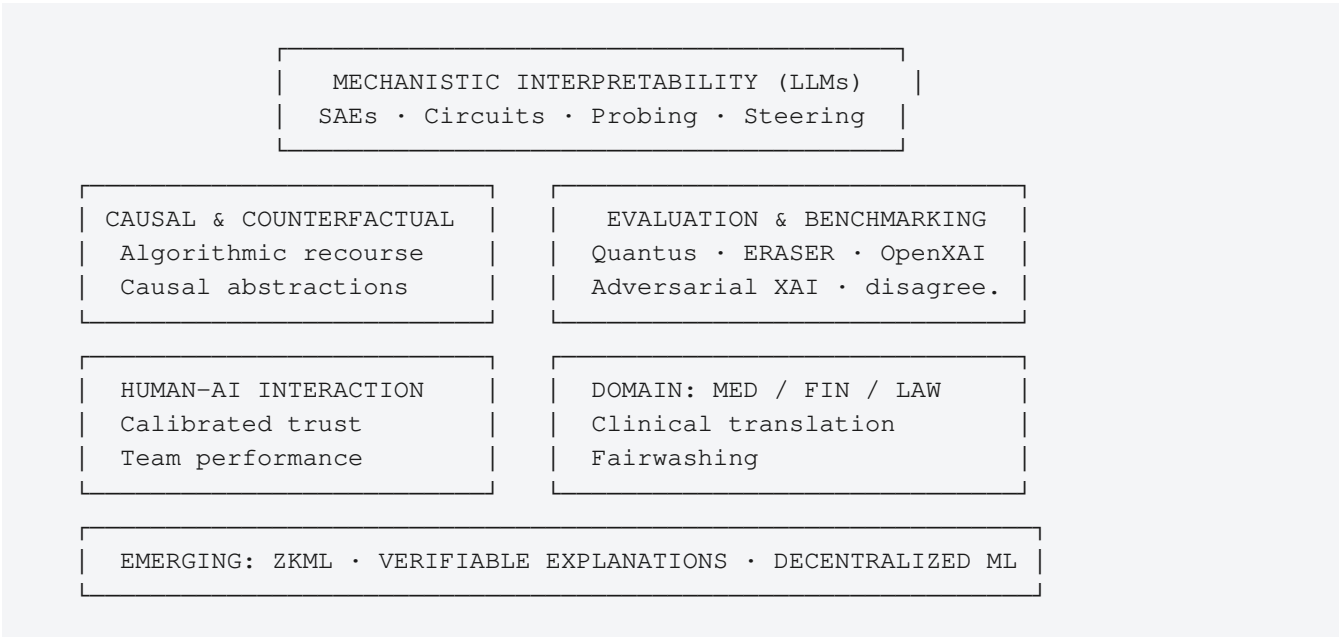
12.13 Quiz

1. Why did the COMPAS debate produce an impossibility theorem?
 2. What was Caruana's "asthma is protective" finding and what does it teach?
 3. Why are concept-based methods better than saliency for clinician communication?
 4. Name two ways shortcut learning shows up in medical imaging.
 5. Why might LLM self-explanations be unfaithful?
-

XAI Module 13 — Research Frontiers (2025–2027)

Goal: Identify the active research clusters and where a Master's student can make a tractable contribution.

13.1 The XAI research map (2026)



13.2 Hot subareas

1. Mechanistic interpretability at scale

- SAEs on 100B+ parameter models.
- Automated circuit discovery (ACDC, EAP-IG).
- Cross-model universality of circuits.
- Practical alignment use cases (deception, jailbreak detection).

2. Faithful CoT and chain-of-thought interpretability

- Turpin et al. showed CoT may be unfaithful.
- Open: how to *train* models with faithful reasoning?
- Process supervision (Lightman et al. 2023 OpenAI) is a partial answer.

3. Multimodal & agent interpretability

- VLMs (CLIP, BLIP, Gemini): cross-modal feature attribution.
- LLM agents: explaining tool-use sequences.
- Reward model interpretability (what does the RLHF reward model actually reward?).

4. Causal XAI

- Counterfactuals with causal graphs.
- Algorithmic recourse under uncertain causal knowledge.
- Causal discovery from observational data + XAI integration.

5. Adversarial XAI

- Robust explanations that survive adversarial perturbations.
- Detecting fairwashing.
- "Honest" model architectures that can't lie to explainers.

6. XAI for foundation models / diffusion models

- Interpreting CLIP-like models.
- Concept activation for image generation control.
- Diffusion model interpretability via score landscapes.

7. Probing and feature discovery

- Sparse probing.
- Linear representation hypothesis tests.
- Geometry of representations (Park et al. 2023, *Linear Representation Hypothesis*).

8. Memorization vs generalization

- Influence functions at scale (Datamodels, TRAK).
- Membership inference and unlearning explanations.

9. Evaluation crisis

- Most explanation methods evaluated on toy benchmarks.
- Need standardized, application-grounded evals.
- Quantus, OpenXAI lead this.

10. XAI + Privacy

- Differentially-private explanations.
- ZK-proven explanations (ZK + XAI).

11. XAI + Fairness

- Explanations as fairness audits.
- Fairwashing detection.
- Subgroup-aware explanations.

12. Verifiable / On-Chain ML (zkML)

- Prove inference was correct + model unchanged.
- See [Intersection module](#).

13.3 Tractable Master's-thesis-scale topics

Topic	Why tractable
Empirical disagreement study on a new domain (legal text, audio)	Reuse existing tools
SAE feature analysis on a specific small model	TransformerLens makes setup fast
User study comparing 2 explanation forms on a real task	High novelty, modest infrastructure
Counterfactual quality for credit data under causal constraints	Data is available, methods extant
Reproducibility study of 5 saliency methods	Always publishable as a workshop paper
zkML proof of a tiny network's inference + interpretability	Crossroads area, sparse competition

13.4 Venues for XAI research

Venue	Type
NeurIPS, ICML, ICLR	Tier-1 ML
AAAI, IJCAI	Broader AI
ACL, EMNLP, NAACL	NLP, XAI for language
CVPR, ICCV, ECCV	Vision XAI
FAccT	Fairness, accountability, transparency
CHI, CSCW, UIST	HCI, user studies
AIES (AAAI/ACM)	AI ethics
MLHC	Healthcare ML
JMLR, Nature MI, Distill (defunct but archive)	Journals

For first papers: **XAI4CV**, **Re-XAI**, **BlackBoxNLP** workshops are friendly entry points.

13.5 How to identify a good open problem (workflow)

1. Pick a frontier (e.g., faithful CoT).
2. Read the 10 most-cited papers in that area from the past 2 years.

3. Tabulate **claims, assumptions, limitations** of each.

4. Find:

- A claim with weak empirical support → reproduce + extend.
- An assumption no one tested → test it.
- A limitation that has a small-scope fix.

5. Run a 2-week pilot. If it works, commit to a 3-month project.

13.6 Reading list — recent must-reads

- Bricken et al. — *Towards Monosemanticity*, Anthropic 2023.
- Templeton et al. — *Scaling Monosemanticity*, Anthropic 2024.
- Conmy et al. — *Automated Circuit Discovery (ACDC)*, NeurIPS 2023.
- Turpin et al. — *Language Models Don't Always Say What They Think*, NeurIPS 2023.
- Krishna et al. — *The Disagreement Problem in Explainable ML*, 2022.
- Rudin et al. — *Interpretable ML: Fundamental Principles and 10 Grand Challenges*, Statistical Science 2022.
- Park et al. — *The Linear Representation Hypothesis and the Geometry of Large Language Models*, 2023.
- Marks et al. — *Sparse Feature Circuits*, 2024.
- Karimi et al. — *A Survey of Algorithmic Recourse*, ACM CSUR 2022.

13.7 Lab / capstone

Pick **one**:

- Apply Sparse Autoencoders to Pythia-160M on a custom domain (e.g., medical text). Browse and document 50 interesting features.
- Empirical disagreement study: 6 methods × 4 datasets × 3 model classes. Quantify disagreement.
- User study: 30 participants, 2 explanation conditions, 1 domain task. Pre-register.
- Reproduce ACDC's IOI circuit; extend to a different prompt template.

13.8 Quiz

1. Name three SAE-related results from 2023–2024.
2. Why is faithful CoT an unsolved problem?
3. Define "linear representation hypothesis" in one sentence.
4. What makes algorithmic recourse harder than counterfactual explanation?
5. Pick one workshop venue you'd target with a first paper, and justify the fit.

Intersection — Verifiable, Explainable, and Decentralized AI

Goal: *This is the research-value module. Where Blockchain meets XAI, several emerging research lines need contributors. A Master's thesis here has unusually high impact-per-effort ratio.*

I.1 Why these two fields converge

Blockchain solves: **verifiability and integrity of computation in adversarial environments**. XAI solves: **understanding and trust of model behavior**.

Together, they tackle the question:

Can we know — and prove — what an AI did, why it did it, and that no one tampered with it?

Three convergence areas:

1. **Verifiable AI** — cryptographic proofs that a model output is correct (ZKML).
 2. **Decentralized AI** — model training/inference distributed and incentivized via tokens.
 3. **Auditable / Explainable AI** — on-chain logs and explanations for regulatory compliance.
-

I.2 ZKML — Zero-Knowledge Machine Learning

The problem

A model owner can prove to a verifier that:

- "This output y is the result of running model M on input x ."

Optionally hiding:

- The model weights (IP protection)
- The input (privacy)
- Both

Why hard

Neural networks use:

- **Floating-point arithmetic** — ZK circuits work over finite fields → quantization required.
- **Non-linear ops** (ReLU, GELU, softmax) — expensive to express in arithmetic constraints.
- **Large parameter counts** — billions of multiplications → billions of constraints.

Active toolchains (2024–2026)

Tool	Approach
EZKL	Halo2 backend, ONNX-friendly. Quantization + lookup-arg optimizations
Modulus Labs (Remainder)	GKR / sum-check for matrix multiplications
Risc Zero zkVM	General-purpose zkVM, run PyTorch in Rust via tract
Giza	StarkNet + Cairo for on-chain inference
DDKang's zkml	Halo2, focused on ResNet-scale
Inference Labs	Production ZK inference SaaS
Worldcoin	Uses semaphores + ZK for proof of personhood

State of the art

- ResNet-50 inference: provable in ~minutes on modern hardware, ~MB proofs.
- Small CNN MNIST: seconds, KB proofs.
- LLMs (>1B parameters): largely impractical end-to-end today; partial proofs possible.

Why it matters for XAI

A ZK proof of *correct inference* is necessary but not sufficient for trust. The next step: **ZK proofs of explanation**. Examples:

- Prove that a SHAP value reported externally was correctly computed from the model.
- Prove that a counterfactual explanation is genuinely the minimum-cost one.
- Prove that an attention-based "reason" was correctly extracted.

This is **mostly open research as of 2026** — there's room for early entrants.

I.3 On-chain machine learning

Approaches

Approach	Trust assumption
Inference off-chain , post proof on-chain	ZK or optimistic
Inference on-chain in a smart contract	High gas; bounded models only
Inference in TEE with attestation	Hardware trust (Intel SGX, AMD SEV)
Inference via MPC	Multiple non-colluding parties
Inference with FHE	Pure cryptography, very expensive

EVM-based on-chain ML (Modulus Labs' Ocean, ORA) typically posts ZK proofs of small models for

verifiable randomness, oracle aggregation, content moderation triggers, etc.

I.4 Decentralized AI infrastructure

Compute marketplaces

Project	Layer
Akash	General Kubernetes compute
Render	GPU rendering + ML inference
io.net	Distributed GPU cluster
Gensyn	Distributed training with verification
Bittensor	Subnet-based incentivized model marketplace
Ritual	AI inference oracle for smart contracts

Trust model varies — some use ZK, some use challenge-response / fraud proofs.

Model marketplaces & reputation

Open problems:

- How do you verify a model's claimed performance?
- How do you reward good models without enabling sybils?
- How do you handle adversarial submissions (model poisoning)?

Bittensor's "Yuma consensus" weights subnet contributions by peer voting — a research-grade mechanism design effort.

I.5 Federated learning + blockchain

Federated learning (FL) trains a model across many devices without centralizing data. Blockchains can:

- Coordinate aggregation rounds.
- Reward participants by stake / contribution.
- Audit updates for poisoning.
- Verify aggregation via ZK / MPC.

Research papers stack: FL + blockchain + differential privacy + XAI.

Trust model: typically a permissioned consortium chain (Hyperledger Fabric, IBC chains). Honest-majority assumption.

I.6 Decentralized identity and AI

Tool	Use
W3C DIDs / Verifiable Credentials	Self-sovereign identity
Soulbound tokens (SBT)	Non-transferable reputation
zkID / Sismo / Worldcoin	Privacy-preserving "uniqueness" proofs
Polygon ID	Selective disclosure of attributes

Why for AI: deepfake / agentic-AI era requires **proof of personhood** to distinguish humans from agents. Worldcoin's iris-scan-based PoP is the most controversial deployment.

I.7 Audit trails: explanations on-chain

For regulatory compliance, you may want:

```
inputs_hash + model_hash + outputs_hash + explanation_hash → on-chain commitment
```

Anyone can later verify the audit trail. Variants:

- **Off-chain explanation, on-chain hash** (cheap, requires data availability).
- **On-chain explanation** (expensive, only for small models or critical decisions).
- **ZK explanation** (prove the explanation was correctly computed without revealing input or model).

Use cases:

- Credit decision explanations stored immutably for regulator audit.
- Healthcare AI outputs with retroactive verifiability.
- Election counting AI with public auditability.

I.8 Adversarial concerns

This intersection inherits **both** fields' attack surfaces:

From blockchain	From XAI
51% attack on verification chain	Adversarial explanations (Slack et al.)
Bridge hacks (cross-chain proofs)	Fairwashing
Oracle manipulation	Manipulated saliency
MEV on inference auctions	CoT unfaithfulness
Smart contract bugs in verifier	Proof-system soundness bugs

New combined risks:

- **Adversarial ZK proofs** — soundness errors in custom circuits.
- **Quantization attacks** — exploit the fp→fixed conversion to skew explanations.
- **Replay of stale model commitments.**
- **Sybil attacks on decentralized model voting.**

I.9 Open research questions

1. **Sub-second ZK proofs of small NN inference + SHAP** — currently minutes; gap is 100x.
2. **Faithful, provable LLM explanations** — combining mech-interp with ZK.
3. **Decentralized model training with verifiable convergence** — beyond FedAvg + Merkle commits.
4. **Privacy-preserving fairness audits** — prove fairness without revealing individual records.
5. **Soulbound reputation for model providers** with non-gameable scoring.
6. **Cross-chain AI oracles** — AI predictions consumed across multiple L1s.
7. **MEV-protected AI inference markets.**
8. **Compliance-grade audit trails** for EU AI Act, with ZK for trade-secret protection.
9. **On-chain explanation registries** that survive model versioning.
10. **Watermarking AI outputs** with blockchain-anchored provenance (deepfake defense).

I.10 Sample research project (Master's-scale)

Title: "Verifiable Feature Attribution for Black-Box Inference: A ZK-SHAP Prototype"

Plan:

1. Train a small (1–5M param) tabular model.
2. Implement KernelSHAP in a ZK-friendly form (fixed-point, batch evaluation).
3. Compile to EZKL / Risc Zero.
4. Generate proofs for ~10 test inputs; measure proof size, prover time, verifier gas.
5. Test soundness via adversarial model (Slack attack adapted) — does the proof catch it?
6. Write up as a workshop paper (target: AFT, IEEE Blockchain, BlackBoxNLP, or zkProof workshop).

This is a clean, novel intersection topic with a 3–6 month timeline.

I.11 Reading list

- Kang, Hashimoto, Stoica, Sun — *zkML and verifiable inference* (various 2023–2024 papers).
- Lee, Kang, Lee, Boneh — *EZKL and Halo2 ZK Inference*, 2023.
- Feng, Yan, Zhao, Lai, Lin — *Cerberus: ZK-SNARK-based Verifiable ML Inference*, 2021.
- Liu, Xie, Zhang — *zkCNN: Zero-Knowledge Proofs for Convolutional Neural Networks*, CCS 2021.
- Modulus Labs — *Cost of Intelligence* benchmark.
- Bittensor whitepaper.
- McMahan et al. — *Communication-Efficient Learning of Deep Networks from Decentralized Data (FedAvg)*, AISTATS 2017.

- Bonawitz et al. — *Practical Secure Aggregation for Privacy-Preserving ML*, CCS 2017.
- Karimi et al. — *Verifiable Federated Learning via ZK proofs*, various 2023.
- W3C — *DID Specification; Verifiable Credentials Data Model*.
- *EU AI Act* — Articles on high-risk systems and transparency.

I.12 Lab (5 hrs)

1. Install EZKL.
2. Train a 2-layer MLP on MNIST in PyTorch; export to ONNX.
3. Run `ezkl gen-settings, compile-circuit, setup, gen-witness, prove, verify`.
4. Time the proving step; measure proof size.
5. Compute a SHAP explanation off-chain; commit its hash on Sepolia.

I.13 Quiz

1. Why are ReLU and softmax expensive in ZK circuits?
2. State three trust models for on-chain ML.
3. How would you prove "this SHAP value was correctly computed" without revealing the model?
4. Why does Bittensor's Yuma consensus need a mechanism-design contribution to remain non-gameable?
5. Pick one of the 10 open research questions and write a 3-sentence research plan.

Books — The Permanent Shelf

Books worth owning (paper or PDF) for both fields. Listed by track, then priority.

Blockchain — must-own

1. **Bitcoin and Cryptocurrency Technologies** — Narayanan, Bonneau, Felten, Miller, Goldfeder. Free PDF. The most rigorous Bitcoin textbook ever written.
 2. **Mastering Bitcoin (2nd ed)** — Andreas Antonopoulos. Practical reference; chapters on UTXO, scripts, addresses are gold.
 3. **Mastering Ethereum** — Antonopoulos & Wood. Practical companion for EVM, Solidity, smart contract security.
 4. **A Graduate Course in Applied Cryptography** — Dan Boneh & Victor Shoup. *Free PDF*. The crypto reference. Read chapters 5, 12, 13, 18.
 5. **Introduction to Modern Cryptography (3rd ed)** — Katz & Lindell. The standard graduate textbook. Heavier on formal definitions.
 6. **Building Bitcoin in C** — Brandon Williams. (If you really want to feel the metal.)
 7. **Mastering Blockchain** — Imran Bashir. Broad survey including Hyperledger, Corda — useful for enterprise context.
 8. **DeFi and the Future of Finance** — Campbell Harvey et al. Lecture-book companion; concise.
 9. **Radical Markets** — Posner & Weyl. Quadratic voting, Harberger taxes — mechanism-design foundation for DAO research.
 10. **The Infinite Machine** — Camila Russo. Narrative history of Ethereum; great context for protocol decisions.
 11. **Programming Bitcoin** — Jimmy Song. Build a Bitcoin client from scratch in Python. Highly recommended hands-on.
 12. **The Art of Bitcoin** — Various — protocol-level deep dives via Bitcoin Optech newsletter archives.
-

XAI — must-own

1. **Interpretable Machine Learning** — Christoph Molnar. *Free online*. The canonical XAI textbook. Re-read it twice.
2. **Explanatory Model Analysis** — Biecek & Burzykowski. *Free online*. Tabular-focused; pairs well with Molnar.

3. **The Book of Why** — Judea Pearl & Dana Mackenzie. Causal-inference intuition for anyone touching counterfactuals.
 4. **Causal Inference in Statistics: A Primer** — Pearl, Glymour, Jewell. Formal but readable.
 5. **Causal Inference: What If** — Hernán & Robins. *Free PDF*. Epidemiologist's perspective; the standard.
 6. **Trustworthy Machine Learning** — Kush Varshney. *Free online*. Spans XAI, fairness, robustness, privacy.
 7. **Fairness and Machine Learning** — Barocas, Hardt, Narayanan. *Free online*. Definitive fairness textbook.
 8. **Pattern Recognition and Machine Learning** — Bishop. Bayesian baseline; needed to read XAI papers fluently.
 9. **Deep Learning** — Goodfellow, Bengio, Courville. *Free online*. Reference for the neural-net building blocks XAI methods explain.
 10. **Probabilistic Machine Learning: Advanced Topics** — Kevin Murphy. Bayesian XAI roots; influence functions, etc.
-

Cross-cutting

- **The Mythical Man-Month** — Brooks. Engineering wisdom.
 - **High Output Management** — Grove. For when you start collaborating.
 - **Designing Data-Intensive Applications** — Kleppmann. Distributed-systems intuition for blockchain.
 - **The Elements of Statistical Learning** — Hastie, Tibshirani, Friedman. *Free PDF*.
 - **An Introduction to Statistical Learning** — James, Witten, Hastie, Tibshirani. *Free PDF*. Friendlier.
-

Suggested reading order (12 weeks)

Week	Blockchain	XAI
1–2	Narayanan Ch 1–3	Molnar Ch 1–4
3–4	Narayanan Ch 4–7	Molnar Ch 5–7
5–6	Mastering Bitcoin Ch 5–8	Molnar Ch 8 (SHAP/LIME); Pearl: Book of Why
7–8	Mastering Ethereum Ch 6–9	Molnar Ch 9 (counterfactuals); Causal Primer
9–10	Boneh/Shoup Ch 5, 18	Hardt fairness textbook
11–12	Programming Bitcoin (project)	Distill Circuits + Anthropic write-ups

Top Web Courses — Curated, Ranked, with Notes

This is the synthesis we used to design this entire syllabus. If you want to **supplement** a particular module, jump to a course matching it.

Blockchain — top free/online courses

Foundational

Course	Provider	Why
Bitcoin and Cryptocurrency Technologies	Princeton (Coursera + book)	The single best deep dive into Bitcoin. Free textbook. Narayanan, Bonneau, Felten, Miller, Goldfeder.
MIT 15.S08 — Blockchain and Money	MIT OCW	Gary Gensler's lectures. Business + tech + policy.
CS251 — Cryptocurrencies and Blockchain Technologies	Stanford	Dan Boneh. Strongest crypto-focused syllabus.
Berkeley CS294-164 — Blockchain Technology	Berkeley	Research seminar; reading list is gold.
Coursera Blockchain Specialization	University at Buffalo	Hands-on Solidity + Truffle.

Smart contracts / Ethereum

Course	Provider	Why
CryptoZombies	Loom Network	Best Solidity onboarding game.
Patrick Collins — Smart Contract Developer (Cyfrin / Updraft)	YouTube + cyfrin.io	The de facto modern Solidity bootcamp. Free.
Alchemy University	Alchemy	Modules across Ethereum dev, security, ZK.
ConsenSys Academy Bootcamp	ConsenSys	Career-track, in-depth, paid.
Speedrun Ethereum	Scaffold-ETH (BuidlGuidl)	Project-based mastery; do all challenges.

Zero-Knowledge

Course	Provider	Why
ZK MOOC (CS251A)	Stanford / Berkeley (Boneh, Goldberg, Wahby)	Definitive ZK course online. zk-learning.org
0xPARC ZK Learning	0xPARC	Hands-on Circom, applications
Programming ZK Apps with Noir	Aztec	Modern, friendly
RareSkills ZK Bootcamp	RareSkills	Paid, well-regarded
Vitalik's Quadratic Arithmetic Programs from Zero	blog	Hand-derived ZK; brilliant for intuition

DeFi & advanced

Course	Provider	Why
a16z Crypto Startup School	a16z (free)	Founder-focused, breadth
Token Engineering Academy	TEA	Cryptoeconomics + mechanism design
WTF Academy	Wang Xiaofei	Best Mandarin + bilingual Solidity track
OpenZeppelin Learn	OpenZeppelin	Security-focused tutorials

XAI — top free/online courses

Foundational

Course	Provider	Why
Interpretable Machine Learning	Christoph Molnar (book, free online)	The canonical XAI textbook. Read cover-to-cover.
Kaggle — Machine Learning Explainability	Kaggle	4-hour micro-course; hands-on SHAP/PDP/permutation
DeepLearning.AI — AI Explainability short courses	DLAI	Modular; recent additions on LLM interp
Stanford CS329T — Trustworthy ML	Stanford	Includes XAI, fairness, robustness
Harvard CS282BR — Topics in Interpretability	Harvard	Research-style; readings are excellent
CMU 10-718 — Machine Learning in Practice	CMU	Practical XAI modules

Mechanistic interpretability (the hot frontier)

Course	Provider	Why
Neel Nanda — Mechanistic Interpretability Quickstart Guide	neelnanda.io	The single-best entry point. Free.
ARENA (Alignment Research Engineer Accelerator)	LessWrong / Apollo Research	6-week intensive; TransformerLens + IOI + SAEs
MATS (ML Alignment & Theory Scholars)	MATS Program	Selective; produces frontier researchers
Anthropic Transformer Circuits Thread	Anthropic	Living research blog; foundational reading
Distill.pub Circuits Thread	Distill (archived)	Visual classics — start here

Causal / counterfactual XAI

Course	Provider	Why
Causal Inference for The Brave and True	Matheus Facure (free book)	Best practical primer
Brady Neal — Introduction to Causal Inference	bradyneal.com	Video course
Pearl, Glymour, Jewell — Causal Inference in Statistics: A Primer	Wiley book	Definitive

Fairness / FAccT

Course	Provider	Why
Berkeley CS294 — Fairness in ML	Berkeley	Solid academic intro
FairLearn / AI Fairness 360 docs	Microsoft / IBM	Tool-grounded
fairmlbook.org	Barocas, Hardt, Narayanan	The textbook

Domain-specific XAI

Course	Provider	Why
MITx 6.S897 — ML for Healthcare	MIT	Includes clinical XAI sections
Coursera AI for Medicine Specialization	DLAI / Pranav Rajpurkar	Practical, vision-heavy
FAccT workshops & tutorials	annual	Best venue to learn FAccT-flavor XAI

Intersection courses

These either don't exist standalone or are very new:

- **ZK MOOC + Anthropic Interpretability** — assembling the intersection yourself is, today, table stakes.
- **EZKL workshops** (occasional, see ezkl.xyz).
- **Modulus Labs talks** (YouTube).
- **0xPARC ZKML reading group** archives.
- **a16z Crypto + AI track** — keep an eye on their content.

YouTube channels worth subscribing to

Blockchain

- *EatTheBlocks*
- *Smart Contract Programmer*
- *Patrick Collins / Cyfrin*
- *Whiteboard Crypto* (intro)
- *Justin Drake* (Ethereum protocol)
- *David Hoffman / Bankless* (industry)
- *Finematics* (DeFi)

XAI / interpretability

- *Yannic Kilcher* (paper walkthroughs)
- *AI Coffee Break with Letitia*
- *Neel Nanda*
- *Anthropic* (occasional talks)
- *Distill.pub archive* (text but cited heavily)
- *Christoph Molnar* (when he posts)

Twitter / X follow list (XAI + Blockchain)

- @vitalikbuterin, @drakefjustin, @sreeramkannan (blockchain protocol)
- @VitalikButerin's blog (vitalik.eth.limo) is essentially a free MS course
- @neelnanda5, @ch402 (Chris Olah), @AnthropicAI (interpretability)
- @CynthiaRudin, @christophmolnar (XAI)
- @hima_lakkaraju (XAI + HCI)
- @rajpurkar (medical AI XAI)
- @ezkl_xyz, @modulus_labs (ZKML)

Build a list, scan weekly.

Conferences to attend / livestream

- **DevConnect / DevCon** (Ethereum) — annual
- **EthCC, ETH Denver, ETHGlobal hackathons**

- **ZKProof Workshop** (annual)
- **AFT (Advances in Financial Technologies)**
- **IEEE S&P, USENIX Security, CCS**
- **NeurIPS, ICML, ICLR** — XAI workshops attached
- **FAccT** (ACM Fairness, Accountability, and Transparency)
- **BlackBoxNLP** (workshop at EMNLP/ACL)

--	--	--

--	--	--

Curated Paper List

Single-source-of-truth list of the most-cited and most-important papers across both tracks. Organized by topic.

Blockchain — foundational

- Nakamoto (2008) — *Bitcoin: A Peer-to-Peer Electronic Cash System*
- Lamport, Shostak, Pease (1982) — *The Byzantine Generals Problem*
- Fischer, Lynch, Paterson (1985) — *Impossibility of Distributed Consensus with One Faulty Process*
- Castro, Liskov (1999) — *Practical Byzantine Fault Tolerance*
- Buterin (2013) — *Ethereum White Paper*
- Wood (2014) — *Ethereum Yellow Paper*
- Garay, Kiayias, Leonardos (2015) — *The Bitcoin Backbone Protocol*

Consensus algorithms

- Eyal, Sirer (2014) — *Majority Is Not Enough: Bitcoin Mining Is Vulnerable*
- Heilman et al. (2015) — *Eclipse Attacks on Bitcoin's P2P Network*
- Sompolinsky, Zohar (2015) — *Secure High-Rate Transaction Processing in Bitcoin*
- Kiayias et al. (2017) — *Ouroboros*
- Chen, Micali (2017) — *Algorand*
- Yin et al. (2019) — *HotStuff*
- Buchman, Kwon, Milosevic — *Tendermint / The Latest Gossip on BFT Consensus*
- Buterin, Griffith (2017) — *Casper FFG*
- Miller et al. (2016) — *HoneyBadger BFT*
- Rocket et al. — *Avalanche / Snowman*
- Yakovenko (2018) — *Solana: Proof of History*
- Danezis et al. (2022) — *Narwhal & Tusk*
- Spiegelman et al. (2022) — *Bullshark*
- Olshansky et al. (2024) — *Mysticeti*

Scalability & rollups

- Poon, Dryja (2016) — *Lightning Network*
- Buterin (2021) — *An Incomplete Guide to Rollups* (blog)
- Al-Bassam, Sonnino, Buterin (2018) — *Fraud and Data Availability Proofs*
- Kalodner et al. (2018) — *Arbitrum*
- *EIP-4844 Proto-Danksharding*
- *Celestia whitepaper*
- *Polygon AggLayer documentation*
- *OP Stack technical specs*

Cryptography & ZK

- Goldwasser, Micali, Rackoff (1985) — *The Knowledge Complexity of Interactive Proof Systems*
- Groth (2016) — *On the Size of Pairing-Based NIZK Arguments*
- Ben-Sasson et al. (2018) — *Scalable, Transparent, Post-Quantum Computational Integrity (STARK)*
- Gabizon, Williamson, Ciobotaru (2019) — *PLONK*
- Bowe, Grigg, Hopwood (2019) — *Halo*
- Kothapalli, Setty, Tzialla (2022) — *Nova*
- Bünz et al. (2020) — *Recursive Proof Composition without a Trusted Setup*
- Boneh et al. — *KZG Polynomial Commitments* (talks + papers)

DeFi

- Adams et al. (2021) — *Uniswap v3 Core*
- Egorov (2019) — *StableSwap (Curve)*
- Daian et al. (2020) — *Flash Boys 2.0*
- Qin, Zhou, Gervais (2022) — *Quantifying Blockchain Extractable Value*
- Werner et al. (2022) — *SoK: Decentralized Finance*
- Klages-Mundt, Minca (2020) — *(In)stability for the Blockchain*
- Lo, Verma (2021) — *DeFi Protocols for Loanable Funds*

Security

- Atzei, Bartoletti, Cimoli (2017) — *Survey of Attacks on Ethereum Smart Contracts*
- Mehar et al. (2019) — *Understanding a Revolutionary and Flawed Grand Experiment: The DAO Attack*
- Rekt.news postmortem corpus (web)
- Trail of Bits, OpenZeppelin, ConsenSys Diligence — published audit reports
- Daian et al. (2020) — *Flash Boys 2.0*

Governance, tokenomics

- Buterin (2017) — *Notes on Blockchain Governance*
 - Posner, Weyl (2018) — *Radical Markets*
 - Buterin, Hitzig, Weyl (2019) — *A Flexible Design for Funding Public Goods*
 - Fritsch, Müller, Wattenhofer (2022) — *Analyzing Voting Power in Decentralized Governance*
-

XAI — foundational

- Lipton (2016) — *The Mythos of Model Interpretability*
- Doshi-Velez, Kim (2017) — *Towards a Rigorous Science of Interpretable ML*
- Miller (2019) — *Explanation in AI: Insights from the Social Sciences*
- Rudin (2019) — *Stop Explaining Black Box ML for High-Stakes Decisions*
- Adadi, Berrada (2018) — *Peeking Inside the Black-Box*
- Guidotti et al. (2018) — *A Survey of Methods for Explaining Black Box Models*

- Arrieta et al. (2020) — *Explainable AI (XAI): Concepts, Taxonomies, Opportunities*
- Rudin et al. (2022) — *Interpretable ML: Fundamental Principles and 10 Grand Challenges*

Feature attribution

- Ribeiro, Singh, Guestrin (2016) — *LIME*
- Lundberg, Lee (2017) — *SHAP*
- Lundberg et al. (2020) — *From Local to Global with TreeSHAP*
- Aas, Jullum, Løland (2021) — *SHAP with Dependent Features*
- Shrikumar, Greenside, Kundaje (2017) — *DeepLIFT*
- Bach et al. (2015) — *LRP*
- Sundararajan, Taly, Yan (2017) — *Integrated Gradients*
- Erion et al. (2021) — *Expected Gradients*
- Smilkov et al. (2017) — *SmoothGrad*
- Selvaraju et al. (2017) — *Grad-CAM*
- Petsiuk, Das, Saenko (2018) — *RISE*
- Ribeiro, Singh, Guestrin (2018) — *Anchors*
- Slack et al. (2020) — *Fooling LIME and SHAP*
- Koh, Liang (2017) — *Influence Functions*

Counterfactual & recourse

- Wachter, Mittelstadt, Russell (2017) — *Counterfactual Explanations Without Opening the Black Box*
- Mothilal, Sharma, Tan (2020) — *DiCE*
- Poyiadzi et al. (2020) — *FACE*
- Karimi et al. (2022) — *Survey of Algorithmic Recourse*
- Dhurandhar et al. (2018) — *Contrastive Explanations / Pertinent Positives & Negatives*
- Verma, Dickerson, Hines (2024) — *Counterfactual Explanations Review*

Concept-based

- Kim et al. (2018) — *TCAV*
- Ghorbani et al. (2019) — *ACE*
- Bau et al. (2017) — *Network Dissection*
- Koh et al. (2020) — *Concept Bottleneck Models*
- Chen et al. (2019) — *ProtoPNet (This Looks Like That)*
- Fel et al. (2023) — *CRAFT*

Mechanistic interpretability

- Olah et al. (2017–2020) — *Distill Circuits* thread
- Elhage et al. (2021) — *A Mathematical Framework for Transformer Circuits*
- Olsson et al. (2022) — *In-Context Learning and Induction Heads*
- Wang et al. (2022) — *Indirect Object Identification Circuit*
- Meng et al. (2022) — *ROME: Locating and Editing Factual Associations in GPT*
- Conmy et al. (2023) — *ACDC: Automated Circuit Discovery*
- Bricken et al. (2023) — *Towards Monosemanticity*

- Templeton et al. (2024) — *Scaling Monosemanticity*
- Cunningham et al. (2024) — *Sparse Autoencoders Find Interpretable Features*
- Marks et al. (2024) — *Sparse Feature Circuits*

Attention / transformer interpretability

- Jain, Wallace (2019) — *Attention Is Not Explanation*
- Wiegrefe, Pinter (2019) — *Attention Is Not Not Explanation*
- Abnar, Zuidema (2020) — *Quantifying Attention Flow*
- Chefer, Gur, Wolf (2021) — *Transformer Interpretability Beyond Attention Visualization*
- Hewitt, Liang (2019) — *Designing and Interpreting Probes with Control Tasks*
- Belrose et al. (2023) — *Tuned Lens*

Evaluation

- Adebayo et al. (2018) — *Sanity Checks for Saliency Maps*
- Hooker et al. (2019) — *ROAR*
- Yeh et al. (2019) — *(In)fidelity and Sensitivity of Explanations*
- DeYoung et al. (2020) — *ERASER*
- Krishna et al. (2022) — *Disagreement Problem*
- Hedström et al. (2023) — *Quantus*

HCI / user studies

- Bansal et al. (2021) — *Does the Whole Exceed Its Parts?*
- Poursabzi-Sangdeh et al. (2021) — *Manipulating and Measuring Model Interpretability*
- Vasconcelos et al. (2022) — *Explanations Can Reduce Overreliance*
- Bussone, Stumpf, O'Sullivan (2015) — *Role of Explanations on Trust in CDSS*
- Turpin et al. (2023) — *Language Models Don't Always Say What They Think*
- Hoffman et al. (2018) — *Metrics for Explainable AI*

Domain-specific

- Caruana et al. (2015) — *Intelligible Models for HealthCare (pneumonia)*
- DeGrave, Janizek, Lee (2021) — *Shortcuts in COVID-19 Radiograph Detection*
- Tonekaboni et al. (2019) — *What Clinicians Want*
- Chouldechova (2017) — *Fair Prediction with Disparate Impact*
- Kleinberg, Mullainathan, Raghavan (2016) — *Inherent Trade-Offs in Fair Risk Scores*

ZKML / Verifiable AI

- Liu, Xie, Zhang (2021) — *zkCNN*
- Lee et al. (2023) — *EZKL*
- Modulus Labs reports
- Kang et al. — *ZKML benchmarks (various)*
- Bonawitz et al. (2017) — *Secure Aggregation for Federated Learning*

Reading workflow recommendation

1. Read **one foundational** paper from each track per week.
2. Skim **three frontier** papers per week (read abstract, intro, conclusion, figures).
3. Maintain a **paper-card** for each (Zettelkasten style):
 - Problem
 - Method (one paragraph)
 - Result (key numbers)
 - Limitation
 - One follow-up question
4. After 3 months you will have ~50 cards — enough for a meaningful research-direction decision.

Tools & Libraries — A Practitioner's Stack

Pick one tool per row; the alternatives are listed in priority order.

Blockchain dev

Need	Tool
Local EVM dev	Foundry (anvil, forge, cast) > Hardhat
Bitcoin testing	bitcoind regtest
Block explorer / API	Etherscan, Beaconcha.in, Mempool.space
Indexing	The Graph, Subsquid, Goldsky
RPC	Alchemy, Infura, QuickNode, Public RPCs (Llama Nodes)
Wallet (dev)	Metamask, Rabby, Frame
Hardware wallet	Ledger, Trezor, GridPlus, Keystone
Multisig	Safe (formerly Gnosis Safe)
Solidity static analysis	Slither
Solidity fuzzing	Echidna, Foundry invariants
Symbolic execution	Mythril, Halmos
Formal verification	Certora, KEVM, Halmos
Subgraph dev	Graph CLI
L2 dev	op-stack, Arbitrum SDK, zkSync CLI, Scroll, StarkNet CLI
MEV research	Flashbots mev-inspect, Libmev

ZK tools

Need	Tool
Circuit DSL (intermediate)	Circom
Circuit DSL (modern)	Noir (Aztec)
STARK-native circuits	Cairo (StarkWare)
Halo2 circuits	Halo2 (Zcash)

ZK virtual machines	Risc Zero, SP1 (Succinct), Jolt, Nexus, Zeth
ML inference proofs	EZKL, Modulus, Giza, DDKang/zkml
Recursion / folding	Nova, HyperNova, ProtoStar implementations
Trusted setup	Powers of Tau (Ethereum KZG ceremony reuse)

ML / XAI libraries

Need	Tool
Deep learning	PyTorch (primary), JAX, TensorFlow
Tabular ML	XGBoost, LightGBM, CatBoost, scikit-learn
Interpretable tabular	InterpretML (EBM), pyGAM, Linear/Ridge
Feature attribution	SHAP, LIME, Captum (PyTorch), tf-explain
Counterfactuals	DiCE, AlibiExplain, CARLA, OmniXAI
Concept-based	TCAV (Google), CRAFT (Fel)
Saliency for CV	Captum, Grad-CAM++, pytorch-grad-cam
LLM interpretability	TransformerLens, nnsight, SAELens, Neuronpedia
Probing	DiscoLib, custom
Evaluation	Quantus, OpenXAI, ERASER datasets
Fairness	Fairlearn, AIF360, Aequitas
Robustness	CleverHans, Foolbox, AutoAttack

Datasets to know

Tabular XAI staples

Dataset	Use
UCI Adult / German Credit	Tabular fairness + counterfactuals
California Housing	Regression baseline
FICO HELOC	Interpretable credit modeling
COMPAS	Recidivism / fairness debates

Vision

Dataset	Use
ImageNet	Classic CNN attribution
CIFAR-10/100	Quick saliency experiments
Broden	Network Dissection concepts
Caltech-UCSD Birds 200	ProtoPNet baseline

NLP

Dataset	Use
IMDB	Sentiment baseline
SNLI	Reasoning
ERASER suite	Rationale benchmarks
TruthfulQA	Hallucination

Medical

Dataset	Use
MIMIC-III/IV	Clinical EHR
CheXpert / NIH ChestX-ray14	Radiology
HAM10000	Dermatology

Blockchain data

Dataset / API	Use
Dune Analytics	SQL on chain data
Flipside Crypto	Analytics
Etherscan API	Address-level
DefiLlama API	TVL, protocol stats
Allium, Footprint	Enterprise datasets
Token Terminal	Revenue / metrics
Artemis	Cross-chain metrics

Notebooks & monitoring

Need	Tool
Experiment tracking	Weights & Biases, MLflow, Aim
Hyperparameter search	Optuna, Ray Tune
Pipeline orchestration	Prefect, Airflow, DVC
Notebook reproducibility	Papermill, JupyterText
Paper management	Zotero, Obsidian, Roam

"If I had to pick five"

If you can only learn five tools, pick:

- 1. **PyTorch** — the deep-learning lingua franca.
- 2. **SHAP** — most-used XAI library.
- 3. **TransformerLens** — modern interpretability work depends on it.
- 4. **Foundry** — the Solidity dev kit.
- 5. **EZKL** — entry point to ZKML.

Master these five and you can read 90% of the field's papers and reproduce most experiments.

Hands-On Labs

Six structured labs that take you from beginner to research-ready. Each lab targets ~4–8 hours of work and has a concrete deliverable.

#	Title	Tracks	Time	Deliverable
1	Build a toy blockchain in Python	Blockchain	4 hrs	Working 5-block chain with Merkle proofs
2	Reentrancy attack reproduction	Blockchain / security	3 hrs	Foundry exploit + fix
3	Your first ZK circuit	Blockchain / ZK	4 hrs	Verified Sepolia tx
4	SHAP from scratch	XAI	4 hrs	KernelSHAP implementation + validation
5	Saliency methods + sanity checks	XAI	4 hrs	Table comparing 4 methods
6	zkML prototype	Intersection	8 hrs	EZKL proof of small NN inference

See `lab_1.md` ... `lab_6.md` for full instructions.

Lab philosophy

Each lab follows the **plan** → **build** → **verify** → **write** loop:

1. **Plan** — write down what you'll do in 5 lines before opening an editor.
2. **Build** — minimum viable code.
3. **Verify** — explicit test or assertion that the lab worked.
4. **Write** — a half-page journal entry: what surprised you, what you'd improve.

The writing step is where you build the research intuition. Don't skip it.

Lab 1 — Build a Toy Blockchain in Python

Goal: implement, from scratch, the smallest blockchain that demonstrates hashing, signing, Merkle roots, and tamper detection.

Time: ~4 hours.

Prereqs: Python 3.10+, hashlib, ecdsa (pip install ecdsa).

Step 1 — Wallet

```
from ecdsa import SigningKey, SECP256k1
import hashlib, json

class Wallet:
    def __init__(self):
        self.sk = SigningKey.generate(curve=SECP256k1)
        self.vk = self.sk.get_verifying_key()
        self.pubkey = self.vk.to_string().hex()
        self.address = hashlib.sha256(self.pubkey.encode()).hexdigest()[:40]

    def sign(self, msg: bytes) -> bytes:
        return self.sk.sign(msg)

    def verify(self, msg: bytes, sig: bytes) -> bool:
        try:
            return self.vk.verify(sig, msg)
        except: return False
```

Step 2 — Transaction

```
class Transaction:
    def __init__(self, sender_pub, recipient_addr, amount, signature=None):
        self.sender_pub = sender_pub
        self.recipient_addr = recipient_addr
        self.amount = amount
        self.signature = signature

    def payload(self) -> bytes:
        return json.dumps({
            "from": self.sender_pub, "to": self.recipient_addr, "amt": self.amount
        }, sort_keys=True).encode()

    def is_valid(self) -> bool:
        from ecdsa import VerifyingKey
        vk = VerifyingKey.from_string(bytes.fromhex(self.sender_pub), curve=SECP256k1)
        try: return vk.verify(self.signature, self.payload())
        except: return False
```

Step 3 — Merkle root

```
def merkle_root(tx_hashes):
    if not tx_hashes: return b"\0" * 32
    layer = tx_hashes[:]
    while len(layer) > 1:
        if len(layer) % 2: layer.append(layer[-1])
        layer = [hashlib.sha256(layer[i] + layer[i+1]).digest()
                  for i in range(0, len(layer), 2)]
    return layer[0]
```

Step 4 — Block

```
import time
class Block:
    def __init__(self, prev_hash, txs, nonce=0):
        self.prev_hash = prev_hash
        self.txs = txs
        self.timestamp = int(time.time())
        self.nonce = nonce
        tx_hashes = [hashlib.sha256(tx.payload()).digest() for tx in txs]
        self.merkle = merkle_root(tx_hashes)

    def header(self) -> bytes:
        return self.prev_hash + self.merkle + self.timestamp.to_bytes(8, "big") + self.nonce.to_bytes(4, "big")

    def hash(self) -> bytes:
        return hashlib.sha256(self.header()).digest()
```

Step 5 — Chain + verification

```
class Chain:
    def __init__(self):
        self.blocks = [Block(b"\0"*32, [])] # genesis

    def add(self, txs):
        prev = self.blocks[-1].hash()
        block = Block(prev, txs)
        self.blocks.append(block)

    def verify(self) -> bool:
        for i in range(1, len(self.blocks)):
            if self.blocks[i].prev_hash != self.blocks[i-1].hash():
                return False
            for tx in self.blocks[i].txs:
                if not tx.is_valid(): return False
        return True
```

Step 6 — Tamper detection demo

```
alice, bob = Wallet(), Wallet()
chain = Chain()
for i in range(5):
    tx = Transaction(alice.pubkey, bob.address, i)
    tx.signature = alice.sign(tx.payload())
    chain.add([tx])

assert chain.verify()
print("chain valid")

# tamper:
chain.blocks[2].txs[0].amount = 999
print("after tamper:", chain.verify()) # False
```

Deliverable

- `toy_chain.py` — ~150 lines.
- A printed demo showing valid chain → tampering → invalid chain.
- A 1-paragraph reflection: which property does this demonstrate (integrity / authentication / tamper-evidence)?

Stretch

- Add Proof-of-Work: block header must hash below a target.
- Add longest-chain rule for fork resolution.
- Add a tiny P2P layer with `asyncio`.

Lab 2 — Reentrancy Attack Reproduction

Goal: write a vulnerable Solidity contract, exploit it with an attacker contract, then fix using the checks-effects-interactions pattern.

Time: ~3 hours.

Prereqs: install Foundry (`curl -L https://foundry.paradigm.xyz | bash; foundryup`).

Setup

```
forge init reentrancy-lab
cd reentrancy-lab
```

Step 1 — Vulnerable bank

src/Bank.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VulnerableBank {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 bal = balances[msg.sender];
        require(bal > 0, "no balance");
        (bool ok,) = msg.sender.call{value: bal}("");
        require(ok, "transfer failed");
        balances[msg.sender] = 0; // updated AFTER call ← vulnerability
    }

    receive() external payable {}
}
```

Step 2 — Attacker

src/Attacker.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```

import "./Bank.sol";

contract Attacker {
    VulnerableBank public bank;
    address public owner;

    constructor(address _bank) {
        bank = VulnerableBank(payable(_bank));
        owner = msg.sender;
    }

    function attack() external payable {
        require(msg.value >= 1 ether, "need 1 ETH");
        bank.deposit{value: 1 ether}();
        bank.withdraw();
    }

    receive() external payable {
        if (address(bank).balance >= 1 ether) {
            bank.withdraw();
        }
    }

    function collect() external {
        require(msg.sender == owner);
        payable(owner).transfer(address(this).balance);
    }
}

```

Step 3 — Test the attack

test/Reentrancy.t.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/Bank.sol";
import "../src/Attacker.sol";

contract ReentrancyTest is Test {
    VulnerableBank bank;
    Attacker attacker;
    address alice = address(0xA);
    address bob = address(0xB);

    function setUp() public {
        bank = new VulnerableBank();
        vm.deal(alice, 5 ether);
        vm.deal(bob, 1 ether);

        vm.prank(alice);
        bank.deposit{value: 5 ether}();

        vm.prank(bob);
        attacker = new Attacker(address(bank));
    }
}

```

```

    }

    function testAttackDrainsBank() public {
        assertEq(address(bank).balance, 5 ether);
        vm.prank(bob);
        attacker.attack{value: 1 ether}();
        // Bank should be drained; attacker should hold ~6 ETH
        assertEq(address(bank).balance, 0);
        assertGe(address(attacker).balance, 5 ether);
    }
}

```

Run:

```
forge test -vvv
```

You should see the attacker drain Alice's 5 ETH + their own deposit.

Step 4 — Fix with checks-effects-interactions

```

function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0, "no balance");
    balances[msg.sender] = 0;           // effect FIRST
    (bool ok,) = msg.sender.call{value: bal}("");
    require(ok, "transfer failed");
}

```

Re-run tests. The drain should fail. Add a passing test that proves the bank is no longer drainable:

```

function testAttackFails() public {
    vm.prank(bob);
    vm.expectRevert();
    attacker.attack{value: 1 ether}();
    assertEq(address(bank).balance, 5 ether);
}

```

Step 5 — Alternative fix: ReentrancyGuard

Use OpenZeppelin's mutex:

```

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    // ...
    function withdraw() external nonReentrant { ... }
}

```

Discuss in your journal:

- Which fix is cheaper in gas?
 - Which fix protects against **cross-function** reentrancy?
-

Deliverable

- `src/Bank.sol`, `src/Attacker.sol`, `test/Reentrancy.t.sol` (both attack-succeeds and attack-fails tests).
- A 1-page postmortem explaining the bug class and listing 2 other real-world incidents where it appeared (e.g., DAO 2016, Curve July 2023, dForce, Cream).

Stretch

- Modify the bank to use `transfer()` (which forwards 2300 gas). Does it still get drained? Discuss why `transfer()` is no longer recommended (post-Istanbul gas changes).
- Add an invariant test: `bank balance >= sum(balances)`. Run `forge test --invariant`.

Lab 3 — Your First ZK Circuit

Goal: write, compile, and verify a Circom-based zk-SNARK that proves "I know x such that $x * x = 25$ " without revealing x . Then deploy the verifier to Sepolia.

Time: ~4 hours.

Prereqs:

```
npm i -g circom snarkjs
```

You'll also need Node 18+, a funded Sepolia address (~0.05 ETH), and a wallet (Metamask or Foundry's cast).

Step 1 — The circuit

square.circom:

```
pragma circom 2.1.4;

template Square() {
    signal input x;          // private witness
    signal input y;          // public input
    signal product;

    product <== x * x;
    y === product;
}

component main { public [y] } = Square();
```

Compile:

```
circom square.circom --rlcs --wasm --sym
```

Step 2 — Trusted setup (Powers of Tau + Groth16)

```
snarkjs powersoftau new bn128 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau --name="me" -v
snarkjs powersoftau prepare phase2 pot12_0001.ptau pot12_final.ptau -v
snarkjs groth16 setup square.rlcs pot12_final.ptau square_0000.zkey
snarkjs zkey contribute square_0000.zkey square_final.zkey --name="me" -v
snarkjs zkey export verificationkey square_final.zkey verification_key.json
```


Step 3 — Generate witness + proof

input.json:

```
{ "x": 5, "y": 25 }
```

```
node square_js/generate_witness.js square_js/square.wasm input.json witness.wtns
snarkjs groth16 prove square_final.zkey witness.wtns proof.json public.json
snarkjs groth16 verify verification_key.json public.json proof.json
```

You should see OK!.

Now try with { "x": -5, "y": 25 } (sign flips in the field). Still verifies — that's the zero-knowledge point: from $y=25$, the verifier can't tell whether $x=5$ or $x=-5$.

Step 4 — Solidity verifier

```
snarkjs zkey export solidityverifier square_final.zkey Verifier.sol
snarkjs zkey export soliditycalldata public.json proof.json
```

Copy the resulting calldata. Deploy Verifier.sol to Sepolia via Foundry:

```
forge create Verifier.sol:Groth16Verifier --rpc-url $SEPOLIA_RPC --private-key $PK
```

Call verifyProof() via cast:

```
cast call $VERIFIER "verifyProof(uint256[2],uint256[2][2],uint256[2],uint256[1]) (bool) "
    $PI_A_HEX $PI_B_HEX $PI_C_HEX $PUBLIC_INPUTS --rpc-url $SEPOLIA_RPC
```

Returns true.

Step 5 — Reflect

In your journal:

- Why does this construction reveal y but not x ?
- How would you extend it to prove "I know (p, q) such that $p \cdot q = n$ without revealing the factors"? (Hint: factoring is a one-way function — this is the kind of statement RSA-based ZK proofs use.)
- What does it cost in Sepolia gas to verify?

Deliverable

- square.circom, proof.json, public.json, Verifier.sol.
- Sepolia tx hash of the successful verifyProof() call.

- 1-page journal entry.

Stretch

1. Replace Groth16 with PLONK:

```
snarkjs plonk setup square.r1cs pot12_final.ptau square_plonk.zkey
snarkjs plonk prove square_plonk.zkey witness.wtns proof.json public.json
```

Compare proof sizes and verifier gas.

1. Build a Merkle-inclusion circuit using `circomlib`'s Poseidon hash. Prove that "I know a leaf and its path that produces this Merkle root" — the building block of Tornado Cash, zk identity, and rollups.

Lab 4 — Implement SHAP from Scratch

Goal: build a working KernelSHAP in ~80 lines, validate it against the `shap` library, and explore where it disagrees with LIME.

Time: ~4 hours.

Prereqs: `pip install numpy scikit-learn xgboost shap lime matplotlib`.

Step 1 — Train a model

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import xgboost as xgb

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)

clf = xgb.XGBClassifier(n_estimators=200, max_depth=4)
clf.fit(Xtr, ytr)
print("test acc:", clf.score(Xte, yte))
```

Step 2 — KernelSHAP from scratch

```
import numpy as np
from itertools import combinations
from math import comb

def kernel_weight(M, s):
    if s == 0 or s == M: return 1e9
    return (M - 1) / (comb(M, s) * s * (M - s))

def kernel_shap(predict_fn, x, background, num_samples=2000, seed=0):
    """
    predict_fn:  function taking 2D array -> 1D probas
    x:           single instance (1D array of length M)
    background:  reference data (2D array)
    """
    rng = np.random.default_rng(seed)
    M = len(x)
    base = predict_fn(background).mean()

    Z = []    # coalition masks
    f = []    # model output on masked instance
    w = []    # kernel weights

    for _ in range(num_samples):
        # sample a binary mask
        z = rng.integers(0, 2, M)
```

```

        # build masked input: features in coalition = x, else = mean of background
        x_in = np.where(z, x, background.mean(0))
        Z.append(z)
        f.append(predict_fn(x_in.reshape(1, -1))[0])
        w.append(kernel_weight(M, z.sum()))

Z = np.array(Z)
f = np.array(f) - base    # center
w = np.array(w)

# Weighted linear regression:  $f \approx Z @ \phi$  (no intercept; centered)
W = np.diag(w)
A = Z.T @ W @ Z
b = Z.T @ W @ f
phi = np.linalg.lstsq(A, b, rcond=None)[0]
return phi, base

```

Step 3 — Validate against the SHAP library

```

import shap

x = Xte.iloc[0].values
background = Xtr.values[:100]
predict = lambda X: clf.predict_proba(X)[:, 1]

phi_mine, base_mine = kernel_shap(predict, x, background, num_samples=4000)

explainer = shap.KernelExplainer(predict, background)
phi_shap = explainer.shap_values(x, nsamples=4000)

# Compare
import pandas as pd
cmp = pd.DataFrame({
    "feature": Xte.columns,
    "mine": phi_mine,
    "shap": phi_shap,
    "diff": phi_mine - phi_shap
}).sort_values("shap", key=abs, ascending=False).head(10)
print(cmp)

```

The two columns should match to within ~10% on most features. Larger samples → tighter match.

Step 4 — Compare with LIME

```

from lime.lime_tabular import LimeTabularExplainer

lime_exp = LimeTabularExplainer(
    Xtr.values, feature_names=Xtr.columns.tolist(),
    class_names=["malignant", "benign"], mode="classification")

exp = lime_exp.explain_instance(x, clf.predict_proba, num_features=10)
for f, w in exp.as_list():
    print(f, round(w, 4))

```

Compare LIME's top features with SHAP's. They often disagree; that's the **disagreement problem**.

Step 5 — Visualize

```
import matplotlib.pyplot as plt

idx = np.argsort(np.abs(phi_mine))[:, :-1][:10]
plt.barh(range(10), phi_mine[idx][:, :-1])
plt.yticks(range(10), [Xte.columns[i] for i in idx][:, :-1])
plt.title("My SHAP values for one prediction")
plt.tight_layout()
plt.savefig("my_shap.png")
```

Deliverable

- `kernel_shap.py` (~80 lines).
- A table comparing your top-10 features to the SHAP library + LIME.
- A 1-paragraph reflection on (a) why your KernelSHAP isn't exact, (b) where you observed disagreement with LIME.

Stretch

- Implement **TreeSHAP** (much harder; consult the original paper).
- Implement **DeepSHAP** for a small neural net on the same data.
- Compute the **Shapley interaction index** for the top-2 features.
- Apply your KernelSHAP to a tiny PyTorch model and compare against `shap.GradientExplainer`.

Lab 5 — Saliency Methods + Sanity Checks

Goal: implement vanilla saliency, SmoothGrad, Integrated Gradients, and Grad-CAM on a pretrained ResNet-50. Then run Adebayo's sanity checks. Discover which methods are faithful and which are decorative.

Time: ~4 hours.

Prereqs:

```
pip install torch torchvision matplotlib pillow opencv-python
```

Step 1 — Setup

```
import torch, torchvision
from torchvision.models import resnet50, ResNet50_Weights
from PIL import Image
from torchvision import transforms
import numpy as np, matplotlib.pyplot as plt

weights = ResNet50_Weights.IMAGENET1K_V2
model = resnet50(weights=weights).eval()
preprocess = weights.transforms()

img = Image.open("cat.jpg").convert("RGB")
x = preprocess(img).unsqueeze(0)
logits = model(x)
pred = logits.argmax(1).item()
print("predicted:", weights.meta["categories"][pred])
```

(Any ImageNet-class image works.)

Step 2 — Vanilla saliency

```
def saliency(model, x, class_idx):
    x = x.clone().requires_grad_(True)
    out = model(x)
    out[0, class_idx].backward()
    return x.grad.abs().max(1)[0][0].numpy() # H x W
```

Step 3 — SmoothGrad

```
def smoothgrad(model, x, class_idx, n=30, sigma=0.15):
    grads = []
```

```

for _ in range(n):
    noise = torch.randn_like(x) * sigma
    x_noisy = (x + noise).clone().requires_grad_(True)
    out = model(x_noisy)
    out[0, class_idx].backward()
    grads.append(x_noisy.grad.abs().max(1)[0][0].numpy())
return np.mean(grads, axis=0)

```

Step 4 — Integrated Gradients

```

def integrated_gradients(model, x, class_idx, steps=50, baseline=None):
    if baseline is None: baseline = torch.zeros_like(x)
    alphas = torch.linspace(0, 1, steps).view(-1, 1, 1, 1)
    interps = baseline + alphas * (x - baseline) # steps x C x H x W
    interps.requires_grad_(True)
    out = model(interps)
    g = torch.autograd.grad(out[:, class_idx].sum(), interps)[0] # steps x C x H x W
    avg = g.mean(0, keepdim=True)
    ig = (x - baseline) * avg
    return ig.abs().max(1)[0][0].numpy()

```

Step 5 — Grad-CAM

```

def grad_cam(model, x, class_idx, target_layer):
    features, grads = [], []
    def fhook(_m, _i, o): features.append(o)
    def bhook(_m, _gi, go): grads.append(go[0])
    fh = target_layer.register_forward_hook(fhook)
    bh = target_layer.register_full_backward_hook(bhook)

    out = model(x)
    model.zero_grad()
    out[0, class_idx].backward()
    fh.remove(); bh.remove()

    A = features[0][0] # C x h x w
    grad = grads[0][0] # C x h x w
    weights = grad.mean(dim=(1, 2)) # C
    cam = torch.relu((weights[:, None, None] * A).sum(0)).cpu().numpy()
    cam = cam / (cam.max() + 1e-9)
    return cam

# Target layer for ResNet-50: layer4
cam = grad_cam(model, x, pred, model.layer4[-1])

```

Step 6 — Visualize all four

```

maps = {
    "saliency": saliency(model, x, pred),
    "smoothgrad": smoothgrad(model, x, pred),

```

```

    "ig": integrated_gradients(model, x, pred),
    "gradcam": cam,
}
fig, axs = plt.subplots(1, 5, figsize=(20, 4))
axs[0].imshow(img); axs[0].set_title("input")
for ax, (name, m) in zip(axs[1:], maps.items()):
    ax.imshow(m, cmap="hot"); ax.set_title(name)
plt.savefig("saliency_compare.png")

```

Step 7 — Adebayo sanity check: model randomization

```

def randomize_top_layers(model, layers_to_randomize):
    """Re-initialize the named layers."""
    import copy
    m = copy.deepcopy(model)
    for n, p in m.named_parameters():
        if any(layer in n for layer in layers_to_randomize):
            torch.nn.init.normal_(p) if p.dim() > 1 else torch.nn.init.zeros_(p)
    return m.eval()

m_rand = randomize_top_layers(model, ["fc", "layer4"])

cam_rand = grad_cam(m_rand, x, pred, m_rand.layer4[-1])
ig_rand = integrated_gradients(m_rand, x, pred)

# Plot side-by-side: original vs randomized
# A faithful method should produce very different maps.

```

Compute the **structural similarity (SSIM)** between the original and randomized maps:

```

from skimage.metrics import structural_similarity as ssim
print("Grad-CAM SSIM:", ssim(maps["gradcam"], cam_rand, data_range=cam_rand.max()-cam_ra
print("IG SSIM:", ssim(maps["ig"], ig_rand, data_range=ig_rand.max()-ig_rand.min()))

```

Higher SSIM = less change = less faithful. (Adebayo et al. report Guided BP / Guided Grad-CAM as high-SSIM = failing.)

Step 8 — Build the comparison table

Method	Visual quality	SSIM (lower=better)	Notes
Vanilla saliency	noisy	low	passes
SmoothGrad	cleaner	low	passes
Integrated Gradients	cleanest	low	passes axiomatically
Grad-CAM	smooth, low-res	medium	passes class-discriminative test

(Numbers will depend on your specific image.)

Deliverable

- `saliency_methods.py` (~150 lines).
- `saliency_compare.png` and `saliency_sanity.png`.
- A markdown table summarizing visual quality, SSIM, and your judgment of faithfulness.

Stretch

- Implement **Guided Backprop** and **Guided Grad-CAM**. Run the sanity check. Confirm they fail.
- Add a **data randomization** test: train a small CNN on permuted CIFAR-10 labels, compare saliency to the correctly-trained model.
- Use the `quantus` library to compute faithfulness, sensitivity, and stability metrics.

--	--	--	--

Lab 6 — zkML Prototype with EZKL

Goal: prove (in zero-knowledge) that a small neural network's MNIST inference was performed correctly. Then commit the explanation hash on-chain.

Time: ~8 hours (most of it is one-time setup and proving wait).

Prereqs:

```
pip install torch torchvision ezkl onnx onnxruntime
```

EZKL must be installed natively:

```
curl https://raw.githubusercontent.com/zkonduit/ezkl/main/install_ezkl_cli.sh | bash
```

Step 1 — Train a tiny MLP

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

tx = transforms.Compose([transforms.ToTensor(), transforms.Lambda(lambda x: x.view(-1))])
tr = DataLoader(datasets.MNIST(".", train=True, download=True, transform=tx), batch_size=100)

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 32)
        self.fc2 = nn.Linear(32, 10)
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

net = Net()
opt = torch.optim.Adam(net.parameters(), 1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(2):
    for x, y in tr:
        opt.zero_grad()
        loss = loss_fn(net(x), y)
        loss.backward()
        opt.step()

torch.save(net.state_dict(), "net.pt")
```

Step 2 — Export to ONNX

```
sample = torch.randn(1, 784)
torch.onnx.export(net, sample, "net.onnx",
                  input_names=["input"], output_names=["output"],
                  dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}},
                  opset_version=13)
```

Verify with onnxruntime:

```
import onnxruntime as ort
sess = ort.InferenceSession("net.onnx")
print(sess.run(None, {"input": sample.numpy()})[0])
```

Step 3 — EZKL pipeline

Create `input.json` with one MNIST sample (flattened, normalized to [0,1]):

```
import json
from torchvision import datasets
dset = datasets.MNIST(".", train=False, download=True, transform=transforms.ToTensor())
x, y = dset[0]
x_flat = x.view(-1).tolist()
json.dump({"input_data": [x_flat]}, open("input.json", "w"))
```

Then run EZKL:

```
ezkl gen-settings -M net.onnx -O settings.json
ezkl calibrate-settings -M net.onnx -D input.json -O settings.json --target=resources
ezkl compile-circuit -M net.onnx --compiled-circuit net.compiled -S settings.json
ezkl get-srs -S settings.json
ezkl setup -M net.compiled --vk-path vk.key --pk-path pk.key
ezkl gen-witness -D input.json -M net.compiled --witness witness.json
ezkl prove --witness witness.json -M net.compiled --pk-path pk.key --proof-path proof.js
ezkl verify --proof-path proof.json --vk-path vk.key -S settings.json
```

Last line should print `verified: true`.

Step 4 — Measure

```
ls -lh proof.json      # proof size (KB)
time ezkl prove ...    # prover time
```

Typical for this tiny network: a few hundred KB proof, tens of seconds prover time on a laptop.

Step 5 — On-chain verifier

Generate Solidity verifier:

```
ezkl create-evm-verifier -S settings.json --vk-path vk.key --sol-code-path Verifier.sol
```

Deploy with Foundry:

```
forge create Verifier.sol:Halo2Verifier --rpc-url $SEPOLIA_RPC --private-key $PK
```

Call `verifyProof()` with the proof bytes. Record the gas cost.

Step 6 — Commit explanation hash

Compute SHAP for the same input off-chain:

```
import shap, torch, numpy as np
background = torch.randn(50, 784)
explainer = shap.DeepExplainer(net, background)
phi = explainer.shap_values(torch.tensor(x_flat).unsqueeze(0))
```

Hash the explanation:

```
import hashlib
phi_bytes = np.array(phi).tobytes()
explanation_hash = hashlib.sha256(phi_bytes).hexdigest()
```

Commit on Sepolia in a one-off contract:

```
contract Registry {
    event Committed(bytes32 inputHash, bytes32 modelHash, bytes32 explanationHash);
    function commit(bytes32 i, bytes32 m, bytes32 e) external {
        emit Committed(i, m, e);
    }
}
```

Anyone can later verify (off-chain) that a published SHAP matches the committed hash.

Step 7 — Reflect

In your journal:

- Time and gas costs at each step.
- What would change to scale to ResNet-50? (Hint: orders of magnitude more constraints.)
- What's the trust model? Who verifies what?
- What's missing for **provable** explanations (vs only provable inference)?

Deliverable

- `net.pt`, `net.onnx`, `settings.json`, `proof.json`, `Verifier.sol`.
- Sepolia tx hash of the successful verification.
- Sepolia tx hash of the `Registry.commit()` call.
- 2-page write-up.

Stretch

- Use **Risc Zero** instead of EZKL: write the inference as Rust + tract, generate STARK.
- Implement a tiny SHAP computation inside the zk circuit (very hard; this is open research).
- Benchmark proving time as you scale the MLP hidden dim from 32 → 128 → 512.
- Add input privacy: hide the MNIST image, prove only the *class*.